2 Le istruzioni: il linguaggio dei calcolatori

Parlo spagnolo con Dio, italiano con le donne, francese con gli uomini e tedesco col mio cavallo.

Carlo V, *imperatore del Sacro Romano Impero,* 1500-1558

2.1 Introduzione

Per impartire comandi all'hardware di un calcolatore occorre saper parlare il suo linguaggio. Le parole del linguaggio del calcolatore sono dette istruzioni e l'intero vocabolario viene chiamato **insieme delle istruzioni** (*instruction set*). In questo capitolo verrà descritto l'insieme delle istruzioni di un calcolatore reale, sia nella forma scritta dall'essere umano sia in quella interpretata dal calcolatore. Le istruzioni verranno presentate "dall'alto al basso": partendo dalla notazione tipica di un linguaggio di programmazione ad alto livello, anche se limitato, le trasformeremo passo dopo passo fino a rivelare il vero linguaggio di un calcolatore reale.

Insieme delle istruzioni: il vocabolario dei comandi compresi da una data architettura.

Nel Capitolo 3 il nostro viaggio continuerà verso il basso per scoprire come è fatto l'hardware utilizzato per i calcoli e come vengono rappresentati i numeri in virgola mobile.

Si potrebbe pensare che i linguaggi dei calcolatori siano molti e variegati come quelli degli esseri umani. In realtà sono abbastanza simili tra loro: sono più affini a dialetti regionali derivanti da un'unica radice linguistica che a vere e proprie lingue differenti. Perciò, una volta appreso un linguaggio, è molto semplice imparare anche gli altri.

L'insieme di istruzioni prescelto è il **RISC-V**, che è stato sviluppato all'Università di Berkeley a partire dal 2010.

Per mostrare come sia facile imparare anche ulteriori insiemi di istruzioni, ne illustreremo altri due molto diffusi.

1. Il **MIPS** è un esempio elegante di insieme di istruzioni progettato a partire dal 1980. Sotto molti aspetti, il RISC-V è molto simile.
2. L'**Intel x86** è nato nel 1970, ma alimenta ancora oggi sia i PC sia i calcolatori dei cloud dell'era post-PC.

Questa somiglianza tra insiemi di istruzioni è dovuta al fatto che tutti i calcolatori sono costruiti a partire da tecnologie hardware basate sugli stessi principi fondamentali e al fatto che esistono alcune operazioni di base che ogni calcolatore deve poter eseguire. Inoltre, i progettisti dei calcolatori hanno un obiettivo comune: trovare un linguaggio che renda semplice costruire l'hardware e realizzare i compilatori e, allo stesso tempo, consenta di massimizzare le prestazioni e minimizzare costi ed energia assorbita.

Quest'obiettivo è rimasto valido nel tempo; la seguente citazione fu scritta ben prima che voi poteste acquistare un calcolatore ed è vera oggi come lo era nel 1946:

> *È facile verificare tramite la logica formale che esistono certi [insiemi di istruzioni] che in linea di principio si prestano bene a controllare l'hardware per eseguire una qualsiasi sequenza di operazioni [...] Le considerazioni davvero decisive, dal punto di vista attuale, per la scelta di un insieme di istruzioni sono di natura molto più pratica: la semplicità dei dispositivi richiesti [dall'insieme delle istruzioni], la chiarezza della sua applicazione alla soluzione di problemi realmente importanti e la velocità con cui questi problemi vengono gestiti.*
>
> Burks, Goldstine e von Neuman, 1946

La "semplicità dei dispositivi" viene considerata un parametro di grande valore per i calcolatori moderni, come lo era per quelli degli anni '40. L'obiettivo del presente capitolo è mostrare un insieme di istruzioni progettato seguendo queste indicazioni: vedremo come viene rappresentato nell'hardware e qual è la relazione tra i linguaggi di programmazione ad alto livello e questo linguaggio più primitivo.

Gli esempi che mostreremo nel corso del capitolo sono realizzati utilizzando il C come linguaggio di programmazione ad alto livello; nel Paragrafo 2.15 mostreremo gli stessi esempi utilizzando un linguaggio di programmazione orientato agli oggetti come Java.

Concetto di programma memorizzato: l'idea che le istruzioni e i dati di diverso tipo possano essere memorizzati come numeri; questa idea porta al calcolatore con programma memorizzato.

Imparando il modo in cui sono rappresentate le istruzioni, potrete anche scoprire il segreto dell'informatica: il **concetto di programma memorizzato**. Inoltre, potrete fare pratica di "lingua straniera" scrivendo programmi nel linguaggio del calcolatore, per farli poi eseguire al simulatore che troverete sul sito web del libro. Potrete anche osservare l'impatto dei linguaggi di programmazione e delle ottimizzazioni del compilatore sulle prestazioni.

Il capitolo si conclude con uno sguardo all'evoluzione storica degli insiemi di istruzioni, con un accenno ad altri dialetti usati dai calcolatori. L'insieme di istruzioni RISC-V verrà presentato in maniera graduale, un sottoinsieme alla volta, spiegando ogni volta la motivazione e le strutture del calcolatore interessate. Questo approccio dall'alto al basso intreccia passo dopo passo la descrizione dei componenti con la spiegazione del loro funzionamento, rendendo il linguaggio assembler più "digeribile". Nella **Figura 2.1** viene anticipato l'insieme delle istruzioni presentate in questo capitolo.

Approfondimento. Il RISC-V è un'architettura aperta e controllata dalla RISC-V International; non è un'architettura di proprietà di una particolare azienda come l'ARM, il MIPS, o l'x86. Nel 2020, più di 200 aziende erano membri della RISC-V International, e la sua popolarità sta crescendo rapidamente.

Operandi RISC-V

Nome	Esempio	Commenti
32 registri	x0-x31	Accesso veloce ai dati. Nel RISC-V gli operandi devono essere contenuti nei registri per potere eseguire delle operazioni. Il registro x0 contiene sempre il valore 0
2^{30} parole di memoria	Memoria[0], Memoria[4], … Memoria[4 294 967 292]	Alla memoria si accede solamente attraverso istruzioni di trasferimento dati. Il RISC-V utilizza l'indirizzamento al byte, perciò due variabili successive) hanno indirizzi in memoria a distanza 4. La memoria consente di memorizzare strutture dati, vettori, o il contenuto dei registri

Linguaggio assembler RISC-V

Tipo di istruzioni	Istruzioni	Esempio	Significato	Commenti
Aritmetiche	Somma	`add x5, x6, x7`	`x5 = x6 + x7`	Operandi in tre registri
	Sottrazione	`sub x5, x6, x7`	`x5 = x6 - x7`	Operandi in tre registri
	Somma immediata	`addi x5, x6, 20`	`x5 = x6 + 20`	Utilizzata per sommare delle costanti
Trasferimento dati	Lettura parola	`lw x5, 40(x6)`	`x5 = Memoria[x6 + 40]`	Spostamento di una parola da memoria a registro
	Lettura parola, senza segno	`lwu x5, 40(x6)`	`x5 = Memoria[x6+40]`	Spostamento di una parola senza segno da memoria a registro
	Memorizzazione parola	`sw x5, 40(x6)`	`Memoria[x6+40] = x5`	Spostamento di una parola da registro a memoria
	Lettura mezza parola	`lh x5, 40(x6)`	`x5 = Memoria[x6+40]`	Spostamento di una mezza parola da memoria a registro
	Lettura mezza parola, senza segno	`lhu x5, 40(x6)`	`x5 = Memoria[x6+40]`	Spostamento di una mezza parola senza segno da memoria a registro
	Memorizzazione mezza parola	`sh x5, 40(x6)`	`Memoria[x6+40]= x5`	Spostamento di una mezza parola da registro a memoria
	Lettura byte	`lb x5, 40(x6)`	`x5 = Memoria[x6+40]`	Spostamento di un byte da memoria a registro
	Lettura byte, senza segno	`lbu x5, 40(x6)`	`x5 = Memoria[x6+40]`	Spostamento di un byte senza segno da memoria a registro
	Memorizzazione byte	`sb x5, 40(x6)`	`Memoria[x6+40] = x5`	Spostamento di un byte da registro a memoria
	Lettura di una parola e blocco	`lr.d x5, (x6)`	`x5 = Memoria[x6]`	Caricamento di una parola come prima fase di un'operazione atomica sulla memoria
	Memorizzazione condizionata di una parola	`sc.d x7, x5, (x6)`	`Memoria[x6] = x5; x7 = 0/1`	Memorizzazione di una parola come seconda fase di un'operazione atomica sulla memoria
	Caricamento costante nella mezza parola superiore	`lui x5, 0x12345`	`x5 = 0x12345000`	Caricamento di una costante su 20 bit nei 12 bit più significativi di una parola
Logiche	And	`and x5, x6, x7`	`x5 = x6 & x7`	Operandi in tre registri; AND bit a bit
	Or inclusivo	`or x5, x6, x8`	`x5 = x6 | x8`	Operandi in tre registri; OR bit a bit
	Or esclusivo (Xor)	`xor x5, x6, x9`	`x5 = x6 ^ x9`	Operandi in tre registri; XOR bit a bit
	And immediato	`andi x5, x6, 20`	`x5 = x6 & 20`	AND bit a bit tra un operando in registro e una costante

Figura 2.1 Il linguaggio assembler del RISC-V esaminato in questo capitolo. Si possono trovare queste informazioni anche nella prima colonna della scheda tecnica riassuntiva del RISC-V in fondo al libro. *(continua)*

Tipo di istruzioni	Istruzioni	Esempio	Significato	Commenti
Logiche *(continua)*	Or esclusivo immediato	`ori x5, x6, 20`	`x5 = x6 | 20`	OR bit a bit tra un operando in registro e una costante
	Xor immediato	`xori x5, x6, 20`	`x5 = x6 ^ 20`	XOR bit a bit tra un operando in registro e una costante
Scorrimento (shift)	Scorrimento logico a sinistra	`sll x5, x6, x7`	`x5 = x6 << x7`	Scorrimento a sinistra mediante registro
	Scorrimento logico a destra	`srl x5, x6, x7`	`x5 = x6 >> x7`	Scorrimento a destra mediante registro
	Scorrimento aritmetico a destra	`sra x5, x6, x7`	`x5 = x6 >> x7`	Scorrimento a destra aritmetico mediante registro
	Scorrimento logico a sinistra immediato	`slli x5, x6, 3`	`x5 = x6 << 3`	Scorrimento a sinistra mediante costante
	Scorrimento logico a destra immediato	`srli x5, x6, 3`	`x5 = x6 >> 3`	Scorrimento a destra mediante costante
	Scorrimento aritmetico a destra immediato	`srai x5, x6, 3`	`x5 = x6 >> 3`	Scorrimento a destra aritmetico mediante costante
Salti condizionati	Salta se uguale	`beq x5, x6, 100`	`Se (x5 == x6) vai a PC+100`	Test di uguaglianza; salto relativo al PC
	Salta se non è uguale	`bne x5, x6, 100`	`Se (x5 != x6) vai a PC+100`	Test di disuguaglianza; salto relativo al PC
	Salta se minore di	`blt x5, x6, 100`	`Se (x5 < x6) vai a PC+100`	Comparazione di minoranza; salto relativo al PC
	Salta se maggiore o uguale di	`bge x5, x6, 100`	`Se (x5 >= x6) vai a PC+100`	Comparazione di maggioranza o uguaglianza; salto relativo al PC
	Salta se minore di, senza segno	`bltu x5, x6, 100`	`Se (x5 < x6) vai a PC+100`	Comparazione di minoranza senza segno; salto relativo al PC
	Salta se maggiore o uguale di, senza segno	`bgeu x5, x6, 100`	`Se (x5 >= x6) vai a PC+100`	Comparazione di maggioranza o uguaglianza senza segno; salto relativo al PC
Salti incondizionati	Salta e collega	`jal x1, 100`	`x1 = PC+4; vai a PC+100`	Chiamata a procedura con indirizzamento relativo al PC
	Salta e collega mediante registro	`jalr x1, 100(x5)`	`x1 = PC+4; vai a x5+100`	Ritorno da procedura; chiamata indiretta

Figura 2.1 Il linguaggio assembler del RISC-V esaminato in questo capitolo. *(continua)*

2.2 Operazioni svolte dall'hardware del calcolatore

Devono certamente essere previste istruzioni per il calcolo delle operazioni aritmetiche fondamentali.

Burks, Goldstine e von Neumann, 1947

Qualsiasi calcolatore deve saper eseguire le operazioni aritmetiche. La notazione del linguaggio assembler RISC-V:

```
add a,b,c
```

indica al calcolatore di sommare le due variabili `b` e `c` e di porre il risultato della somma nella variabile `a`.

Questa è una notazione rigida, nel senso che ciascuna istruzione aritmetica RISC-V esegue solo un'operazione e deve contenere esattamente tre variabili. Si supponga, per esempio, di volere memorizzare in `a` la somma delle variabili `b`, `c`, `d` ed `e` (in questo paragrafo il termine "variabile" sarà lasciato deliberatamente vago, rimandando al paragrafo successivo una spiegazione più precisa). La seguente sequenza di istruzioni effettuerà la somma delle quattro variabili:

```
add a,b,c   // la somma di b e c è posta in a
add a,a,d   // la somma di b, c e d è ora posta in a
add a,a,e   // la somma di b, c, d ed e è ora posta in a
```

Occorrono quindi tre istruzioni per sommare le quattro variabili.

Su ciascuna linea di codice, le parole a destra del simbolo "//" rappresentano i *commenti* rivolti a chi legge il programma e sono ignorate dal calcolatore. Si noti che, a differenza di altri linguaggi di programmazione, in linguaggio assembler ciascuna linea può contenere al massimo un'istruzione. Un'altra differenza rispetto al C è che i commenti terminano sempre con la fine della linea.

Il numero di operandi più naturale per un'operazione come la somma è pari a tre: i due numeri da sommare e il riferimento alla locazione in cui memorizzare il risultato. Il fatto di richiedere che tutte le istruzioni abbiano esattamente tre operandi, né uno di più né uno di meno, è conforme alla filosofia di mantenere l'hardware semplice: l'hardware richiesto per un numero variabile di operandi sarebbe più complesso di quello necessario per un numero fisso di operandi. Questa situazione illustra il primo dei tre princìpi fondamentali per la progettazione dell'hardware:

Principio di progettazione numero 1: la semplicità favorisce la regolarità.

Nei due esempi che seguono verrà mostrata la relazione tra programmi scritti in linguaggi ad alto livello e programmi scritti in questa notazione più primitiva.

Compilazione di due assegnamenti in C e loro traduzione in istruzioni RISC-V

ESEMPIO

Questo frammento di programma C contiene cinque variabili `a`, `b`, `c`, `d` ed `e`. Dato che Java è nato dal C, questo esempio e i successivi saranno validi per entrambi questi linguaggi di programmazione ad alto livello:

```
a = b + c;
d = a - e;
```

La traduzione da C ad assembler RISC-V viene effettuata dal *compilatore*. Mostrare il codice RISC-V prodotto dal compilatore.

SOLUZIONE

Un'istruzione RISC-V lavora su due operandi sorgente e inserisce il risultato in un operando destinazione. Il risultato della fase di compilazione delle due istruzioni mostrate sopra è quindi dato semplicemente da queste due istruzioni assembler RISC-V:

```
add a,b,c
sub d,a,e
```

Compilazione di un assegnamento complesso in C e sua traduzione in istruzioni RISC-V

ESEMPIO

Si consideri il seguente assegnamento complesso contenente le cinque variabili `f`, `g`, `h`, `i`, `j`:

```
f = (g + h) - (i + j);
```

Qual è il codice ottenuto in seguito alla compilazione?

SOLUZIONE

Il compilatore deve spezzare questo assegnamento C in più istruzioni assembler, poiché si può effettuare una sola operazione per ogni istruzione RISC-V. La prima istruzione RISC-V calcolerà la somma di `g` e `h`.

(continua)

 © 978-88-08-19966-9

(continua)

Dovendo memorizzare il risultato di questa operazione da qualche parte, il compilatore creerà una variabile temporanea chiamata `t0`:

```
add t0,g,h  // la variabile temporanea t0 contiene g + h
```

Nonostante l'operazione successiva sia una sottrazione, dobbiamo, per motivi di precedenza, effettuare prima la somma tra `i` e `j`. Quindi, la seconda istruzione sarà composta dalla somma di `i` e `j` e verrà creata una nuova variabile temporanea `t1` per memorizzare il risultato:

```
add t1,i,j  // la variabile temporanea t1 contiene i + j
```

Alla fine, verrà effettuata la sottrazione tra la prima e la seconda somma e verrà scritto il risultato nella variabile `f`, completando così la traduzione del codice C:

```
sub f,t0,t1  // f assume il valore t0 - t1,
             // cioè (g + h) - (i + j)
```

Autovalutazione

Per descrivere una data funzione, quale linguaggio di programmazione richiederà verosimilmente più linee di codice? Si ordinino i seguenti tre linguaggi utilizzando questa metrica.

1. Java
2. C
3. Linguaggio assembler RISC-V

Approfondimento. Per aumentarne la portabilità, Java fu originariamente concepito per essere "interpretato" da un software. L'insieme delle istruzioni di questo interprete fu chiamato *bytecode Java* (Paragrafo 2.15 🌐), che è molto differente dall'insieme delle istruzioni del RISC-V. Per avvicinare le prestazioni a quelle di un equivalente programma C, i sistemi Java tipicamente compilano il bytecode Java, traducendolo nell'insieme delle istruzioni native del processore, per esempio in istruzioni RISC-V. Dato che i compilatori Java entrano in azione molto più tardi rispetto ai compilatori C, i compilatori Java vengono spesso chiamati compilatori *Just In Time* (JIT). Il Paragrafo 2.12 illustrerà che cosa si intende affermando che i compilatori di tipo JIT vengono utilizzati più tardi rispetto a quelli C nell'avviamento di un'applicazione; il Paragrafo 2.13 mostrerà l'impatto sulle prestazioni della compilazione rispetto all'interpretazione di un programma Java.

2.3 Operandi dell'hardware del calcolatore

Parola: il numero di bit a cui si accede più naturalmente in un calcolatore; tipicamente è un gruppo di 32 bit. Nell'architettura RISC-V corrisponde alla dimensione dei registri.

Parola doppia: un gruppo di bit di diversa numerosità a cui si accede naturalmente in un calcolatore; tipicamente è un gruppo di 64 bit.

A differenza dei programmi scritti nei linguaggi ad alto livello, gli operandi delle istruzioni aritmetiche del RISC-V devono obbedire ad alcune restrizioni: devono essere scelti tra un numero limitato di locazioni particolari, chiamate *registri*. I registri sono i mattoni che costituiscono il calcolatore, dal momento che rappresentano sia le primitive utilizzate nella progettazione dell'hardware sia gli elementi visibili al programmatore. La dimensione dei registri nell'architettura RISC-V è di 32 bit; gruppi di 32 bit sono così frequenti che, nelle architetture RISC-V, prendono il nome di **parola** (**word**). Un altro gruppo di bit molto utilizzato nell'architettura RISC-V è la **parola doppia** (**doubleword**), costituita da 64 bit.

Una delle differenze più importanti fra le variabili utilizzate nei linguaggi di programmazione e i registri è il numero limitato di questi ultimi, tipicamente 32 nei calcolatori della classe dei RISC-V (Paragrafo 2.24 per una panoramica storica sul numero dei registri). In questo paragrafo, compiendo un altro passo verso la rappresentazione in linguaggio macchina delle istruzioni, introduciamo il vincolo addizionale per cui i tre operandi delle istruzioni aritmetiche RISC-V devono essere scelti tra i 32 registri a 32 bit.

La ragione della limitazione a 32 del numero di registri può essere trovata nel secondo dei tre princìpi fondamentali per la progettazione dell'hardware:

> *Principio di progettazione numero 2*: minori sono le dimensioni, maggiore è la velocità.

Un numero di registri molto elevato potrebbe aumentare la durata del ciclo di clock semplicemente perché i segnali elettrici impiegherebbero un tempo maggiore a compiere il percorso assegnato, dovendo coprire una distanza maggiore.

Princìpi guida come "minori sono le dimensioni, maggiore è la velocità" non sono validi in modo assoluto: un calcolatore con 31 registri potrebbe non essere più veloce di uno con 32. Tuttavia, tali osservazioni sono basate sulla realtà, tanto che i progettisti le prendono sul serio. Nel caso specifico, il progettista deve trovare un compromesso fra la richiesta da parte dei programmi di avere più registri a disposizione e il suo desiderio di mantenere veloce il clock. Un'altra ragione per non utilizzare più di 32 registri è il numero di bit che sarebbe necessario per indirizzarli all'interno dell'istruzione, come si vedrà nel Paragrafo 2.5.

Il Capitolo 4 mostra il ruolo centrale dei registri nella costruzione dell'hardware; come si potrà comprendere, un utilizzo efficiente dei registri è la chiave per ottenere buone prestazioni da un programma.

Anche se potremmo benissimo indicare ogni registro con un numero compreso tra 0 e 31, la convenzione RISC-V prevede l'utilizzo di una `x` seguita dal numero del registro, a eccezione di alcuni registri che hanno un nome personalizzato e che introdurremo più avanti.

Compilazione di un assegnamento in C utilizzando i registri

ESEMPIO

Associare i registri alle variabili di un programma è compito del compilatore. Si prenda come esempio l'assegnamento esaminato in precedenza:

```
f = (g + h) - (i + j);
```

Le variabili `f`, `g`, `h`, `i`, `j` vengono assegnate ai registri `x19`, `x20`, `x21`, `x22` e `x23`. Quale sarà il codice RISC-V ottenuto in seguito alla compilazione?

SOLUZIONE

Il programma compilato è molto simile a quello dell'esempio precedente, a eccezione del fatto che le variabili sono state sostituite dai nomi dei registri precedentemente definiti e che sono comparsi due registri temporanei, `x5` e `x6`, che corrispondono alle variabili temporanee:

```
add x5,x20,x21  // il registro temporaneo x5
                // contiene il risultato g + h
add x6,x22,x23  // il registro temporaneo x6
                // contiene il risultato i + j
sub x19,x5,x6   // f assume il valore x5 - x6,
                // cioè (g + h) - (i + j)
```

 © 978-88-08-19966-9

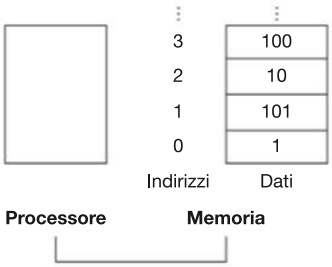


Figura 2.2 Indirizzi di memoria e contenuto della memoria nelle posizioni corrispondenti. Se i dati memorizzati fossero delle parole, i loro indirizzi sarebbero errati, dato che il RISC-V utilizza l'indirizzamento della memoria al byte e ogni parola è costituita da 4 byte. La Figura 2.3 mostra come vengano indirizzate in memoria parole consecutive.

Operandi allocati in memoria

I linguaggi di programmazione possono utilizzare variabili semplici che, come nel caso appena esposto, sono composte da un singolo elemento, ma possono utilizzare anche strutture di dati più complesse, come i vettori e le strutture dati. Queste strutture complesse possono contenere un numero di elementi molto maggiore del numero di registri presenti nel calcolatore: come può un calcolatore rappresentare e accedere a strutture dati così grandi?

I cinque componenti del calcolatore, introdotti nel Capitolo 1, suggeriscono che, benché il processore possa contenere un limitato numero di dati nei registri, la memoria può contenere miliardi di dati. Di conseguenza le strutture dati (vettori e strutture) vengono allocate in memoria.

Istruzione di trasferimento dati: un comando che sposta i dati tra la memoria e i registri.

Indirizzo: un numero utilizzato per identificare la posizione di uno specifico elemento all'interno di una memoria, vista come un vettore unidimensionale.

Come detto precedentemente, nel linguaggio assembler RISC-V le istruzioni aritmetiche richiedono che gli operandi siano memorizzati nei registri; l'assembler RISC-V deve quindi contenere delle **istruzioni di trasferimento dati** che trasferiscano dati fra la memoria e i registri. Per accedere a una parola in memoria, l'istruzione deve fornire l'**indirizzo** di memoria corrispondente. La memoria può essere vista come un grande vettore monodimensionale, con l'indirizzo che funge da indice e parte a contare da zero. Per esempio, nella **Figura 2.2**, l'indirizzo del terzo dato è 2 e il valore di Memoria[2] è 10.

L'istruzione di trasferimento che sposta un dato dalla memoria a un registro viene chiamata tradizionalmente *load* (carica). Il formato dell'istruzione load prevede il nome dell'operazione seguito dal registro in cui deve essere trasferito il dato e da una coppia costante-registro utilizzata per accedere alla memoria. L'indirizzo del dato in memoria viene ottenuto dalla somma della costante e del contenuto del secondo registro. Il nome convenzionale dell'istruzione di load nel linguaggio RISC-V è `lw`, che sta per *load word* (carica una parola).

Compilazione di un'istruzione di assegnamento quando uno degli operandi risiede in memoria

ESEMPIO

Si supponga che `A` sia un vettore di 100 parole e che il compilatore associ le variabili `g` e `h` ai registri `x20` e `x21`, come nel caso precedente. Si supponga inoltre che l'indirizzo di partenza del vettore, detto *indirizzo base*, sia memorizzato in `x22`. Si traduca in assembler il seguente assegnamento scritto in C:

```
g = h + A[8];
```

(continua)

(continua)

SOLUZIONE

Benché l'assegnamento in C sia costituito da una sola operazione, uno degli operandi si trova in memoria e quindi, per prima cosa, occorre trasferire il contenuto di `A[8]` in un registro. L'indirizzo di questo elemento del vettore è dato dalla somma dell'indirizzo base del vettore `A`, che si trova nel registro `x22`, e del numero che permette di selezionare l'elemento 8. Il dato letto dovrà essere memorizzato in un registro temporaneo e potrà essere utilizzato dall'istruzione successiva. Osservando la Figura 2.2, la prima istruzione in linguaggio assembler sarà:

```
lw x9, 8(x22)  // il valore di A[8] viene caricato
               // nel registro temporaneo x9
```

Nel prossimo esempio faremo una piccola modifica a questa istruzione, per ora utilizziamo questa versione semplificata. L'istruzione successiva potrà utilizzare il valore memorizzato in `x9` (che è uguale ad `A[8]`), dal momento che esso è ora contenuto in un registro.

L'istruzione dovrà quindi sommare `h` (contenuto in `x21`) ad `A[8]` (contenuto in `x9`) e scrivere il risultato nel registro assegnato a `g` (`x20`):

```
add x20, x21, x9 // g = h + A[8]
```

Il registro utilizzato per costruire l'indirizzo (`x22`) è chiamato *registro base* e la costante in un'istruzione di trasferimento (8 in questo caso) si chiama *offset* (spiazzamento).

Interfaccia hardware/software

Oltre ad associare le variabili ai registri, il compilatore alloca strutture dati, quali i vettori e le strutture dati più complesse in locazioni di memoria, e pone quindi il loro indirizzo di partenza corretto nelle istruzioni di trasferimento dati.

Dal momento che i byte sono utilizzati in molti programmi, virtualmente tutte le architetture indirizzano il singolo byte; di conseguenza, l'indirizzo di una parola corrisponde all'indirizzo di uno dei 4 byte che la compongono e l'indirizzo di due parole consecutive differisce di 4 unità. La **Figura 2.3**, per esempio, mostra i veri indirizzi RISC-V per le parole della Figura 2.2: l'indirizzo in byte della terza parola è ora 8.

I calcolatori con indirizzamento relativo al singolo byte si dividono fra quelli che utilizzano il byte più significativo, quello più a sinistra (*big end*), per specificare l'indirizzo della parola, e quelli che utilizzano il byte meno significativo, quello più a destra (*little end*). Il RISC-V fa parte della seconda categoria detta *Little Endian*. Dato che l'ordinamento importa solo se si legge o scrive una parola byte per byte, a pochi interessa se la codifica sia big o little endian.

L'indirizzamento al byte influenza anche il calcolo degli indici dei vettori: per ottenere l'indirizzo in byte corretto per il frammento di codice precedente, l'offset da sommare al registro di base, `x22`, deve essere uguale a 8 × 4, cioè 32, in modo che l'istruzione load possa selezionare `A[8]` e non `A[8/4]`. Nel Paragrafo 2.22 vedremo i trabocchetti che si possono generare.

L'istruzione complementare a quella di load è chiamata tradizionalmente *store* (memorizza) e trasferisce un dato da un registro alla memoria. Il formato di questa istruzione è simile a quello dell'istruzione load: il nome dell'istruzione seguito dal registro che contiene il dato da trasferire in memoria, dall'offset corrispondente all'elemento del vettore e dal registro base.

Come nel caso precedente, l'indirizzo di memoria viene specificato in parte da una costante e in parte dal contenuto di un registro. Il nome convenzionale dell'istruzione store in linguaggio RISC-V è `sw`, che sta per *store word* (memorizza una parola).

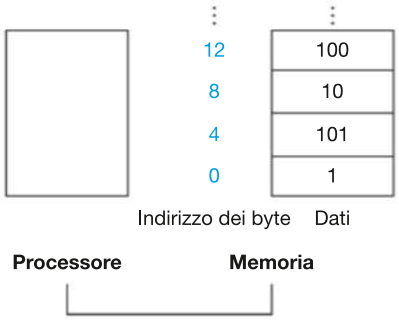


Figura 2.3 Indirizzi di memoria effettivi nel RISC-V e contenuto delle locazioni corrispondenti. Gli indirizzi evidenziati sono quelli che cambiano rispetto alla Figura 2.2. Dato che il RISC-V indirizza il singolo byte, gli indirizzi delle parole sono multipli di 4 dato che ci sono 4 byte in una parola.

Vincolo di allineamento: requisito per cui, in molte architetture, le parole devono iniziare sempre a indirizzi multipli di 4 e le parole doppie a indirizzi multipli di 8.

Approfondimento. In molte architetture, le parole devono iniziare sempre a indirizzi multipli di 4. Questo requisito si chiama **vincolo di allineamento** ed è comune a molte altre architetture (nel Capitolo 4 viene suggerito come l'allineamento produca una velocità maggiore nel trasferimento dei dati). I RISC-V e gli Intel x86 non hanno il vincolo di allineamento, mentre i MIPS sì.

Interfaccia hardware/software

Dal momento che gli indirizzi delle istruzioni load e store sono numeri binari, si può capire perché gli indirizzi della memoria principale DRAM siano espressi in multipli di 2 invece che di 10. Vengono cioè espressi in Gibibyte (2^{30}), o in Tebibyte (2^{40}), e non in Gigabyte (10^9) o Terabyte (10^{12}). *Vedi* anche la Figura 1.1.

Compilazione utilizzando load e store

ESEMPIO

Si supponga che la variabile `h` sia associata al registro `x21` e che il registro base del vettore `A` sia `x22`. Quale sarà il codice assembler RISC-V relativo al seguente assegnamento in linguaggio C?

```
A[12] = h + A[8];
```

SOLUZIONE

Benché l'assegnamento sia costituito in C da una sola operazione, adesso gli operandi che si trovano in memoria sono due e quindi occorrono ancora più istruzioni RISC-V che nel caso precedente. Le prime due istruzioni sono le stesse dell'esempio precedente, tranne che per l'utilizzo dell'offset corretto di 32 per realizzare l'indirizzamento al byte nell'istruzione di caricamento in registro di una parola (`lw`) che seleziona `A[8]`. L'istruzione di somma `add` inserirà il risultato in `x9`:

```
lw x9, 32(x22)  // il registro temporaneo x9
                // assume il valore A[8]
add x9, x21, x9 // il registro temporaneo x9
                // assume il valore h + A[8]
```

L'ultima istruzione memorizza la somma in `A[12]`, utilizzando 48 (4×12) come offset e `x22` come registro base:

```
sw x9, 48(x22)  // memorizza h + A[8] in A[12]
```

Nelle architetture RISC-V le istruzioni load word e store word servono per trasferire una parola tra la memoria e i registri. Altre architetture utilizzano altre istruzioni, oltre a load e store, per trasferire dati; una di queste architetture è l'x86 Intel, che verrà descritta nel Paragrafo 2.17.

Interfaccia hardware/software

Molti programmi contengono più variabili di quanti siano i registri del calcolatore. Di conseguenza il compilatore cerca di mantenere nei registri le variabili utilizzate più di frequente e mette le altre in memoria, utilizzando le istruzioni di trasferimento dati per spostare le variabili tra i registri e la memoria. Il processo di trasferimento in memoria delle variabili utilizzate meno di frequente (oppure di quelle che verranno utilizzate successivamente) si chiama *register spilling* (versamento dei registri).

Il già citato principio hardware che lega le dimensioni alla velocità suggerisce che l'accesso alla memoria sia più lento dell'accesso ai registri, dal momento che il numero dei registri è minore del numero di locazioni della memoria. Ed è effettivamente così: l'accesso ai dati risulta più veloce se i dati si trovano nei registri piuttosto che in memoria.

Inoltre i dati risultano più utili se si trovano nei registri. Le istruzioni aritmetiche del RISC-V nello stesso ciclo di clock possono leggere due registri, eseguire l'operazione e scrivere il risultato. Le istruzioni di trasferimento dati del RISC-V, invece, possono solamente leggere o scrivere un singolo operando, senza potere eseguire nessuna operazione su di esso.

I registri, quindi, richiedono un tempo di accesso minore della memoria e offrono un throughput considerevolmente più elevato, rendendo i dati in essi contenuti accessibili più velocemente e più semplici da utilizzare. Per ottenere le prestazioni migliori e ridurre il consumo di energia, un'architettura deve avere un sufficiente numero di registri e i compilatori devono farne uso in maniera efficiente.

Approfondimento. Guardiamo in prospettiva l'energia e le prestazioni dei registri in funzione della memoria. Supponendo dati su 32 bit, i registri erano circa 200 volte più veloci (0,25 ns in confronto a 50 ns) e avevano un'efficienza energetica superiore di 10 000 volte (0,1 picoJoule in confronto a 1000 picoJoule) delle DRAM nel 2020. Questa grossa differenza ha portato allo sviluppo delle memorie cache, che riducono le penalità sulle prestazioni e sul consumo di energia quando si accede alla memoria (Capitolo 5).

Operandi immediati o costanti

Spesso i programmi utilizzano all'interno di un'operazione una costante, per esempio per incrementare l'indice di un contatore in modo da puntare all'elemento successivo di un vettore. In più della metà delle operazioni aritmetiche RISC-V contenute nei programmi benchmark dello SPEC CPU2006, uno degli operandi è una costante.

Utilizzando solamente le istruzioni viste finora dovremmo caricare una costante dalla memoria per poterla utilizzare (la costante verrà scritta in memoria all'atto del caricamento dei programma). Per esempio, per sommare 4 al registro `x22` possiamo utilizzare le seguenti istruzioni:

```
lw x9, IndirizzoCostante4(x3)  // x9 = costante 4
add x22, x22, x9               // x22 = x22 + x9
                               // (dove x9 == costante 4)
```

supponendo che `x3 + IndirizzoCostante4` sia l'indirizzo di memoria della costante 4.

Un'alternativa che consente di evitare l'operazione di load è costituita da una versione delle istruzioni aritmetiche nella quale uno degli operandi è una

costante. La versione dell'istruzione di somma in cui un operando è una costante è chiamata *add immediate* (somma immediata) o `addi`. Per sommare 4 al registro `x22`, possiamo semplicemente scrivere:

```
addi x22, x22, 4    // x22 = x22 + 4
```

Le operazioni su costanti sono molto frequenti (e l'istruzione `addi` è l'istruzione più utilizzata nella maggior parte dei programmi RISC-V). Inserendo le costanti all'interno delle istruzioni aritmetiche, le operazioni risultano molto più veloci e richiedono meno energia rispetto al caso in cui le costanti siano caricate dalla memoria.

La costante zero ha anche il ruolo di semplificare l'insieme delle istruzioni, consentendo di realizzare delle utili varianti. Per esempio, si può negare il contenuto di un registro utilizzando l'istruzione `sub` con zero come primo operando. Per questo motivo, i RISC-V dedicano il registro `x0` a contenere il valore prefissato di zero. Giustificare l'introduzione di una costante attraverso la frequenza del suo utilizzo è un ulteriore esempio della grande idea introdotta nel Capitolo 1 di **rendere veloci le situazioni più comuni**.

SITUAZIONI COMUNI

Autovalutazione

Data l'importanza dei registri, qual è il tasso di crescita del numero dei registri su un singolo chip?

1. Molto veloce: il loro incremento seguiva la legge di Moore, che prediceva il raddoppio del numero di transistor su un singolo chip ogni 24 mesi.
2. Molto lento: dato che i programmi sono tipicamente distribuiti ai clienti nella versione tradotta in linguaggio macchina, c'è una notevole inerzia nelle generazione di nuove architetture di insiemi di istruzioni. Il numero di registri aumenta alla velocità con cui vengono resi disponibili nuovi insiemi di istruzioni.

Approfondimento. Nonostante i registri RISC-V considerati in questo libro abbiano un'ampiezza di 32 bit, gli architetti hanno pensato a diverse varianti dell'ISA. Oltre a questa variante nota come RV32, la variante RV64 ha registri a 64 bit; l'ampiezza maggiore rende l'architettura RV64 più adatta ai processori per i server e per i telefoni cellulari.

Approfondimento. Il sistema di indirizzamento dei RISC-V tramite offset e registro base è ottimo per accedere sia alle strutture dati sia ai vettori, dal momento che il registro può puntare all'inizio della struttura e l'offset può selezionare l'elemento desiderato. Troverete un esempio nel Paragrafo 2.13.

Approfondimento. Nelle istruzioni per il trasferimento dati, i registri furono originariamente introdotti per memorizzare l'indice dell'elemento del vettore, mentre l'offset conteneva l'indirizzo di partenza del vettore stesso. Perciò il registro base è anche chiamato *registro indice*. Oggi le memorie sono molto più grandi e il modello software per l'allocazione dei dati è molto più sofisticato; l'indirizzo base di un vettore viene quindi solitamente memorizzato in un registro, dato che la dimensione del campo offset non sarebbe sufficiente a contenerlo, come si vedrà in seguito.

Approfondimento. La migrazione dai calcolatori da 32 bit a quelli da 64 bit ha lasciato ai programmatori dei compilatori la scelta della dimensione dei tipi di dati in C. Chiaramente, i puntatori dovranno essere a 64 bit, ma che dimensione dovranno avere gli interi? Inoltre, il C ha i tipi di dati `int`, `long int` e `long long int`. I problemi nascono quando si devono convertire i dati da un tipo all'altro e quando si verifica inaspettatamente un overflow in un programma C non completamente ortodosso, cosa che sfortunatamente si verifica non raramente. La tabella seguente riporta due opzioni popolari:

Sistema operativo	Puntatori	int	long int	long long int
Windows Microsoft	64 bit	32 bit	32 bit	64 bit
Linux, la maggior parte di Unix	64 bit	32 bit	64 bit	64 bit

2.4 Numeri con e senza segno

Per prima cosa ripassiamo velocemente la rappresentazione dei numeri nei calcolatori. Poiché noi abbiamo dieci dita, ci è stato insegnato a contare in base 10, ma i numeri possono essere rappresentati in qualunque base; per esempio 123 in base 10 = 1111011 in base 2.

I numeri vengono rappresentati nell'hardware del calcolatore tramite una serie di segnali elettrici di livello alto o basso, quindi la rappresentazione più naturale è quella in base 2 (come i numeri in base 10 sono chiamati numeri *decimali*, così i numeri in base 2 sono chiamati numeri *binari*).

Le cifre che costituiscono un numero binario costituiscono quindi le "unità atomiche" del calcolo, dato che tutte le informazioni in un calcolatore sono costituite da **cifre binarie** (*binary digits*) o **bit**. Le cifre binarie possono assumere due soli valori, che possono essere descritti in vario modo: alto o basso, acceso o spento, vero o falso, 1 o 0.

Cifra binaria o bit: una delle due cifre binarie, 0 o 1, con cui vengono rappresentati i numeri in base 2, che sono il fondamento della rappresentazione dell'informazione.

Qualunque sia la base, il valore della i-esima cifra d di un numero è pari a:

$$d \times \text{Base}^i$$

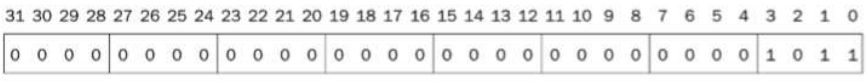
dove i assume un valore che parte da 0 e viene incrementato spostandosi da destra verso sinistra. Di conseguenza, un modo immediato per associare un numero ai bit all'interno di una parola è quello di utilizzare la potenza della base relativa a quel bit. Indicheremo i numeri decimali con il pedice *dec* (decimali) e i numeri binari con il pedice *due*. Per esempio,

$$1011_{due}$$

rappresenta:

$$\begin{aligned} & (1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (1 \times 20)_{dec} \\ = \; & (1 \times 8) + (0 \times 4) + (1 \times 2) + (1 \times 1)_{dec} \\ = \; & 8 + 0 + 2 + 1_{dec} \\ = \; & 11_{dec} \end{aligned}$$

Abbiamo assegnato un numero ai bit: 0, 1, 2, 3 ... analizzati da *destra verso sinistra* all'interno della parola. Lo schema seguente illustra il valore assunto dai diversi bit all'interno di una parola del RISC-V e la posizione che assumerebbe il numero 1011_{due}:



(Ampiezza di 32 bit)

Dal momento che le parole vengono scritte a volte verticalmente e a volte orizzontalmente, i termini destra e sinistra possono risultare poco chiari. Di qui l'utilizzo dei termini **bit meno significativo** (*least significant bit*) per indicare il bit più a destra di una parola (il bit 0 nell'esempio precedente) e **bit più significativo** (*most significant bit*) per indicare il bit più a sinistra (il bit 31 per una parola).

Bit meno significativo: il bit più a destra in una parola RISC-V.

Bit più significativo: il bit più a sinistra in una parola RISC-V.

Poiché la generica parola dei RISC-V è lunga 32 bit, possiamo rappresentare 2^{32} diverse combinazioni di 32 bit ciascuna. È naturale utilizzare le combinazioni di questi 32 bit per rappresentare i numeri da 0 a $2^{32} - 1$ ($4\,294\,967\,295_{dec}$):

$0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000_{due} = 0_{dec}$
$0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0001_{due} = 1_{dec}$
$0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0010_{due} = 2_{dec}$
$\ldots \qquad\qquad \ldots$
$1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1101_{due} = 4294967293_{dec}$
$1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1110_{due} = 4294967294_{dec}$
$1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111_{due} = 4294967295_{dec}$

Cioè, i numeri binari a 32 bit si possono rappresentare come somma di potenze di 2 (dove x_i sta per i-esimo bit di x):

$$(x_{31} \times 2^{31}) + (x_{30} \times 2^{30}) + (x_{29} \times 2^{29}) + \ldots + (x_1 \times 2^1) + (x_0 \times 2^0)$$

Per i motivi che vedremo fra poco, questi numeri sono chiamati numeri *unsigned* (senza segno).

Interfaccia hardware/software

La base 2 non è naturale per gli esseri umani: avendo dieci dita, a noi risulta più naturale la base 10. Perché i calcolatori non utilizzano la numerazione in base 10? In realtà, i primi calcolatori commerciali *utilizzavano* l'aritmetica in base 10. Il problema è che un calcolatore utilizza sempre i segnali acceso e spento, per cui i numeri decimali venivano comunque rappresentati da un insieme di cifre binarie. La codifica decimale si rivelò così inefficiente che i calcolatori delle generazioni successive ritornarono alla codifica binaria, convertendo i numeri nella rappresentazione in base 10 solamente per le operazioni, relativamente poco frequenti, di input/output.

Si tenga presente che le combinazioni di bit introdotte precedentemente sono semplicemente delle *rappresentazioni* dei numeri. In realtà i numeri hanno un numero infinito di cifre, delle quali quasi tutte assumono il valore 0 tranne quelle nelle posizioni più a destra. Normalmente non si considerano gli 0 iniziali.

Overflow: si verifica quando il risultato di un'operazione non può essere rappresentato in un registro.

È possibile progettare un circuito logico che sommi, sottragga, moltiplichi e divida queste combinazioni di bit, come vedremo nel Capitolo 3. Se il risultato di queste operazioni non può essere rappresentato con i bit resi disponibili dall'hardware, si dice che si è verificato un **overflow** (tracimazione). È compito del linguaggio ad alto livello, del sistema operativo e del programma determinare cosa fare quando si verifica un overflow.

I programmi dei calcolatori eseguono operazioni su numeri sia positivi sia negativi, ed è quindi necessaria una rappresentazione che permetta di distinguerli. La soluzione più ovvia consiste nell'aggiungere un'informazione di segno, che può essere convenientemente rappresentata da un singolo bit. Il nome di questa rappresentazione è *modulo e segno*.

Purtroppo la rappresentazione in modulo e segno ha diversi limiti. Anzitutto non è chiaro dove posizionare il bit di segno. A destra? A sinistra? Nei primi calcolatori vennero sperimentate ambedue le soluzioni. In secondo luogo, i sommatori che operano su numeri rappresentati mediante modulo e segno richiedono un passo aggiuntivo per determinare il segno, dal momento che questo non può essere determinato a priori. Infine, un bit di segno

separato dagli altri significa che la rappresentazione in modulo e segno possiede sia uno 0 positivo sia uno 0 negativo, cosa che può creare dei problemi ai programmatori meno attenti. La conseguenza di questi limiti è che la rappresentazione in modulo e segno fu molto presto abbandonata.

Mentre si cercavano alternative più convenienti, si presentò il problema di come si potesse rappresentare il risultato di un'operazione di sottrazione tra due numeri senza segno quando si sottraeva un numero grande da uno piccolo. Applicando le normali regole della sottrazione, il risultato è una stringa di 1 iniziali, ottenuti dai riporti generati dall'operazione.

Dal momento che non sembravano esserci alternative migliori, la rappresentazione adottata fu quella che rendeva l'hardware più semplice: prevedeva che una sequenza di 0 iniziali indicasse un valore positivo e una sequenza di 1 un numero negativo. Questa convenzione per la rappresentazione dei numeri binari dotati di segno prende il nome di rappresentazione in *complemento a 2* (uno degli approfondimenti alla fine del paragrafo spiega la ragione di questo strano nome):

$00000000\ 00000000\ 00000000\ 00000000_{due} = 0_{dec}$
$00000000\ 00000000\ 00000000\ 00000001_{due} = 1_{dec}$
$00000000\ 00000000\ 00000000\ 00000010_{due} = 2_{dec}$
...
$01111111\ 11111111\ 11111111\ 11111101_{due} = 2\,147\,483\,645_{dec}$
$01111111\ 11111111\ 11111111\ 11111110_{due} = 2\,147\,483\,646_{dec}$
$01111111\ 11111111\ 11111111\ 11111111_{due} = 2\,147\,483\,647_{dec}$
$10000000\ 00000000\ 00000000\ 00000000_{due} = -2\,147\,483\,648_{dec}$
$10000000\ 00000000\ 00000000\ 00000001_{due} = -2\,147\,483\,647_{dec}$
$10000000\ 00000000\ 00000000\ 00000010_{due} = -2\,147\,483\,646_{dec}$
...
$11111111\ 11111111\ 11111111\ 11111101_{due} = -3_{dec}$
$11111111\ 11111111\ 11111111\ 11111110_{due} = -2_{dec}$
$11111111\ 11111111\ 11111111\ 11111111_{due} = -1_{dec}$

La metà positiva dei numeri, da 0 a $2\,147\,483\,647_{dec}$ ($2^{31} - 1$), utilizza la stessa rappresentazione vista in precedenza. La combinazione successiva ($1000 \ldots 0000_{due}$) rappresenta il numero negativo di valore assoluto maggiore, ossia $-2\,147\,483\,648_{dec}$ (-2^{31}). Subito dopo vi è una sequenza decrescente di numeri negativi, da $-2\,147\,483\,647_{dec}$ ($1000 \ldots 0001_{due}$) fino a -1_{dec} ($1111 \ldots 1111_{due}$).

La rappresentazione in complemento a 2 comprende un numero negativo, $-2\,147\,483\,648_{dec}$, che non ha un corrispondente numero positivo. Questo sbilanciamento richiede ai programmatori una certa attenzione, ma la rappresentazione in modulo e segno creava problemi sia ai programmatori sia ai progettisti hardware. Pertanto tutti i calcolatori di oggi utilizzano la rappresentazione binaria in complemento a 2 per i numeri dotati di segno.

La rappresentazione in complemento a due ha il vantaggio che tutti i numeri negativi hanno il bit più significativo uguale a 1. Perciò l'hardware deve controllare soltanto questo bit per verificare se il numero è positivo o negativo (un numero che ha inizio con lo zero viene considerato positivo). Questo bit è a volte chiamato *bit di segno*. Considerando il ruolo particolare assunto da questo bit, si possono rappresentare numeri positivi e negativi su 32 bit moltiplicando ciascun bit per una potenza di 2:

$$(x_{31} \times -2^{31}) + (x_{30} \times 2^{30}) + (x_{29} \times 2^{29}) + \ldots + (x_1 \times 2^1) + (x_0 \times 2^0)$$

Il bit di segno è moltiplicato per -2^{31}, mentre tutti gli altri bit sono moltiplicati per il valore, positivo, delle rispettive basi.

Conversione da binario a decimale

ESEMPIO

Qual è il numero decimale associato al seguente numero su 32 bit in complemento a 2?

$11111111\ 11111111\ 11111111\ 11111100_{due}$

SOLUZIONE

Sostituendo il valore dei bit nella formula precedentemente introdotta si ha:

$$(1 \times -2^{31}) + (1 \times 2^{30}) + (1 \times 2^{29}) + \ldots + (1 \times 2^{2}) + (0 \times 2^{1}) + (0 \times 2^{0})$$
$$= -2^{31} + 2^{30} + 2^{29} + \ldots + 2^{2} + 0 + 0$$
$$= -2\,147\,483\,648_{dec} + 2\,147\,483\,644_{dec} = -4_{dec}$$

Verrà introdotta a breve una scorciatoia che rende più semplice la conversione.

Come per il risultato di un'operazione su numeri senza segno, anche in un'operazione su numeri in complemento a 2 si può eccedere la capacità di rappresentazione del risultato da parte dell'hardware. La condizione di overflow si verifica quando il bit più a sinistra non è uguale agli infiniti bit presenti alla sinistra del numero stesso (ossia il bit di segno non è corretto): uno 0 a sinistra della combinazione di bit che rappresenta un numero negativo oppure un 1 a sinistra di quella che rappresenta un numero positivo.

Interfaccia hardware/software

La distinzione tra numeri con e senza segno riguarda sia le istruzioni di trasferimento dalla memoria sia le istruzioni aritmetiche. Quando si effettua il caricamento di un numero dotato di segno, il bit di segno viene ripetutamente copiato fino a riempire il resto del registro. Questa operazione, chiamata *estensione del segno*, ha l'obiettivo di scrivere la rappresentazione corretta del numero nel registro. Quando invece si carica un numero senza segno, tutte le posizioni libere a sinistra del numero vengono riempite con degli 0, dato che il numero rappresentato è privo di segno.

Quando si carica dalla memoria una parola su 32 bit in un registro a 32 bit la questione diventa irrilevante, dato che l'operazione di caricamento diventa la stessa sia che il numero abbia segno sia che non ce l'abbia. Invece, per il caricamento dei byte, il RISC-V offre due istruzioni: *load byte unsigned* (`lbu`) tratta il byte come un numero privo di segno e perciò riempie con zeri gli altri bit alla sinistra del byte letto, mentre l'istruzione *load byte* (`lb`) considera il byte dotato di segno e copia il bit più significativo nelle posizioni del registro alla sinistra del byte letto. Dato che i programmi C utilizzano i byte quasi sempre per rappresentare caratteri alfanumerici e non per rappresentare numeri interi, viene utilizzata principalmente l'istruzione `lbu` per caricare i byte dalla memoria.

Interfaccia hardware/software

A differenza dei numeri esaminati prima, gli indirizzi della memoria partono dal valore 0 e proseguono fino all'indirizzo più alto; in altre parole, gli indirizzi negativi non hanno senso. Per questo motivo, a volte, i programmi richiedono l'utilizzo di numeri che possono assumere valori positivi o negativi, mentre a volte richiedono l'utilizzo di numeri che possono essere solo positivi. Alcuni linguaggi di programmazione riflettono questa distinzione. Il C, per esempio, chiama i numeri del primo tipo *integers* (interi, dichiarati come `int` all'interno dei programmi), mentre chiama i numeri solo positivi *unsigned integers* (interi senza segno, dichiarati come `unsigned int` all'interno dei programmi). Inoltre, alcuni manuali di programmazione in C per evitare confusione suggeriscono di dichiarare i numeri dotati di segno come `signed int`.

Esaminiamo alcune utili scorciatoie per lavorare con i numeri in complemento a 2. La prima scorciatoia è un modo veloce per cambiare il segno a un numero binario in complemento a 2. Basta semplicemente trasformare ogni 0 in 1 e ogni 1 in 0 e poi sommare 1 al risultato. Questa scorciatoia si basa sull'osservazione che la somma di un numero e del suo inverso dà sempre $111 \ldots 111_2$, che rappresenta il numero –1. Dato che $x + \bar{x} = -1$, segue che $x + \bar{x} + 1 = 0$, ossia $\bar{x} + 1 = -x$; dove utilizziamo la notazione $\bar{x}$ per indicare un numero binario ottenuto da x, invertendo tutte le cifre binarie da 0 a 1 e viceversa.

Scorciatoia per cambiare di segno un numero

ESEMPIO

Si cambi di segno il numero 2_{dec} e si verifichi poi il risultato cambiando di segno il numero -2_{dec}.

2_{dec} = 00000000 00000000 00000000 00000010_{due}

SOLUZIONE

Invertendo questo numero tramite l'inversione dei bit e sommando poi 1, si ottiene:

$$\begin{array}{rr} & 11111111\ 11111111\ 11111111\ 11111101_{due} \\ + & 1_{due} \\ \hline = & 11111111\ 11111111\ 11111111\ 11111110_{due} \\ = & -2_{dec} \end{array}$$

Procedendo nell'altra direzione, si parte da:

$11111111\ 11111111\ 11111111\ 11111110_{due}$

che prima si inverte e poi si incrementa:

$$\begin{array}{rr} & 00000000\ 00000000\ 00000000\ 00000001_{due} \\ + & 1_{due} \\ \hline = & 00000000\ 00000000\ 00000000\ 00000010_{due} \\ = & 2_{dec} \end{array}$$

La seconda scorciatoia serve per convertire un numero binario rappresentato su n bit in un numero rappresentato con più di n bit. La scorciatoia consiste nel prendere il bit più significativo (ossia il bit di segno) del numero su meno bit e nel replicarlo, in maniera da riempire gli altri bit presenti nel numero definito su un numero maggiore di bit.

Gli altri bit non contenenti il segno vengono semplicemente copiati nei bit più a destra della nuova parola doppia creata. Questa scorciatoia prende il nome di *estensione del segno*.

Scorciatoia per l'estensione del segno

ESEMPIO

Si convertano i numeri 2_{dec} e -2_{dec} codificati su 16 bit nei corrispondenti numeri binari codificati su 32 bit.

La versione binaria su 16 bit del numero 2 è:

$00000000\ 00000010_{due} = 2_{dec}$

(continua)

 © 978-88-08-19966-9

(continua)

SOLUZIONE

La conversione in un numero a 32 bit viene effettuata copiando 16 volte il valore del bit più significativo (0) nella metà sinistra della parola. La metà destra della parola riceve invece gli altri bit:

$$00000000\ 00000000\ 00000000\ 00000010_{due} = 2_{dec}$$

Si esegua ora il cambiamento di segno del numero 2 su 16 bit, utilizzando la scorciatoia illustrata in precedenza. Quindi,

$$0000\ 0000\ 0000\ 0010_{due}$$

diventa:

$$\begin{array}{rr} & 1111\ 1111\ 1111\ 1101_{due} \\ + & 1_{due} \\ \hline = & 1111\ 1111\ 1111\ 1110_{due} \end{array}$$

Per creare la versione su 32 bit del numero negativo si deve copiare il bit di segno per 16 volte e metterlo nella parte sinistra della parola:

$$11111111\ 11111111\ 11111111\ 11111110_{due} = -2_{dec}$$

Questo trucco funziona perché i numeri positivi in complemento a 2 hanno un numero infinito di 0 a sinistra, mentre quelli negativi un numero infinito di 1. Nella stringa di cifre binarie utilizzata per rappresentare un numero non vengono riportati i bit iniziali in modo tale da fare rientrare la lunghezza della cifra nell'ampiezza della parola dell'hardware; l'estensione del segno semplicemente ripristina alcuni di questi bit.

Riepilogo

Abbiamo trattato la necessità di rappresentare nella parola di un calcolatore numeri interi sia positivi sia negativi. Benché in ogni decisione vi siano pro e contro, la scelta unanime a 1965 è stata la rappresentazione in complemento a 2.

Approfondimento. Per quanto riguarda i numeri decimali dotati di segno, utilizzeremo il segno meno (–) per rappresentare i numeri negativi, perché non ci sono limiti alla dimensione dei numeri. Con la notazione binaria ed esadecimale, invece, la codifica è su parole di un numero prefissato di bit (Figura 2.4) e il segno viene espresso nella codifica stessa, e quindi i simboli + e – in queste notazioni normalmente non vengono utilizzati.

Autovalutazione

Qual è il valore decimale di questo numero di 64 bit codificato in complemento a 2?

$$11111111\ 11111111\ 11111111\ 11111111\ 11111111\ 11111111\ 11111111\ 11111000_{due}$$

1. -4_{dec}
2. -8_{dec}
3. -16_{dec}
4. $18446744073709551609_{dec}$

Quale sarebbe il suo valore decimale se fosse un numero *unsigned* su 64 bit?

Approfondimento. La rappresentazione in complemento a 2 deve il suo nome alla proprietà in base alla quale la somma senza segno di un numero di n bit e del suo complemento è pari a 2^n, e quindi il complemento (o negazione) di un numero x in complemento a 2 è pari a $2^n - x$, ovvero il suo complemento a 2.

Una terza possibile rappresentazione dei numeri dotati di segno è denominata **complemento a 1**. Il numero negativo di un numero in complemento a 1 si ottiene invertendo ciascun bit, da 0 a 1 e da 1 a 0, e viene indicato con x^-. Questo spiega il suo nome, dato che il complemento a 1 di x sarà $2^n - x - 1$. Questa rappresentazione, quando fu introdotta, era un tentativo di avere una codifica migliore della rappresentazione in modulo e segno e diversi tra i primi calcolatori di tipo scientifico adottarono tale notazione. Essa è simile al complemento a 2, tranne che prevede anch'essa due rappresentazioni per lo 0: lo 0 positivo (00 ... 00_{due}) e quello negativo (11 ... 11_{due}). Il numero negativo di valore assoluto maggiore su 32 bit è 10 … 000_{due} e rappresenta il valore $-2\ 147\ 483\ 647_{dec}$, perciò l'insieme dei numeri positivi e quello dei numeri negativi sono bilanciati. Tuttavia i sommatori in complemento a 1 richiedono un passo in più per eseguire la sottrazione di un numero e per questo il complemento a 2 è diventata la scelta dominante.

Un'ultima notazione prevede di rappresentare il numero più negativo con la sequenza 00 … 000_{due} e il numero più positivo con 11 … 11_{due}; lo 0 tipicamente assume il valore 10 … 00_{due}. Questa notazione è chiamata **notazione polarizzata**, dato che il numero viene polarizzato in modo tale che la somma del numero e della polarizzazione produca un numero non negativo. La esamineremo quando tratteremo i numeri in virgola mobile nel Capitolo 3.

Complemento a 1: codifica che rappresenta il minimo numero negativo come 10 ... 000_{due} e il massimo numero positivo come 01 ... 11_{due}. Gli insiemi dei numeri positivi e dei numeri negativi sono bilanciati ma hanno due codifiche per lo zero, lo zero positivo (00 … 00_{due}) e lo zero negativo (11 … 11_{due}). Complemento a 1 indica anche l'inversione dei bit di una stringa, da 0 a 1 e da 1 a 0.

Notazione polarizzata: notazione che prevede che il minimo valore negativo venga rappresentato da 00 … 000_{due} e il massimo valore positivo da 11 … 11_{due}, con lo 0 che tipicamente assume il valore 10 … 00_{due}; quindi, sommando un numero polarizzato con la polarizzazione si ottiene sempre un numero non negativo.

2.5 Rappresentazione delle istruzioni nel calcolatore

Siamo pronti ora a esaminare la differenza tra il modo in cui gli esseri umani danno istruzioni a un calcolatore e quello in cui il calcolatore vede queste istruzioni.

Anche le istruzioni sono memorizzate nel calcolatore come una sequenza di segnali elettrici alto/basso e possono quindi essere viste come numeri. In effetti, ciascuna parte di un'istruzione può essere vista come un numero e l'istruzione vera e propria si può ottenere disponendo questi numeri uno di fianco all'altro. I 32 registri dei RISC-V sono semplicemente individuati dal loro numero progressivo da 0 a 31.

Traduzione di un'istruzione assembler RISC-V nell'istruzione macchina corrispondente

ESEMPIO

Facciamo il prossimo passo nella definizione del linguaggio RISC-V mediante un esempio. Mostreremo nel vero linguaggio RISC-V la versione dell'istruzione rappresentata simbolicamente dall'espressione:

```
add x9, x20, x21
```

dapprima come sequenza di numeri decimali e poi come sequenza di numeri binari.

SOLUZIONE

La rappresentazione decimale è:

0	21	20	0	9	51

Ciascuna delle parti che costituiscono un'istruzione è chiamato *campo* (*field*). La combinazione del primo, del quarto e del sesto campo (che in questo caso contengono rispettivamente 0, 0 e 51) indica al processore RISC-V che questa istruzione esegue una somma. Il secondo campo indica il numero del registro che contiene il primo operando (21 = x21), il terzo campo il secondo operando della somma (20 = x20) e il quinto campo contiene il numero del registro in cui viene memorizzata la somma (9 = x9). Questa istruzione dunque somma il contenuto dei registri x21 e x20 e pone il risultato in x9.

(continua)

(continua)

Questa stessa istruzione può anche essere rappresentata come sequenza di campi contenenti numeri binari invece che decimali:

0000000	10101	10100	000	01001	0110011
7 bit	5 bit	5 bit	3 bit	5 bit	7 bit

Formato dell'istruzione: un modo di rappresentare un'istruzione attraverso un insieme di campi di numeri binari.

Linguaggio macchina: rappresentazione binaria utilizzata per la comunicazione all'interno dei calcolatori.

Numeri esadecimali: numeri in base 16.

Questa scomposizione in campi di un'istruzione è chiamata **formato dell'istruzione**. Come si può notare contando il numero di bit, l'istruzione RISC-V richiede esattamente 32 bit, una parola. In accordo con il principio di progetto per cui la semplicità favorisce la regolarità, tutte le istruzioni RISC-V sono lunghe 32 bit.

Per distinguere la versione numerica delle istruzioni da quella in linguaggio assembler, la prima viene chiamata **linguaggio macchina**; una sequenza di istruzioni in linguaggio macchina viene definita *codice macchina*.

Da quanto abbiamo visto, sembrerebbe che si debbano leggere e scrivere lunghe e noiose stringhe di numeri binari; per evitare ciò si ricorre all'uso di una base di numerazione più alta di quella binaria, che possa però essere convertita facilmente in binario. Dato che quasi tutti i calcolatori hanno una dimensione dei dati multipla di 4, i **numeri esadecimali** (in base 16) sono molto diffusi. Inoltre, essendo la base 16 una potenza di 2, si può facilmente convertire un numero binario in esadecimale raggruppando le cifre binarie a gruppi di 4 e sostituendo a ciascun gruppo una singola cifra esadecimale, e viceversa. La **Figura 2.4** mostra la conversione da esadecimale a binario (e viceversa) dei primi numeri interi.

Poiché utilizzeremo spesso numeri in basi diverse, per evitare di fare confusione i decimali saranno indicati con il pedice *dec*, i binari con il pedice *due* e gli esadecimali con il pedice *esa*. (I numeri senza pedici vanno considerati in base dieci.) A questo proposito, sia C che Java utilizzano la notazione 0x*nnnn* per identificare un numero esadecimale.

Conversione da binario a esadecimale e viceversa

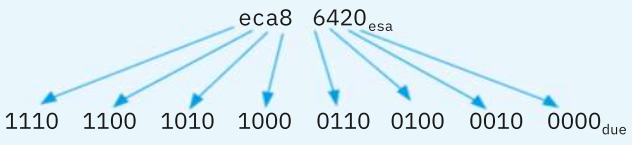
ESEMPIO

Convertire i seguenti numeri, uno esadecimale su 8 cifre e l'altro binario su 32 bit, rispettivamente in base 2 e 16:

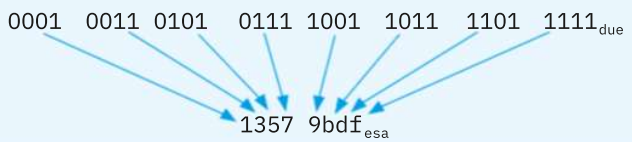
$\texttt{eca8 6420}_{esa}$
$\texttt{0001 0011 0101 0111 1001 1011 1101 1111}_{due}$

SOLUZIONE

Utilizzando la Figura 2.4, la risposta è una semplice trasformazione diretta dei simboli:



La conversione da base 2 a base 16 sarà:



Esadecimale	Binario	Esadecimale	Binario	Esadecimale	Binario	Esadecimale	Binario
0_{esa}	0000_{due}	4_{esa}	0100_{due}	8_{esa}	1000_{due}	c_{esa}	1100_{due}
1_{esa}	0001_{due}	5_{esa}	0101_{due}	9_{esa}	1001_{due}	d_{esa}	1101_{due}
2_{esa}	0010_{due}	6_{esa}	0110_{due}	a_{esa}	1010_{due}	e_{esa}	1110_{due}
3_{esa}	0011_{due}	7_{esa}	0111_{due}	b_{esa}	1011_{due}	f_{esa}	1111_{due}

Figura 2.4 Tabella di conversione tra binario ed esadecimale. È sufficiente sostituire una cifra esadecimale con le corrispondenti quattro cifre binarie e viceversa. Se la lunghezza del numero binario non è un multiplo di 4, si procede da destra a sinistra.

Campi delle istruzioni RISC-V

Per rendere più facile la trattazione, ai diversi campi delle istruzioni RISC-V viene associato un nome:

funz7	rs2	rs1	funz3	rd	codop
7 bit	5 bit	5 bit	3 bit	5 bit	7 bit

Il significato del nome dato a ciascun campo è il seguente:

- *codop*: operazione base dell'istruzione, tradizionalmente chiamata **codice operativo** o **codop** (da *operating code*);
- *rd*: registro destinazione: riceve il risultato dell'operazione;
- *funz3*: un codice operativo aggiuntivo;
- *rs1*: registro contenente il primo operando sorgente;
- *rs2*: registro contenente il secondo operando sorgente;
- *funz7*: un codice operativo aggiuntivo.

Codice operativo (codop): il campo di un'istruzione che specifica l'operazione e il formato dell'istruzione stessa.

Può nascere un problema quando un'istruzione richiede campi di dimensioni maggiori rispetto a quelle sopra specificate. Per esempio, l'istruzione load richiede di specificare due registri e una costante: se la costante venisse inserita in uno dei campi da 5 bit, non potrebbe superare il valore di $2^5 - 1$, cioè 31.

Questa costante viene utilizzata di solito per selezionare un elemento specifico all'interno di vettori o altri tipi di strutture dati e spesso deve assumere valori molto maggiori di 31: un campo da 5 bit risulterebbe troppo piccolo per essere utile.

Nasce dunque un conflitto fra il desiderio di mantenere la stessa lunghezza per tutte le istruzioni e quello di avere un unico formato. Ciò conduce all'ultimo principio della progettazione dell'hardware:

> *Principio di progettazione numero 3*: un buon progetto richiede buoni compromessi.

Il compromesso scelto dai progettisti del RISC-V è quello di mantenere uguale la lunghezza di tutte le istruzioni, predisponendo formati differenti per tipi di istruzioni diverse. Per esempio, il formato descritto precedentemente è chiamato di *tipo R* (R sta per registro).

Un secondo tipo di formato è detto di *tipo I* (I sta per immediato) e viene utilizzato dalle operazioni aritmetiche nelle quali un operando è una costante, compreso `addi`, e dalle istruzioni di trasferimento dati con la memoria. I campi del formato I sono:

immediato	rs1	funz3	rd	codop
12 bit	5 bit	3 bit	5 bit	7 bit

Il campo immediato a 12 bit viene interpretato come un valore in complemento a due, per cui rappresenta gli interi compresi tra -2^{11} e $+2^{11} - 1$. Quando il formato di tipo I viene utilizzato per le istruzioni di trasferimento dati, il campo immediato rappresenta l'offset in byte, per cui le istruzioni di caricamento di una parola possono indirizzare tutte le parole in un intervallo di $\pm 2^{11}$, cioè 2048 byte ($\pm 2^8$ cioè 512 parole) rispetto all'indirizzo contenuto nel registro di base rd. Osserviamo che avere più di 32 registri sarebbe difficile con questo formato di istruzioni, poiché occorrerebbe almeno un bit in più per ognuno dei campi rd e rs1, e sarebbe quindi più difficile far stare tutto in una sola parola.

Si consideri l'istruzione di load esaminata a pagina 66:

```
lw x9, 32(x22) // il registro temporaneo x9
               // assume il valore di A[8]
```

In questo caso il numero 22 (che rappresenta il registro `x22`) viene posto nel campo rs1, il numero 32 nel campo immediato e il numero 9 (che rappresenta `x9`) nel campo rd. Occorre definire un formato anche per l'istruzione di trasferimento in memoria di una parola, `sw`, che ha bisogno di due registri sorgenti (uno per l'indirizzo base e l'altro per il dato da memorizzare) e un valore immediato per l'offset dell'indirizzo. I campi del formato di tipo S sono:

immediato[11:5]	rs2	rs1	funz3	Immediato[4:0]	codop
7 bit	5 bit	5 bit	3 bit	5 bit	7 bit

Il valore immediato su 12 bit nel formato di tipo S è suddiviso in due campi, che forniscono rispettivamente i 5 bit meno significativi e i 7 bit più significativi. I progettisti del RISC-V hanno scelto questa soluzione perché consente di mantenere i campi rs1 e rs2 nella stessa posizione in tutti i formati di istruzioni. (La Figura 4.14 mostra come questa suddivisione semplifichi l'hardware). Avere dei formati delle diverse istruzioni più simili possibili tra loro riduce la complessità. Analogamente, il codice operativo e il campo funct3 hanno la stessa dimensione e si trovano nella stessa posizione in tutte le istruzioni.

Si possono distinguere i due formati in base al valore contenuto nel codice operativo: a ciascun formato è assegnato un insieme di valori del campo codop, in modo tale che l'hardware sappia esattamente come deve trattare i rimanenti bit dell'istruzione. La **Figura 2.5** mostra il contenuto di ciascun campo delle istruzioni RISC-V introdotte finora.

Istruzione	Formato	funz7	rs2	r1	funz3	rd	codop
`add` (somma)	R	0000000	reg	reg	000	reg	0110011
`sub` (sottrazione)	R	0100000	reg	reg	000	reg	0110011
Istruzione	**Formato**	**immediato**		**r1**	**funz3**	**rd**	**codop**
`addi` (somma immediata)	I	costante		reg	000	reg	0010011
`lw` (load parola)	I	indirizzo		reg	010	reg	0000011
Istruzione	**Formato**	**immediato**	**rs2**	**r1**	**funz3**	**immediato**	**codop**
`sw` (store parola)	S	indirizzo	reg	reg	010	indirizzo	0100011

Figura 2.5 Codifica delle istruzioni RISC-V. In questa tabella "reg" indica il numero di un registro ed è compreso tra 0 e 31, e "indirizzo" è un indirizzo su 12 bit o una costante. I campi funz3 e funz7 agiscono come codici operativi aggiuntivi.

Traduzione in linguaggio macchina delle istruzioni assembler del RISC-V

ESEMPIO

È ora possibile mostrare un esempio completo, partendo da ciò che scrive il programmatore fino ad arrivare a ciò che viene eseguito dal calcolatore. Se `x10` contiene l'indirizzo base del vettore `A` e `x21` corrisponde a `h`, l'istruzione di assegnamento:

```
A[30] = h + A[30] + 1;
```

viene compilata in:

```
lw   x9, 120(x10)  // il registro temporaneo x9
                   // assume il valore A[30]
add  x9, x21, x9   // il registro temporaneo x9
                   // assume il valore h + A[30]
addi x9, x9, 1     // il registro temporaneo x9
                   // assume il valore h + A[30] + 1
sw   x9, 120(x10)  // memorizza h + A[30] + 1 nuovamente
                   // in A[30]
```

Qual è il codice macchina RISC-V corrispondente a queste quattro istruzioni?

SOLUZIONE

Per comodità conviene inizialmente rappresentare le istruzioni in linguaggio macchina nel formato decimale. Dalla Figura 2.5 possiamo determinare queste quattro istruzioni in linguaggio macchina:

immediato	rs1	funz3	rd	codop
120	10	3	9	3

funz7	rs2	rs1	funz3	rd	codop
0	9	21	0	9	51

immediato	rs1	funz3	rd	codop
1	9	0	9	19

immediato[11:5]	rs2	rs1	funz3	immediato[4:0]	codop
3	9	10	2	24	35

L'istruzione `lw` è identificata dal numero 3 (Figura 2.5), contenuto nel campo codop e dal numero 2 contenuto nel campo funz3. Il registro base 10 è specificato nel campo rs1 e il registro destinazione 9 è specificato nel campo rd. L'offset per selezionare `A[30]`($120 = 300 \times 4$) si trova nel campo immediato.

La successiva istruzione di somma, `add`, è identificata dal numero 51 nel campo codop, dal numero 0 nel campo funz3 e dal numero 0 nel campo funz7. I tre registri degli operandi (9, 21 e 9) sono contenuti rispettivamente nei campi rd, rs1 e rs2.

L'istruzione seguente, `addi`, è identificata dal numero 19 contenuto nel campo codop e dal numero 0 contenuto nel campo funz3. I registri degli operandi (9 e 9) sono specificati nei campi rd e rs1 e la costante, l'addendo 1, si trova nel campo immediato.

(continua)

(continua)

L'istruzione `sw` è identificata dal numero 35 contenuto nel campo codop e dal numero 2 contenuto nel campo funz3. I registri degli operandi (9 e 10) sono specificati rispettivamente nei campi rs2 e rs1. L'offset dell'indirizzo, 120, è suddiviso tra i due campi immediato. Dato che il campo immediato con i bit più significativi contiene i bit dal 5 all'11, possiamo decomporre l'offset 120 dividendolo per 2^5. La parte con i bit più significativi conterrà il quoziente della divisione, cioè 3, mentre la parte con i bit meno significativi conterrà il resto cioè 24.

Dato che $120_{dec} = 0000\ 0111\ 1000_{due}$, la rappresentazione binaria equivalente delle quattro istruzioni, definite mediante numeri decimali, è:

immediato	rs1	funz3	Rd	codop
00001111000	01010	010	01001	0000011

funz7	rs2	rs1	funz3	rd	codop
0000000	01001	10101	000	01001	0110011

immediato	rs1	funz3	rd	codop
000000000001	01001	000	01001	0010011

immediato[11:5]	rs2	rs1	funz3	Immediato[4:0]	codop
0000011	01001	01010	010	11000	0100011

Approfondimento. I programmatori assembler in linguaggio RISC-V non sono obbligati a utilizzare `addi` quando lavorano con le costanti. Il programmatore può semplicemente scrivere `add` e l'assemblatore genera il codice operativo e il formato adeguato all'istruzione a seconda che gli operandi siano tutti contenuti nei registri (tipo R) oppure uno di essi sia una costante (tipo I); Paragrafo 2.12.

Quando introduciamo il linguaggio assembler e lo confrontiamo con il linguaggio macchina utilizziamo esplicitamente i diversi nomi delle varianti delle istruzioni RISC-V con i loro codici operativi e formati, perché riteniamo che sia più chiaro per il lettore.

Approfondimento. Anche se il RISC-V contiene sia le istruzioni di `add` che di `sub`, non contiene l'istruzione di `subi` associata alla `addi`. Questo perché il numero contenuto nel campo immediato può essere rappresentato come intero in complemento a due, per cui `addi` può essere utilizzata per sottrarre una costante.

Interfaccia hardware/software

Il desiderio di avere tutte le istruzioni della stessa dimensione è in conflitto con il desiderio di avere un numero di registri grande a piacere. L'aumento del numero di registri richiede l'aggiunta di almeno un bit all'interno del formato dell'istruzione per tutti i campi che contengono un numero di registro. Dati questi vincoli e il principio di progettazione per il quale più piccolo implica più veloce, la maggior parte delle istruzioni, oggi, ha un numero di registri variabile tra 16 e 32.

La **Figura 2.6** riassume le parti di linguaggio macchina RISC-V introdotte in questo paragrafo. Come si vedrà nel Capitolo 4, la somiglianza della rappresentazione binaria di istruzioni correlate consente di semplificare la progettazione dell'hardware. Queste somiglianze sono un altro esempio della regolarità dell'architettura RISC-V.

Istruzioni tipo-R	funz7	rs2	r1	funz3	rd	codop	Esempio
`add` (somma)	0000000	00011	00010	000	00001	0110011	add x1, x2, x3
`sub` (sottrazione)	0100000	00011	00010	000	00001	0110011	sub x1, x2, x3
Istruzioni tipo-I	**immediato**		**r1**	**funz3**	**rd**	**codop**	**Esempio**
`addi` (somma immediata)	001111101000		00010	000	00001	0010011	addi x1,x2,1000
`lw` (load parola)	001111101000		00010	010	00001	0000011	lw x1, 1000 (x2)
Istruzioni tipo-S	**immediato**	**rs2**	**r1**	**funz3**	**immediato**	**codop**	**Esempio**
`sw` (store parola)	0011111	00001	00010	010	01000	0100011	sw x1, 1000 (x2)

Figura 2.6 L'architettura RISC-V descritta fino al Paragrafo 2.5. I tre formati delle istruzioni RISC-V visti finora sono R, I e S. Il formato R ha due registri sorgenti per gli operandi e un registro destinazione. Il formato I sostituisce uno dei due registri sorgente con la parte meno significativa del campo *immediato* su 12 bit. Il formato S ha due operandi sorgente e un campo immediato su 12 bit, ma non ha un registro destinazione. Il campo immediato delle istruzioni di tipo S viene suddiviso in due parti: i bit 11–5 sono contenuti nel campo immediato più a sinistra e i bit 4–0 nel secondo campo da destra.

QUADRO D'INSIEME

Oggi i calcolatori sono costruiti secondo due principi chiave:

1. Le istruzioni vengono rappresentate come numeri.
2. I programmi sono contenuti in memoria e possono essere letti e scritti, proprio come dati.

Questi principi esprimono il concetto di programma memorizzato (*stored program*): un'invenzione fondamentale per l'informatica. La **Figura 2.7** mostra la portata di questo concetto; nel caso specifico, la memoria contiene il codice sorgente di un programma per l'elaborazione di testi, il corrispondente codice macchina compilato, il testo su cui il programma compilato sta lavorando e anche il compilatore che ha generato il codice macchina. Una conseguenza del fatto che le istruzioni sono rappresentate come numeri è che i programmi possono essere trasmessi attraverso file di numeri binari. L'implicazione commerciale è che un calcolatore può ereditare del codice già pronto, purché le istruzioni che costituiscono il codice siano compatibili con il suo insieme di istruzioni. Questa "compatibilità binaria" ha spesso portato l'industria a sviluppare software per un numero ristretto di architetture di insiemi di istruzioni. ■

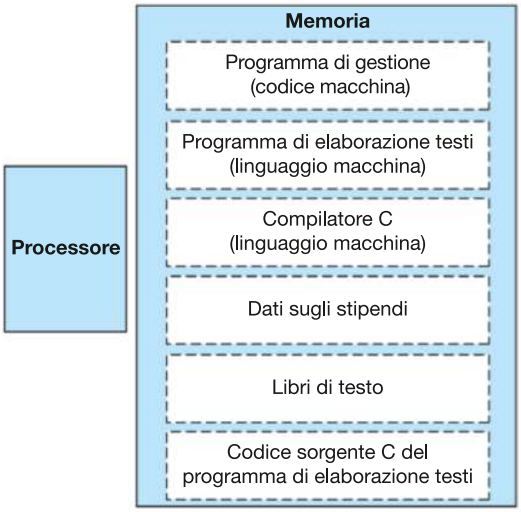


Figura 2.7 Concetto di programma memorizzato. La memorizzazione dei programmi consente a un calcolatore che gestisce bilanci di trasformarsi improvvisamente in un calcolatore che serve a un autore per scrivere un libro. Il cambiamento avviene semplicemente caricando in memoria programmi e dati e poi specificando al calcolatore l'indirizzo di memoria da cui fare partire l'esecuzione. Trattare i programmi e i dati allo stesso modo semplifica sia l'hardware della memoria sia il software dei sistemi operativi. Nel caso specifico, la stessa tecnologia di memoria utilizzata per i dati si può applicare anche ai programmi; programmi quali i compilatori, per esempio, possono tradurre il codice scritto in una notazione molto più comprensibile all'uomo in un codice che il calcolatore è in grado di capire.

Autovalutazione

Quale istruzione RISC-V è rappresentata sotto? Selezionare una delle quattro risposte riportate.

funz7	rs2	rs1	funz3	rd	codop
32	9	10	000	11	51

1. `add x9, x10, x11`
2. `add x11, x9, x10`
3. `sub x11, x10, x9`
4. `sub x11, x9, x10`

Se una persona ha 40 anni in notazione decimale, quale sarà la sua età espressa in notazione esadecimale?

2.6 Operazioni logiche

"Al contrario", continuò Tweedledee, "se fosse così, potrebbe essere; e se così fosse, sarebbe; ma poiché non è, non è. Questa è logica".

Lewis Carrol, *Alice nel paese delle meraviglie*, 1865

Sebbene i primi calcolatori effettuassero operazioni su parole intere, divenne presto evidente che era utile riuscire a operare anche su gruppi di bit o addirittura sui singoli bit di una parola. Esaminare i caratteri all'interno di una parola, dove ciascun carattere è memorizzato su 8 bit, è uno degli esempi di questo tipo di operazioni (Paragrafo 2.9). Il risultato fu che vennero aggiunte ai linguaggi di programmazione e alle architetture degli insiemi di istruzioni operazioni che permettevano di semplificare, per esempio, l'inserimento e la lettura di gruppi di bit all'interno di una parola. Queste istruzioni sono chiamate *operazioni logiche*. Nella **Figura 2.8** vengono riportate le *operazioni logiche* in C, Java e nel RISC-V.

La prima tipologia di queste operazioni è detta *shift* (scorrimento) e consiste nello spostare tutti i bit di una parola a sinistra o a destra, riempiendo i bit vuoti con degli zeri. Per esempio, supponiamo che il registro `x19` contenga:

$$00000000\ 00000000\ 00000000\ 00001001_{due} = 9_{dec}$$

ed eseguiamo l'istruzione di shift a sinistra di 4; il numero ottenuto sarà il seguente:

$$00000000\ 00000000\ 00000000\ 10010000_{due} = 144_{dec}$$

L'operazione duale dello shift a sinistra è lo shift a destra. Il nome delle due istruzioni RISC-V di shift è *shift a sinistra logico immediato* (`slli`, *shift left logical immediate*) e *shift a destra logico immediato* (`srli`, *shift right logical*

Operazioni logiche	Operatori C	Operatori Java	Istruzioni RISC-V
Shift a sinistra	<<	<<	`sll, slli`
Shift a destra	>>	>>>	`srl, srli`
Shift a destra aritmetico	>>	>>	`sra, srai`
AND bit a bit	&	&	`and, andi`
OR bit a bit	|	|	`or, ori`
XOR bit a bit	^	^	`xor, xori`
NOT bit a bit	~	~	`xori`

Figura 2.8 Operatori logici C e Java e le corrispondenti istruzioni RISC-V. Un modo per implementare l'operatore NOT in RISC-V è utilizzare XOR con uno degli operandi impostato a tutti uni (FFFF FFFF FFFF $FFFF_{esa}$).

immediate). L'istruzione seguente permette di eseguire l'operazione riportata in precedenza. Si suppone che l'istruzione legga l'operando dal registro `x19` e metta il risultato nel registro `x11`:

```
slli x11, x19, 4 // reg x11 = reg x19 << 4 bit
```

Queste istruzioni di shift utilizzano il formato di tipo I. Dato che non serve fare scorrere i bit di un registro a 32 bit per più di 31 posizioni, solamente i 6 bit meno significativi del campo immediato del formato di tipo I vengono effettivamente utilizzati. I rimanenti 6 bit vengono utilizzati come un campo aggiuntivo di codice operativo, funz7.

funz7	immediato	rs1	funz3	rd	codop
0	4	19	1	11	19

L'istruzione `slli` prevede 19 nel campo codop, 11 nel campo rd, 1 nel campo funz3, 19 nel campo rs1, 4 nel campo immmediato e 0 nel campo funz7.

L'operazione di shift logico a sinistra fornisce un'ulteriore funzionalità; infatti, lo scorrimento di un numero a sinistra di i cifre produce lo stesso risultato di una moltiplicazione per 2^i, esattamente come lo scorrimento di un numero decimale a sinistra di i cifre è equivalente a una moltiplicazione per 10^i. Per esempio, la precedente operazione di `slli` produce uno shift di 4 posizioni che è lo stesso risultato che si otterrebbe moltiplicando il contenuto del registro per 2^4 (cioè 16): la prima sequenza di bit rappresentava il numero 9 e $9 \times 16 = 144$ è il numero rappresentato nella seconda sequenza di bit.

Il RISC-V fornisce anche un terzo tipo di operazione di scorrimento: *scorrimento a destra aritmetico immediato* (`srai`), Questa variante è simile a `srli` tranne che invece di riempire con degli zero i bit che si liberano a sinistra, questi bit vengono riempiti copiando il bit del segno. Il RISC-V fornisce anche una variante per ciascuna delle tre operazioni di scorrimento che prendono il numero di posizioni di scorrimento da un registro, invece che dal campo immediato: `sll`, `srl` e `sra`.

Un'altra utile operazione logica, che permette di isolare i campi di una parola, è **AND** (è scritta in maiuscolo per evitare la confusione con la congiunzione inglese). L'operazione di AND è un'operazione bit a bit che scrive un 1 nel risultato solamente se entrambi gli operandi hanno un 1 in quella stessa posizione. Per esempio, supponiamo che il registro `x11` contenga:

`00000000 00000000 00001101 11000000`$_{due}$

e il registro `x10` contenga:

`00000000 00000000 00111100 00000000`$_{due}$

dopo l'esecuzione della seguente istruzione RISC-V:

```
and x9, X10, X11, // reg x9 = reg x10 & reg x11
```

il valore contenuto nel registro `x9` sarebbe:

`00000000 00000000 00001100 00000000`$_{due}$

Come si può notare, l'operazione di AND bit a bit può essere utilizzata per forzare a 0 i bit di una parola fornendo in input all'AND una parola che contiene 0 in quelle posizioni, dato che essa "nasconde" alcuni bit (questa parola viene tradizionalmente chiamata *maschera*).

AND: un'operazione logica bit a bit su due operandi, che restituisce 1 se entrambi gli operandi sono uguali a 1.

Per inserire un 1 in questo mare di zeri esiste un'operazione bit a bit duale rispetto alla AND, chiamata **OR**. Quest'ultima è un'operazione bit a bit che inserisce un 1 nel risultato laddove *almeno uno dei due operandi* sia uguale a 1. Per aiutarci a capire meglio, supponiamo che i registri `x10` e `x11` contengano

OR: un'operazione logica bit a bit su due operandi, che restituisce 1 se *almeno* uno dei due operandi è uguale a 1.

 © 978-88-08-19966-9

gli stessi valori visti in precedenza; il risultato dell'esecuzione dell'istruzione RISC-V:

```
or x9, x10, x11 // reg x9 = reg x10 | reg x11
```

verrà scritto nel registro x9 e sarà:

$$00000000\ 00000000\ 00111101\ 11000000_{due}$$

NOT: un'operazione logica bit a bit su un singolo operando che inverte i bit: sostituisce cioè tutti gli 1 con 0, e tutti gli 0 con 1.

L'ultima operazione logica è la negazione: l'operazione di **NOT** prende un operando e pone un 1 nel risultato laddove nell'operando era presente uno 0 e viceversa. Utilizzando la notazione introdotta in precedenza, l'operatore NOT calcola $\bar{x}$.

XOR: un'operazione logica bit a bit su due operandi, che calcola l'OR esclusivo dei due operandi. Restituisce cioè 1 solo se i valori dei due operandi sono diversi.

Per mantenere il formato dell'operazione a tre operandi, i progettisti RISC-V hanno deciso di includere l'istruzione **XOR** (OR esclusivo) invece della negazione semplice (NOT). Dato che XOR produce uno 0 quando due bit sono uguali e 1 altrimenti, si può ottenere l'operatore NOT mediante `xor` di un numero con 111…111.

Supponiamo che il contenuto del registro x10 sia quello degli esempi precedenti e che il registro x12 contenga il valore 0. Il risultato dell'esecuzione dell'istruzione RISC-V:

```
xor x9, x10, x12 // reg x9 = reg x10 ^ reg x12
```

verrà scritto nel registro x9 e sarà:

$$00000000\ 00000000\ 00110001\ 11000000_{due}$$

La Figura 2.8 illustra la relazione tra gli operatori C e Java e le istruzioni RISC-V. Le costanti sono molto utili anche nelle operazioni logiche di AND e OR, come lo erano nelle operazioni aritmetiche. Il RISC-V fornisce quindi anche le istruzioni di *AND immediato* (`andi`), di *OR immediato* (`ori`) e di *OR esclusivo immediato* (`xori`).

Approfondimento. Il C consente di definire all'interno di una parola *campi di bit* o *campi*, che consentono di impaccare degli oggetti all'interno della parola e che sono uguali ai campi esposti all'esterno dalle interfacce per esempio dei dispositivi di I/O. Tutti i campi devono essere contenuti all'interno di una parola singola. I campi sono degli interi senza segno che possono essere piccoli fino a occupare 1 solo bit. I compilatori C inseriscono ed estraggono i diversi campi utilizzando le istruzioni logiche che in RISC-V sono: `andi`, `ori`, `slli` e `srli`.

Autovalutazione

Quali operazioni possono isolare un campo all'interno di una parola?

1. AND.
2. Uno shift a sinistra seguito da uno shift a destra.

L'utilità di un calcolatore automatico sta nella possibilità di utilizzare ripetutamente una sequenza di istruzioni, un numero di volte che dipende dal risultato dei calcoli. […] Questa scelta può dipendere dal segno di un numero (considerando lo zero come un numero positivo per il calcolatore). Di conseguenza, introduciamo una [istruzione] ([l'istruzione] di trasferimento condizionato) che, a seconda del segno di un dato numero, provocherà l'esecuzione di una tra due possibili procedure.

Burks, Goldstine e von Neumann, 1947.

2.7 Istruzioni per prendere decisioni

Ciò che contraddistingue un calcolatore da una semplice calcolatrice è la sua capacità di prendere delle decisioni: in base ai dati di ingresso e ai valori calcolati durante l'elaborazione, possono essere eseguite istruzioni diverse. Nei linguaggi di programmazione un processo di decisione è comunemente rappresentato con il costrutto *if*, a volte seguito dal costrutto *go to* seguito da un'etichetta. Il linguaggio assembler RISC-V contiene due istruzioni per

implementare un processo di decisione, simili a un costrutto *if* seguito da *go to*. La prima istruzione è:

```
beq rs1, rs2, L1
```

e significa: vai all'istruzione etichettata con `L1` se il valore contenuto nel registro `rs1` è uguale al valore contenuto nel registro `rs2`; il codice mnemonico `beq` significa *branch if equal* (salta se uguale). La seconda istruzione è:

```
bne rs1, rs2, L1
```

e significa: vai all'istruzione etichettata con `L1` se il valore contenuto nel registro `rs1` *non* è uguale al valore contenuto nel registro `rs2`; il codice mnemonico `bne` indica *branch if not equal* (salta se non uguale). Queste due istruzioni prendono tradizionalmente il nome di **salti condizionati** (*conditional branches*).

Salto condizionato: istruzione che richiede il confronto fra due valori e che consente il trasferimento del controllo a un altro indirizzo del programma a seconda del risultato del confronto.

Compilazione di un costrutto di tipo *if-then-else* mediante salti condizionati

ESEMPIO

Nel seguente frammento di codice C ci sono cinque variabili: `f`, `g`, `h`, `i` e `j`. Supponiamo che corrispondano ai cinque registri da `x19` a `x23`. Quale sarà il codice RISC-V prodotto dalla compilazione del seguente codice C?

```
if (i == j) f = g + h; else f = g - h;
```

SOLUZIONE

La **Figura 2.9** mostra il diagramma di flusso dei passaggi logici che il codice RISC-V dovrà rispettare; la prima espressione in C confronta le variabili contenute nei due registri per vedere se sono uguali. L'intuito ci suggerisce di utilizzare l'istruzione `beq` se i e j sono uguali. In generale, però, il codice sarà più efficiente se viene utilizzata la condizione opposta e saltiamo le istruzioni seguenti se le due variabili *non* sono uguali (`bne`). Questo è il codice risultante:

```
bne x22, x23, Else // vai a Else se i ≠ j
```

L'istruzione successiva in C è un assegnamento che svolge una singola operazione e, se tutti gli operandi si trovano all'interno dei registri, può essere tradotta in una singola istruzione RISC-V:

```
add x19, x20, x21 // f = g + h (saltata se i ≠ j)
```

Ora è necessario spostarsi alla fine dell'istruzione *if*. Questo esempio ci consente di introdurre un altro tipo di salto, spesso chiamato *salto incondizionato*: l'istruzione corrispondente indica al calcolatore di eseguire sempre il salto. Un modo per esprimere un salto incondizionato in RISC-V è utilizzare un salto condizionato a una condizione che è sempre vera:

```
beq x0, x0, Esci // se 0 == 0, vai a Esci
```

L'assegnamento che si trova nel segmento di codice relativo alla clausola *else* del costrutto *if* può anch'esso essere tradotto in un'unica istruzione RISC-V; occorre solo attaccare l'etichetta `Else` a questa istruzione. Nell'esempio viene mostrata anche l'etichetta `Esci`, posizionata dopo questa istruzione, che segna la fine del costrutto *if-then-else* tradotto in assembler:

```
Else: sub x19, x20, x21 // f = g - h (saltata se i = j)
Esci:
```

 © 978-88-08-19966-9

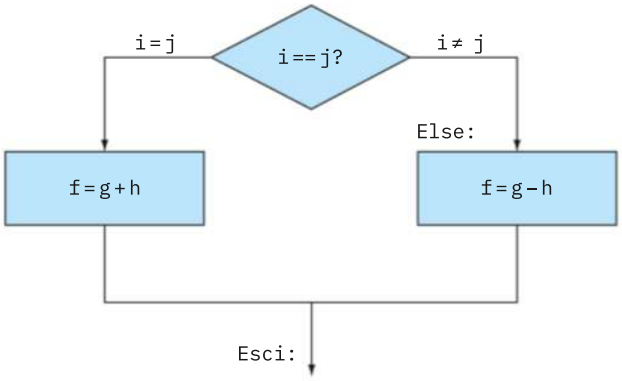


Figura 2.9 Schema delle operazioni eseguite dal costrutto *if*. Il rettangolo a sinistra corrisponde al *then* del costrutto *if*, quello a destra all'*else*.

Si noti che l'assembler evita al compilatore o al programmatore il noioso compito di calcolare gli indirizzi dei salti, proprio come nelle istruzioni di trasferimento dati viene evitato il calcolo dell'indirizzo della memoria (Paragrafo 2.12).

Interfaccia hardware/software

I compilatori creano frequentemente salti ed etichette che non appaiono nel linguaggio di programmazione. Uno dei vantaggi dei linguaggi di programmazione ad alto livello è che permettono di evitare di scrivere esplicitamente le etichette e i salti. Questa è una delle ragioni per cui programmare a questo livello è più veloce.

Cicli

Prendere una decisione è importante sia per scegliere tra due alternative, come avviene nei costrutti *if*, sia per iterare un certo calcolo più volte, come avviene nei cicli. Le stesse istruzioni assembler sono utilizzate in entrambi i casi.

Compilazione di un ciclo while scritto in C

ESEMPIO

Il seguente codice rappresenta un tipico ciclo in C:

```
while (salva[i] == k)
      i += 1;
```

Supponiamo che `i` e `k` corrispondano ai registri `x22` e `x24` e che l'indirizzo base del vettore `salva` sia contenuto in `x25`. Qual è il codice assembler RISC-V corrispondente a questo frammento di codice C?

SOLUZIONE

Il primo passo consiste nel caricare `salva[i]` in un registro temporaneo. Per poter effettuare questa operazione, però, è necessario prima determinare il suo indirizzo in memoria. E prima di sommare `i` all'indirizzo di base del vettore `salva` per comporre l'indirizzo di `salva[i]`, dobbiamo moltiplicare l'indice `i` per 4 per ottenere l'indirizzamento al byte. Fortunatamente possiamo utilizzare l'operazione di shift logico a sinistra per moltiplicare per 2^2, cioè 4 (Paragrafo 2.6). Inoltre, è necessario

(continua)

(continua)

aggiungere l'etichetta `Ciclo` alla prima istruzione, in modo che sia possibile ritornare indietro al termine del ciclo:

```
Ciclo: slli x10, x22, 2  // registro temporaneo
                         // x10 = i * 4
```

Per ottenere l'indirizzo di `salva[i]` è sufficiente sommare `x10` all'indirizzo base di `salva` contenuto in `x25`:

```
add x10, x10, x25  // x10 = indirizzo di salva[i]
```

A questo punto è possibile usare questo indirizzo per caricare il valore di `salva[i]` in un registro temporaneo:

```
lw x9, 0(x10)      // registro temporaneo
                   // x9 = salva[i]
```

L'istruzione successiva esegue il controllo di uscita dal ciclo: si esce dal ciclo se `salva[i]` ≠ `k`:

```
bne x9, x24, Esci  // vai a Esci se salva[i] ≠ k
```

L'istruzione successiva somma 1 a `i`:

```
addi x22, x22, 1   // i = i + 1
```

Alla fine del ciclo c'è un salto all'indietro al test del *while* che si trova all'inizio del ciclo. Per completare la scrittura del codice assembler basta aggiungere l'etichetta `Esci` alla fine del codice:

```
beq x0, x0, Ciclo  // vai a Ciclo
Esci:
```

Nel Paragrafo 2.25 è presente un'ottimizzazione di questa sequenza.

Interfaccia hardware/software

Le sequenze di istruzioni che terminano con un salto condizionato (*branch*) sono talmente fondamentali per la compilazione che viene dato loro un nome particolare: **blocchi di base**. Un blocco di base è una sequenza di istruzioni che non contiene né istruzioni di salto (con la possibile eccezione della fine) né etichette di destinazione di salti (con la possibile eccezione dell'inizio). Una delle prime fasi della compilazione consiste nel suddividere il programma nei blocchi di base da cui è composto.

Blocco di base: sequenza di istruzioni che non contiene né diramazioni né istruzioni di salto, a eccezione eventualmente dell'ultima istruzione, e non contiene etichette di destinazione di salti, a eccezione eventualmente della prima istruzione.

I test per verificare l'uguaglianza o la disuguaglianza fra due variabili sono probabilmente i più diffusi, ma esistono molte altre relazioni tra due numeri. Per esempio, all'interno di un ciclo *for* si può volere controllare se la variabile indice è minore di 0. L'insieme dei confronti possibili prevede: minore di (<), minore di o uguale (≤), maggiore di (>), maggiore di o uguale (≥), uguale (=) e non uguale (≠).

La comparazione di stringhe di bit deve prevedere sia i numeri dotati di segno sia quelli senza: se una stringa di bit con un 1 nel bit più significativo rappresenta un numero negativo, quel numero sarà minore di qualsiasi numero positivo, il quale dovrà avere uno 0 nel bit più significativo. Invece, se la stringa di bit rappresenta un numero senza segno e il bit più significativo è uguale a 1, quel numero sarà *maggiore* di qualsiasi altro numero con uno 0 nel bit più significativo (sfrutteremo a breve questo doppio significato del bit più significativo per rendere più semplice il controllo sullo sforamento dei limiti

dei vettori). Il RISC-V offre istruzioni che gestiscono queste due possibilità. Queste istruzioni hanno la stessa forma di `beq` e `bne`, ma eseguono il confronto in modo diverso. L'istruzione di salta se minore (`blt`, *branch if less than*) confronta il contenuto nei registri rs1 e rs2 e prende il salto quando il valore contenuto in rs1 è inferiore, trattando il contenuto dei due registri come numeri in complemento a due. L'istruzione di salta se maggiore o uguale (`bge`, *branch if greater than or equal*) prende il salto nel caso opposto, cioè quando il valore contenuto di rs1 è maggiore o uguale a quello contenuto in rs2. Salta se minore di senza segno (`bltu`, *branch if less than unsigned*) prende il salto quando il valore contenuto in rs1 è inferiore di quello in rs2, trattando il contenuto dei due registri come numeri senza segno. Infine, l'istruzione salta se maggiore o uguale senza segno (`bgeu`, *branch if greater than or equal unsigned*) prende il segno nel caso opposto.

Un'alternativa a queste altre istruzioni di salto condizionato è impostare un registro in funzione del risultato del confronto, per poi saltare o meno a seconda del contenuto di questo registro con le istruzioni di `beq` o `bne`. Questo approccio, utilizzato dall'insieme delle istruzioni dei MIPS, può rendere l'unità di elaborazione di una CPU leggermente più semplice, ma richiede più istruzioni per scrivere un programma.

Un'ulteriore alternativa, utilizzata nell'insieme delle istruzioni ARM, è prevedere dei bit aggiuntivi per registrare quello che succede durante l'esecuzione di un'istruzione. Questi bit aggiuntivi, chiamati *codici di condizione* o *flag* (letteralmente bandiera), indicano, per esempio, se il risultato di un'operazione aritmetica è negativo, o è uguale a zero, oppure se si è verificato un overflow.

Le istruzioni di salto condizionato utilizzano quindi una combinazione di questi codici di condizione per eseguire il confronto desiderato.

Un inconveniente dei codici di condizione è che se le istruzioni che li impostano sono molte, si creano delle dipendenze che rendono difficile l'esecuzione in pipeline del codice (Capitolo 4).

Scorciatoie per il controllo dei confini di vettori e matrici

Considerare un numero come non dotato di segno fornisce un modo semplice per controllare se $0 \leq x < y$, dove 0 e y sono i limiti di un dato vettore. La chiave è che i numeri negativi in complemento a 2 equivalgono a numeri molto grandi quando sono considerati nella notazione senza segno. In particolare, il bit più significativo è il bit di segno nel caso in cui il numero sia dotato di segno, mentre costituisce una parte importante del numero nel caso in cui il numero sia considerato senza segno. Perciò, il confronto $x < y$ tra due numeri senza segno valuta non solo se x sia minore di y, ma anche se x sia negativo.

Scorciatoia per il controllo dello sforamento dei limiti

ESEMPIO

Utilizzare la scorciatoia mostrata in precedenza per ridurre il controllo di sforamento dei limiti di un vettore: si salti a `IndiceFuoriLimiti` se `x20` ≥ `x11` o se `x20` è negativo.

SOLUZIONE

Il codice utilizza la sola istruzione di salta se maggiore uguale senza segno, `bgeu` per effettuare entrambi i controlli:

```
bgeu x20, x11, IndiceFuoriLimiti // Se x20 >= x11 o x20 < 0
                                 // vai a IndiceFuoriLimiti
```

Costrutto *case/switch*

La maggior parte dei linguaggi di programmazione possiede un costrutto *case* o *switch* che consente al programmatore di scegliere fra più alternative in base al valore che assume un'unica variabile.

Il costrutto *switch* può essere realizzato mediante una sequenza di condizioni, trasformandolo in una catena di costrutti *if-then-else*.

Talvolta, però, le diverse alternative di un'istruzione *switch* possono essere specificate in modo più efficiente memorizzandole in una tabella contenente gli indirizzi dell'inizio dei frammenti di codice alternativi; questa tabella viene chiamata **tabella degli indirizzi di salto** (*branch address table*) o **tabella di salto**. Il programma deve solo indicizzare la tabella e saltare all'inizio della sequenza di istruzioni corretta. La tabella è quindi semplicemente un vettore di parole che contiene gli indirizzi corrispondenti alle etichette presenti nel codice. Il programma caricherà la parola corretta dalla tabella di salto in un registro e salterà all'indirizzo caricato nel registro.

Tabella degli indirizzi di salto: detta anche **tabella di salto**, contiene gli indirizzi dell'inizio dei frammenti di codice da eseguire in alternativa tra loro.

Per supportare quest'operazione, alcuni calcolatori come il RISC-V hanno inserito un'istruzione di *salto indiretto*, che esegue un salto incondizionato all'indirizzo specificato in un registro.

Nel RISC-V viene utilizzata l'istruzione di salta a registro (`jalr`, *jump and link register*). Vedremo un utilizzo molto più comune di questa versatile istruzione nel prossimo paragrafo.

Interfaccia hardware/software

Nonostante nei linguaggi ad alto livello, come C e Java, ci siano molti costrutti per le decisioni o per i cicli, a livello di linguaggio assembler il costrutto base che li implementa tutti è il salto condizionato.

Autovalutazione

I. Il linguaggio C ha molti costrutti per le decisioni e per i cicli, mentre il RISC-V ne ha pochi. Quali delle seguenti affermazioni sono una spiegazione di questo sbilanciamento e quali affermazioni non lo sono? Spiegarne il motivo.

1. Più costrutti decisionali rendono il codice facile da leggere e da capire.
2. Meno costrutti decisionali semplificano il compito del livello sottostante che è responsabile dell'esecuzione.
3. Più costrutti decisionali significano meno linee di codice, il che generalmente riduce il tempo di programmazione.
4. Più costrutti decisionali significano meno linee di codice, il che generalmente comporta l'esecuzione di meno operazioni.

II. Perché il linguaggio C fornisce due tipi di operazioni per l'AND (& e &&) e due tipi di operazioni per l'OR (| e ||), mentre il RISC-V no?

1. Le operazioni logiche AND e OR sono implementate con gli operatori & e | mentre le condizioni multiple verificate nei salti condizionati sono implementate con && e ||.
2. L'opposto della precedente affermazione: && e || corrispondono alle operazioni logiche mentre & e | a quelle utilizzate nei salti condizionati.
3. Sono ridondanti e significano la stessa cosa: && e || sono semplicemente state ereditate dal linguaggio di programmazione B, il predecessore del C.

2.8 Supporto hardware alle procedure

Procedura: sottoprogramma memorizzato che svolge un compito specifico basandosi sui parametri che gli vengono passati in ingresso.

ASTRAZIONE

Le **procedure**, o funzioni, costituiscono uno degli strumenti utilizzati dai programmatori per organizzare i programmi in modo da renderli più comprensibili e per permettere il riutilizzo del codice. Le procedure consentono ai programmatori di concentrarsi su una parte del problema alla volta; l'interfaccia fra la procedura e il resto del programma e dei dati è costituita dai parametri, i quali permettono di passare dei valori alla procedura e di restituire i risultati alla porzione di programma in cui essa è stata chiamata. L'equivalente Java di una procedura è descritto nel Paragrafo 2.15 🌐, ma per questo linguaggio l'analisi non è così diversa, poiché ha bisogno che il calcolatore faccia tutto ciò di cui ha bisogno il C. Le procedure sono un modo per implementare l'**astrazione** nel software.

Potete pensare alle procedure come a delle spie che lavorano in segreto, acquisiscono risorse, svolgono il loro compito, nascondono le tracce e tornano al punto di partenza con i risultati desiderati; nulla viene più modificato dopo che la missione è stata completata. Inoltre, le spie operano soltanto in base a ciò che "devono sapere" e non possono fare ipotesi su chi le abbia assoldate.

In modo simile, per l'esecuzione di una procedura, un programma deve eseguire questi sei passi:

1. Mettere i parametri in un luogo accessibile alla procedura;
2. Trasferire il controllo alla procedura;
3. Acquisire le risorse necessarie per l'esecuzione della procedura;
4. Eseguire il compito richiesto;
5. Mettere il risultato in un luogo accessibile al programma chiamante;
6. Restituire il controllo al punto di origine, dato che la stessa procedura può essere chiamata in diversi punti di un programma.

Come detto, i registri sono il luogo che permette l'accesso ai dati più rapido, quindi è opportuno utilizzarli il più possibile: il software RISC-V, per le chiamate a procedura, utilizza i suoi 32 registri secondo queste convenzioni:

- `x10-x17`: otto registri argomento per il passaggio dei parametri o la restituzione dei valori calcolati;
- `x1`: un registro contenente l'indirizzo di ritorno per tornare al punto di origine.

Oltre a utilizzare questi registri, il linguaggio assembler RISC-V comprende un'istruzione apposita per le procedure; questa istruzione salta all'indirizzo della procedura e contemporaneamente salva l'indirizzo dell'istruzione successiva a quella di salto nel registro rd. L'**istruzione jump and link** (`jal`, salta e crea un collegamento) si scrive come:

```
jal x1, IndirizzoProcedura // salta a IndirizzoProcedura e
                           // scrivi l'indirizzo di ritorno in x1
```

Istruzione jump and link: istruzione che esegue un salto a un dato indirizzo e che contemporaneamente salva in un registro (`x1` nel caso del RISC-V) l'indirizzo dell'istruzione successiva a quella di salto.

La parola *link* del nome si riferisce al fatto che viene creato un indirizzo, o collegamento (*link*), che punta alla posizione della chiamata, così da permettere alla procedura di tornare all'indirizzo corretto. Questo "collegamento", memorizzato nel registro `x1`, è detto **indirizzo di ritorno**. L'indirizzo di ritorno è necessario, perché la stessa procedura può essere chiamata da diversi punti del programma.

Indirizzo di ritorno: collegamento che consente a una procedura di restituire il controllo all'indirizzo opportuno del programma chiamante. Nel RISC-V è mantenuto nel registro `x1`.

Per supportare ciò, i calcolatori come i RISC-V utilizzano un'istruzione di salto indiretto, come l'istruzione *jump-and-link register* (`jalr`), introdotta in precedenza, per la gestione del costrutto case:

```
jalr x0, 0(x1)
```

L'istruzione `jalr` salta all'indirizzo memorizzato nel registro `x1`, che è esattamente quello che desideriamo. Perciò, il programma chiamante, detto anche semplicemente **chiamante**, mette i valori dei parametri da passare alla procedura nei registri da `x10-x17` e utilizza l'istruzione `jal x1, X` per saltare alla procedura `X` (detta anche **chiamata**). Questa esegue le operazioni richieste, memorizza i risultati negli stessi registri dei parametri, `x10-x17`, e restituisce il controllo al chiamante con l'istruzione `jalr x0, 0(x1)`.

(Programma) chiamante: programma che invoca una procedura e le fornisce i parametri necessari.

(Procedura) chiamata: procedura che esegue una sequenza di istruzioni sulla base dei parametri forniti dal chiamante e che al termine restituisce il controllo al chiamante.

Nell'idea di programma memorizzato è implicita la necessità di mantenere in un registro l'indirizzo dell'istruzione in esecuzione. Per ragioni storiche questo registro viene quasi sempre chiamato **program counter** (contatore di programma), abbreviato in *PC* nell'architettura RISC-V, anche se il nome più corretto sarebbe *instruction address register* (registro dell'indirizzo dell'istruzione). L'istruzione `jal` in realtà salva PC + 4 nel registro destinazione designato (di solito `x1`) per creare il collegamento all'indirizzo dell'istruzione successiva, predisponendo così il ritorno dalla procedura.

Program counter (PC): registro contenente l'indirizzo dell'istruzione in esecuzione.

Approfondimento. L'istruzione di *jump-and-link* può essere utilizzata anche per eseguire un salto incondizionato all'interno di una procedura utilizzando `x0` come registro destinazione. Dato che `x0` contiene sempre il valore zero, il risultato è quello di non valutare l'indirizzo di ritorno:

```
jal x0, Etichetta  // Salto incondizionato a Etichetta
```

Utilizzo di più registri

Supponiamo che un compilatore abbia bisogno, all'interno di una procedura, di un numero maggiore di registri rispetto agli otto previsti per il passaggio dei parametri. Dato che quando la missione è compiuta occorre cancellare le tracce, il contenuto dei registri utilizzati dal programma chiamante deve essere ripristinato con il valore *precedente* la chiamata. Questa è una tipica situazione nella quale è necessario copiare il contenuto dei registri in memoria, come descritto nella sezione *Interfaccia hardware/software* del Paragrafo 2.3.

La struttura dati ideale per copiare il contenuto dei registri è lo **stack** (pila), una coda di tipo *last-in-first-out* (ultimo-inserito-primo-estratto). Lo stack ha bisogno di un puntatore all'indirizzo dell'ultimo dato introdotto per indicare dove la procedura successiva possa salvare il contenuto dei registri e da dove possa poi recuperare il vecchio valore. Nel RISC-V il puntatore allo stack (**stack pointer**) è il registro `x2`, chiamato anche `sp`. Lo stack pointer viene incrementato o decrementato di una parola ogni volta che si toglie o si inserisce il contenuto di un registro. L'utilizzo dello stack è così comune che sono stati coniati dei termini onomatopeici per indicare il trasferimento dati da e verso lo stack: per **push** si intende l'inserimento di un dato nello stack e per **pop** la sua estrazione.

Stack: struttura dati contente una coda di tipo *last-in-first-out* (ultimo-inserito-primo-estratto), utilizzata per salvare il contenuto dei registri.

Stack pointer (puntatore a stack): indirizzo dell'elemento dello stack allocato più di recente; indica il punto in cui i registri potrebbero essere salvati o dove si può trovqare il vecchio valore dei registri da ripristinare. Nel RISC-V, lo stack pointer è contenuto nel registro `sp` o `x2`.

Push: aggiunta di un elemento nello stack.

Pop: estrazione di un elemento dallo stack.

Per ragioni storiche lo stack "cresce" a partire da indirizzi di memoria alti verso indirizzi di memoria bassi. Questa convenzione implica che quando vengono inseriti dei dati nello stack, il valore dello stack pointer diminuisce; al contrario, quando i dati sono estratti dallo stack, aumenta il valore dello stack pointer e si riduce la dimensione dello stack.

ESEMPIO

Compilazione di una procedura C che non chiama altre procedure

Si trasformi il secondo esempio del Paragrafo 2.2 in una procedura C:

```
int esempio_foglia(int g, int h, int i, int j)
{
     int f;
```

(continua)

(continua)

```
        f = (g + h) - (i + j);
        return f;
}
```

Qual è il codice assembler RISC-V che si otterrebbe dalla compilazione?

SOLUZIONE

I parametri `g`, `h`, `i` e `j` corrispondono ai registri argomento `x10`, `x11`, `x12` e `x13` rispettivamente, mentre `f` corrisponde a `x20`. Il programma assembler inizia con l'etichetta della procedura:

```
esempio_foglia:
```

Il passo successivo consiste nel salvare tutti i registri utilizzati dalla procedura. L'assegnazione C nel corpo della procedura è identica a quella del primo esempio del Paragrafo 2.3 e utilizza due registri temporanei (`x5` e `x6`); è quindi necessario salvare il contenuto dei tre registri `x5`, `x6` e `x20`. Memorizziamo il contenuto di questi tre registri nello stack (operazione di push), dopo aver allocato lo spazio per 3 parole (12 byte):

```
addi sp, sp, -12  // aggiornamento dello stack pointer
                  // per fare posto a 3 elementi,
sw   x5, 8(sp)    // salvataggio del registro x5 per poter
                  // essere utilizzato successivamente
sw   x6, 4(sp)    // salvataggio del registro x6, per poter
                  // essere utilizzato successivamente
sw   x20, 0(sp)   // salvataggio del registro x20,per poter
                  // essere utilizzato successivamente
```

La **Figura 2.10** mostra lo stack prima, durante e dopo la chiamata alla procedura. Le prossime tre istruzioni corrispondono al corpo della procedura e sono analoghe all'esempio del Paragrafo 2.3:

```
add x5, x10, x11 // il registro x5 contiene g + h
add x6, x12, x13 // il registro x6 contiene i + j
sub x20, x5, x6  // f = x5 - x6, cioè (g + h) - (i + j)
```

Per restituire il valore di `f`, occorre copiarlo in un registro dei parametri:

```
addi x10, x20, 0 // restituzione di f (x10 = x20 + 0)
```

Prima del ritorno al programma chiamante, il vecchio valore dei registri deve essere ripristinato, copiandolo (operazione di pop) dallo stack:

```
lw x20, 0(sp)     // ripristino del registro x20 per il chiamante
lw x6, 4(sp)      // ripristino del registro x6 per il chiamante
lw x5, 8(sp)      // ripristino del registro x5 per il chiamante
addi sp, sp, 12   // aggiornamento dello stack pointer con
                  // l'eliminazione di 3 elementi
```

La procedura termina con un'istruzione di salto a registro, utilizzando l'indirizzo di ritorno:

```
jalr x0, 0(x1)    // ritorno al programma chiamante
```

Nell'esempio qui riportato sono stati utilizzati dei registri temporanei ed è stata fatta l'ipotesi che il loro valore originale dovesse essere salvato e poi ripristinato. Per evitare di salvare e ripristinare registri il cui valore non verrà mai

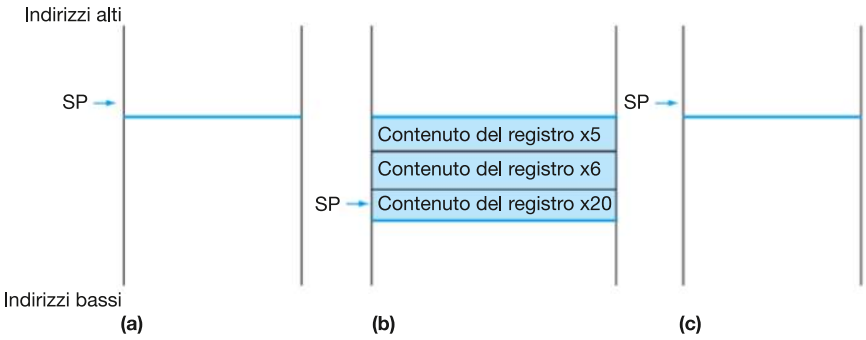


Figura 2.10 Contenuto dello stack pointer e situazione dello stack prima (a), durante (b) e dopo (c) la chiamata a procedura. Lo stack pointer punta sempre alla cima (top) dello stack, cioè all'ultima parola presente nello stack.

utilizzato, cosa che può accadere con i registri temporanei, il software RISC-V suddivide 19 dei registri in due gruppi:

- `x5-x7 e x28-x31`: registri temporanei, che non sono salvati in caso di chiamata (dalla procedura chiamata) di una procedura;
- `x8-x9 e x18-x27`: registri da salvare il cui contenuto deve essere preservato in caso di chiamata a procedura (se vengono utilizzati dalla procedura chiamata, il loro contenuto deve essere prima salvato).

Questa semplice convenzione riduce il numero di registri da salvare in memoria. Nell'esempio precedente, dato che il programma chiamante non si aspetta che i registri `x5` e `x6` siano preservati dalla procedura, possiamo eliminare dal codice della stessa due istruzioni di memorizzazione e due di lettura. Dobbiamo invece salvare e ripristinare il registro `x20`, dal momento che la procedura è costretta a supporre che il chiamante abbia bisogno del suo valore originale.

Procedure annidate

Le procedure che non chiamano altre procedure sono chiamate procedure *foglia*; la vita del programmatore sarebbe più semplice se tutte le procedure fossero di questo tipo, ma non è così. Proprio come una spia può arruolare altre spie per compiere parte di una missione, e queste a loro volta possono arruolarne altre ancora, le procedure possono chiamare altre procedure. Inoltre, le procedure ricorsive invocano perfino "cloni" di se stesse. Se occorre fare attenzione all'utilizzo dei registri nelle procedure, occorre stare ancora più attenti se una procedura non è di tipo foglia.

Si supponga, per esempio, che il programma principale chiami la procedura A con un parametro di valore 3, mettendo quindi il valore 3 nel registro `x10` e utilizzando subito dopo l'istruzione `jal x1, A`. Si supponga poi che la procedura A chiami a sua volta la procedura B con una `jal x1, B`, passandole il valore 7, anch'esso posto in `x10`. Dato che A non ha ancora terminato il proprio compito, si verifica un conflitto nell'utilizzo del registro `x10`; in modo analogo si verifica un conflitto sull'indirizzo di ritorno di B. Se non si adottassero delle contromisure, questo conflitto farebbe sì che la procedura A non sia più in grado di restituire il controllo al suo chiamante.

Una soluzione consiste nel salvare nello stack tutti gli altri registri, il cui contenuto deve essere conservato, esattamente come abbiamo fatto per i registri da salvare. Il programma chiamante, quindi, salverà nello stack qualunque registro argomento (`x10-x17`) o registro temporaneo (`x5-x7 e x28-x31`)

 © 978-88-08-19966-9

quando gli servirà il contenuto di questi registri dopo la chiamata; il chiamato invece salverà nello stack il registro di ritorno, x1, e quei registri da preservare (x8-x9 e x18-x27) di cui ha bisogno. Lo stack pointer, sp, sarà aggiornato per tener conto del numero di registri da memorizzare nello stack; al termine della procedura, il contenuto dei registri verrà ripristinato con il contenuto dello stack e lo stack pointer sarà riaggiornato.

Compilazione di una procedura ricorsiva C come esempio dei collegamenti tra procedure annidate

ESEMPIO

Analizziamo una procedura ricorsiva che calcola i numeri fattoriali:

```
int fatt (int n)
{
   if (n < 1) return (1);
       else return (n * fatt(n-1));
}
```

Quale sarà il codice assembler RISC-V?

SOLUZIONE

Il parametro n corrisponde al registro argomento x10. Il programma compilato inizierà con l'etichetta della procedura seguita dal salvataggio sullo stack di due registri: l'indirizzo di ritorno e x10:

```
fatt:
    addi sp, sp, -8  // aggiorna lo stack per fare
                     // posto a 2 elementi
      sw x1, 4(sp)   // salvataggio dell'indirizzo
                     // di ritorno
      sw x10, 0(sp)  // salvataggio del parametro n
```

La prima volta che la procedura fattoriale viene chiamata, l'istruzione sw salva in stack un indirizzo del programma chiamante. Le due istruzioni successive verificano se n è < 1; se n ≥ 1, la procedura salta a L1.

```
addi x5, x10, 1 // x5 = n - 1
bge  x5, x0, L1 // se (n - 1) >= 0, salta a L1
```

Se n è minore di 1, fattoriale restituisce il valore 1 mettendolo in un registro valore: somma 1 a 0 e memorizza il risultato in x10. Elimina quindi dallo stack i due registri e salta all'indirizzo di ritorno:

```
addi x10, x0,1  // restituisci 1
addi sp, sp, 8  // elimina 2 elementi dallo stack,
jalr x0, 0(x1)  // ritorna al programma chiamante
```

Prima di eliminare i due elementi dallo stack, avremmo potuto salvare x1 e x10; tuttavia, dato che x1 e x10 non cambiano quando n è minore di 1, possiamo evitare queste due istruzioni.

Se n non è minore di 1, il parametro n viene decrementato e viene nuovamente chiamata la procedura fattoriale passandole il nuovo valore:

```
L1: addi x10, x10, -1  // n >= 1: l'argomento diventa (n - 1)
    jal  x1, fatt      // chiamata a fatt con (n - 1)
```

L'istruzione successiva è quella a cui la procedura fattoriale ritornerà al termine della sua esecuzione; il risultato prodotto si trova in x10.

(continua)

(continua)

Occorre ora ripristinare il vecchio indirizzo di ritorno, il vecchio valore del parametro e lo stack pointer:

```
addi x6, x10, 0 // ritorno da jal; sposta il risultato
                // di fatt(n - 1) in x6:
lw x10, 0(sp)   // ripristino dell'argomento, n
lw x1, 4(sp)    // ripristino del registro di ritorno
addi sp, sp, 8  // aggiorna lo stack pointer per
                // eliminare 2 elementi
```

Successivamente, nel registro argomento `x10` viene memorizzato il prodotto del vecchio argomento con il risultato di `fatt(n - 1)` contenuto in `x6`. Supponiamo di avere a disposizione un'istruzione per la moltiplicazione (anche se questa verrà descritta nel Capitolo 3):

```
mul x10, x10, x6 // restituisci n * fatt(n - 1)
```

Infine, la procedura `fatt` salta nuovamente all'indirizzo di ritorno:

```
jalr x0, 0(x1)   // ritorna al chiamante
```

Interfaccia hardware/software

Una variabile di un programma in C corrisponde in genere a una locazione di memoria e la sua interpretazione dipende sia dal *tipo* sia dalla *modalità di allocazione*. Un tipico esempio sono gli interi e i caratteri (Paragrafo 2.9). Il C ha due modalità di memorizzazione: *automatica* e *statica*. Le variabili automatiche sono locali a una procedura e vengono eliminate all'uscita da quest'ultima; le variabili statiche, invece, continuano a esistere anche quando viene chiamata una procedura o la procedura ritorna al chiamante. Le variabili C dichiarate al di fuori di tutte le procedure sono considerate statiche, così come tutte le variabili precedute dalla parola chiave *static*; tutte le altre variabili sono allocate dinamicamente. Per semplificare l'accesso ai dati statici, alcuni compilatori RISC-V utilizzano un altro registro, chiamato **global pointer**, registro `x3`, che viene utilizzato come puntatore globale.

Global pointer: registro riservato all'indirizzamento dei dati statici.

La **Figura 2.11** riassume ciò che viene preservato nella chiamata a una procedura. Si noti che si utilizzano diversi accorgimenti per salvare il contenuto dello stack, in modo da garantire che il chiamante ottenga dallo stack gli stessi valori che vi aveva memorizzato. La parte di stack che sta sopra lo `sp` viene preservata semplicemente assicurandosi che il chiamato non scriva sopra lo stack pointer; `sp` viene a sua volta preservato dal chiamato, che sommerà sempre al valore corrente di `sp` esattamente lo stesso valore che gli aveva sottratto; gli altri registri vengono preservati salvandoli in stack (nel caso in cui vengano utilizzati) per essere ripristinati in un secondo tempo.

Conservato	Non conservato
Registri da salvare: `x8-x9, x18-x27`	Registri temporanei: `x5-x7, x28-x31`
Registro stack pointer: `x2(sp)`	Registri argomento/risultato: `x10-x17`
Registro frame pointer: `x8(fp)`	
Indirizzo di ritorno: `x1(ra)`	
Stack al di sopra dello stack pointer	Stack al di sotto dello stack pointer

Figura 2.11 Che cosa viene e che cosa non viene conservato in una chiamata a procedura. Se il software utilizza il registro frame pointer o il registro global pointer, che esamineremo in seguito, anche il contenuto di questi registri deve essere preservato.

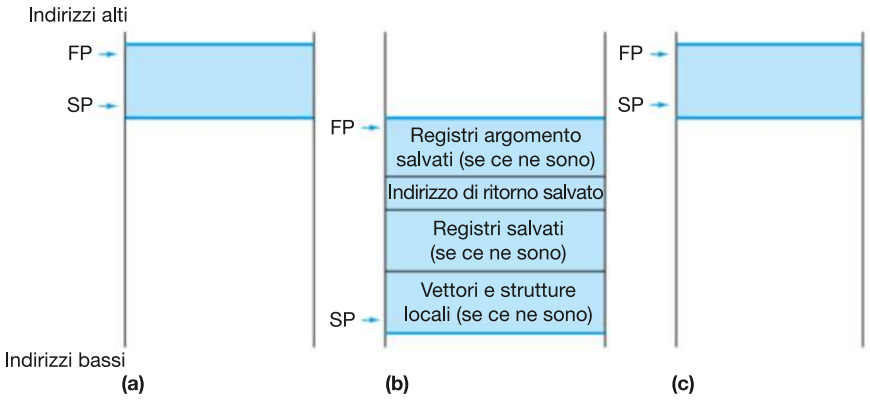


Figura 2.12 Allocazione dello stack prima (a), durante (b) e dopo (c) la chiamata a procedura. Il frame pointer (`fp` o `x8`) punta alla prima parola del frame, spesso contenente un registro da preservare, e lo stack pointer (`sp`) punta alla cima dello stack. Lo stack viene aggiornato per fare spazio a tutti i registri da preservare e alle variabili locali che devono essere salvate in memoria. Dal momento che lo stack pointer può cambiare durante l'esecuzione del programma, è più facile per i programmatori indirizzare le variabili in stack tramite il frame pointer, che invece rimane stabile (anche se ciò potrebbe essere fatto con il solo stack pointer e un po' di aritmetica sugli indirizzi). Se lo stack non contiene variabili locali alla procedura, il compilatore risparmia tempo di esecuzione evitando di impostare e ripristinare il frame. Quando viene utilizzato, il frame pointer viene inizializzato con l'indirizzo che ha `sp` all'atto della chiamata della procedura e `sp` viene ripristinato al termine della procedura utilizzando il valore di `fp`. Si possono trovare queste informazioni anche nella quarta colonna della scheda tecnica riassuntiva del RISC-V in fondo al libro.

Allocazione dello spazio nello stack per nuovi dati

Record di attivazione: detto anche frame della procedura, è il segmento dello stack che contiene i registri salvati da una procedura e le variabili locali.

Un'ultima complicazione consiste nel fatto che lo stack può essere utilizzato anche per memorizzare le variabili locali della procedura che non trovano spazio nei registri, per esempio i vettori o le strutture locali. Il segmento dello stack che contiene i registri salvati da una procedura e le variabili locali prende il nome di **record di attivazione** (blocco di attivazione) o **frame della procedura** (cornice della procedura). La Figura 2.12 mostra lo stato dello stack prima, durante e dopo la chiamata di una procedura.

Frame pointer: valore che individua la posizione dei registri salvati e delle variabili locali di una data procedura.

Alcuni programmi RISC-V utilizzano un **frame pointer** (`fp`, puntatore a frame, registro `x8`) per puntare alla prima parola del frame di una procedura. Lo stack pointer potrebbe cambiare durante l'esecuzione di una procedura; in questo caso il riferimento alle variabili locali in memoria potrebbe assumere offset diversi a seconda della loro posizione nella procedura, rendendo così le procedure più difficili da leggere. In questo caso il frame pointer può essere vantaggiosamente utilizzato come registro di base stabile all'interno della procedura, per l'accesso in memoria alle variabili locali. Si noti che il record di attivazione è presente nello stack indipendentemente dal fatto che il frame pointer sia usato esplicitamente o meno. Nell'esempio precedente non abbiamo utilizzato `fp`, evitando di modificare `sp` all'interno della procedura; la dimensione dello stack veniva modificata solo all'entrata e all'uscita da essa.

Allocazione dello spazio nello heap per nuovi dati

In aggiunta alle variabili ad allocazione dinamica, locali alle procedure, i programmatori C hanno bisogno di spazio in memoria per le variabili di tipo statico e per le strutture dati dinamiche esterne alle procedure. La Figura 2.13 mostra la convenzione utilizzata dal RISC-V per l'allocazione della memoria quando viene utilizzato il sistema operativo Linux. Lo stack inizia nella parte alta della memoria dello spazio di indirizzamento utente (Capitolo 5) e cresce verso il basso. Dal lato opposto, la prima parte di memoria che si incontra

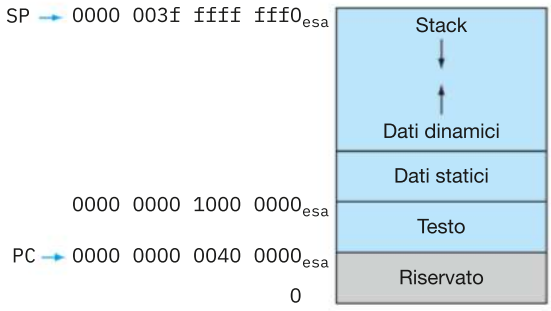


Figura 2.13 Allocazione della memoria per programmi e dati nel RISC-V. Gli indirizzi indicati sono frutto solo di convenzioni software e non costituiscono parte integrante dell'architettura del RISC-V. Lo spazio indirizzamento utente è impostato a 2^{38} rispetto ai teorici 2^{64} valori che si possono raggiungere con un'architettura a 64 bit (Capitolo 5). Lo stack pointer viene inizializzato a `0000 003f ffff fff0`$_{esa}$ e decresce verso il basso verso il segmento dati. Dall'altra parte, il segmento di testo, contenente il codice del programma, parte dall'indirizzo `0000 0000 0040 0000`$_{esa}$. I dati statici nell'esempio partono immediatamente dopo il segmento testo, assumiamo che questo indirizzo sia `0000 0000 1000 0000`$_{esa}$. I dati dinamici, che vengono allocati con una `malloc` in C o con una `new` in Java, sono memorizzati subito dopo i dati statici: questa area di memoria cresce verso l'alto (verso lo stack), ed è denominata *heap*. Si possono trovare queste informazioni anche nella scheda tecnica riassuntiva del RISC-V in fondo al libro.

è riservata, seguita dalla zona in cui risiede il codice macchina RISC-V, tradizionalmente chiamata **segmento di testo**. Sopra il codice si trova il *segmento dei dati statici*, che è il luogo in cui risiedono le costanti e le altre variabili statiche. Nonostante i vettori siano di lunghezza fissa e quindi siano adatti a essere allocati nella zona dei dati statici, strutture come le liste concatenate hanno la caratteristica di crescere e di ridursi durante la loro vita. Il segmento di memoria dati utilizzato per queste strutture è chiamato tradizionalmente *heap* ("cumulo") ed è posizionato sopra la porzione di memoria statica. Si noti che con questa modalità di allocazione lo stack e lo heap crescono uno verso l'altro, permettendo un uso efficiente della memoria nonostante la dimensione dei due segmenti sia altalenante.

Segmento di testo: segmento di memoria che contiene il codice in linguaggio macchina di tutte le procedure di un'applicazione, contenute in un file oggetto UNIX.

Il linguaggio C alloca e libera spazio nello heap mediante funzioni esplicite: `malloc()` alloca spazio nello heap e ritorna il puntatore al primo indirizzo dello spazio allocato, e `free()` libera lo spazio nello heap puntato dal puntatore. L'allocazione della memoria in C viene perciò controllata dal programma ed è fonte di molti errori comuni e difficili da localizzare. Dimenticare di liberare lo spazio allocato provoca una "perdita" di memoria che potrebbe diventare tanto rilevante da portare al *crash* (blocco) del sistema operativo; liberare la memoria troppo presto, invece, rende i puntatori "volanti" (*dangling pointers*), ossia questi potrebbero puntare a cose molto diverse da quelle che interessavano al programma. Java utilizza l'allocazione e la liberazione automatica della memoria principalmente per evitare questi errori.

La **Figura 2.14** riassume la convenzione di utilizzo dei registri nel linguaggio assembler RISC-V. Questa convenzione è un altro esempio di come si possano **rendere veloci le situazioni più comuni**: la maggiore parte delle procedure richiede non più di otto registri per gli argomenti, dodici registri per variabili permanenti e sette registri temporanei, senza dover ricorrere alla memoria.

SITUAZIONI COMUNI

Approfondimento. Che cosa succederebbe se si volessero passare più di otto parametri a una procedura? Per convenzione i parametri aggiuntivi vengono messi nello stack al di sopra dell'indirizzo puntato dal frame pointer: la procedura si aspetterà quindi i primi otto parametri nei registri `x10-x17` e i restanti nell'area di stack, indirizzabili attraverso il frame pointer.

Come citato nella didascalia della Figura 2.12, il frame pointer è comodo perché fa sì che i riferimenti alle variabili in stack contenuti all'interno di una procedura

Nome	Numero del registro	Utilizzo	Da conservare nella chiamata?
x0	0	Costante 0	n.a.
x1 (ra)	1	Registro di ritorno (registro di collegamento)	sì
x2 (sp)	2	Stack pointer (Puntatore a stack)	sì
x3 (gp)	3	Global pointer (Puntatore globale)	sì
x4 (tp)	4	Thread pointer (Puntatore a thread)	sì
x5-x7	5–7	Variabili temporanee	no
x8-x9	8–9	Variabili da preservare	sì
x10-x17	10–17	Argomenti/risultati	no
x18-x27	18–27	Variabili da preservare	sì
x28-x31	28–31	Variabili temporanee	no

Figura 2.14 Convenzioni di utilizzo dei registri RISC-V. Si possono trovare queste informazioni anche nella seconda colonna della scheda tecnica riassuntiva del RISC-V in fondo al libro.

mantengano lo stesso offset. Il frame pointer però non è strettamente necessario: il compilatore per il RISC-V utilizza solo un frame pointer nelle procedure che modificano lo stack pointer all'interno del corpo della procedura.

Approfondimento. Alcune procedure ricorsive possono essere implementate attraverso iterazioni che non implicano la ricorsione. L'utilizzo delle iterazioni può migliorare sensibilmente le prestazioni eliminando il sovraccarico di lavoro associato alle chiamate di procedura. Per esempio, si consideri una procedura che effettua delle somme ricorsive:

```
int somma (int n, int acc) {
     if (n > 0)
          return somma(n - 1, acc + n);
     else
           return acc;
}
```

Esaminiamo la procedura `somma(3,0)`. Si può osservare che essa chiamerà ricorsivamente le procedure `somma(2,3)`, `somma(1,5)` e `somma(0,6)`; il risultato, 6, verrà quindi restituito alla procedura chiamante quattro volte. Questo tipo di chiamata ricorsiva viene detto anche *tail call* (chiamata ad anello) e in questo esempio può essere implementato in modo molto efficiente come segue (si assuma `x10 = n` e `x11 = acc` e il risultato in `x12`):

```
somma: ble x10, x0, somma_esci // vai a somma_esci se n <= 0
       add x11, x11, x10       // aggiungi n ad acc
       addi x10, x10, -1       // sottrai 1 a n
       jal x0, somma           // vai a somma
somma_esci:
       addi x12, x11, 0        // restituisci il valore acc
       jalr x0, 0(x1)          // ritorna al chiamante
```

Autovalutazione

Quali delle seguenti affermazioni su C o su Java sono generalmente vere?

1. I programmatori C gestiscono i dati esplicitamente, mentre la gestione dei dati in Java è automatica.
2. Il linguaggio C induce più errori sui puntatori e sull'allocazione dinamica della memoria di quanto faccia Java.

2.9 Comunicare con le persone

I calcolatori furono inventati per effettuare calcoli matematici complessi, ma non appena diventarono commercialmente disponibili iniziarono a essere utilizzati anche per l'elaborazione di testi. Oggi la maggior parte dei calcolatori utilizza 8 bit (1 byte) per rappresentare i caratteri e la codifica ASCII (*American Standard Code for Information Interchange*) è di gran lunga la più utilizzata. La Figura 2.15 riassume la codifica ASCII.

!(@|=> (wow open tab at bar is great - oh! il conto aperto al bar è fantastico)

La quarta riga del poema della tastiera "Hatless Atlas" 1991 (alcuni associano a ogni carattere ASCII una parola: "!" è "wow", "(" è "open", "@" è "at", "|" è "bar" ecc.).

Confronto tra codifica ASCII e codifica binaria

ESEMPIO

Potremmo rappresentare i numeri come stringhe di caratteri ASCII invece che come numeri interi. Di quanto aumenterebbe la richiesta di memoria se il numero 1 miliardo venisse rappresentato sotto forma di stringa di caratteri invece che come intero su 32 bit?

SOLUZIONE

Il numero 1 miliardo è rappresentato dalla stringa 1 000 000 000, che quindi richiede 10 caratteri ASCII, ciascuno di 8 bit. La richiesta di memoria sarebbe pari a (10 × 8)/32, ovvero 2,5 parole. Oltre a richiedere più memoria, l'hardware diventerebbe molto più complesso e assorbirebbe una maggiore quantità di energia per compiere le operazioni di somma, sottrazione, moltiplicazione e divisione in base 10. Questi problemi spiegano come mai per un calcolatore la codifica binaria è naturale, mentre quella in base 10 risulta non funzionale.

Codice ASCII	Carattere	Codice ASCII	Carattere	Codice ASCII	Carattere	Codice ASCII	Carattere	Codice ASCII	Carattere	Codice ASCII	Carattere
32	Spazio	48	0	64	@	80	P	096	`	112	p
33	!	49	1	65	A	81	Q	097	a	113	q
34	"	50	2	66	B	82	R	098	b	114	r
35	#	51	3	67	C	83	S	099	c	115	s
36	$	52	4	68	D	84	T	100	d	116	t
37	%	53	5	69	E	85	U	101	e	117	u
38	&	54	6	70	F	86	V	102	f	118	v
39	'	55	7	71	G	87	W	103	g	119	w
40	(	56	8	72	H	88	X	104	h	120	x
41	)	57	9	73	I	89	Y	105	i	121	y
42	*	58	:	74	J	90	Z	106	j	122	z
43	+	59	;	75	K	91	[	107	k	123	{
44	,	60	<	76	L	92	\	108	l	124	|
45	-	61	=	77	M	93	]	109	m	125	}
46	.	62	>	78	N	94	^	110	n	126	~
47	/	63	?	79	O	95	_	111	o	127	DEL

Figura 2.15 Codifica ASCII dei caratteri. Si noti che le lettere maiuscole e minuscole differiscono esattamente di 32; ciò consente di utilizzare una scorciatoia per controllare e convertire le minuscole in maiuscole e viceversa. I valori non mostrati sono associati ai caratteri di formattazione: per esempio, il carattere 8 rappresenta lo spostamento di uno spazio a sinistra, il 9 la tabulazione, mentre il 13 rappresenta il ritorno a capo. Un altro carattere particolarmente utile è lo 0, che rappresenta il carattere "null", che viene utilizzato nel linguaggio di programmazione C per segnalare la fine di una stringa.

 © 978-88-08-19966-9

Una serie di istruzioni consente di estrarre un byte da una parola, quindi le istruzioni di trasferimento tra memoria e registri (caricamento di un registro e salvataggio in memoria) sono sufficienti anche per trasferire byte. L'elaborazione di testi è però talmente diffusa che il RISC-V offre istruzioni apposite per trasferire i singoli byte: *load byte unsigned* (`lbu`) prende un byte dalla memoria mettendolo negli 8 bit di un registro, collocati più a destra, mentre *store byte* (`sb`) prende il byte corrispondente agli 8 bit di un registro, collocati più a destra, e lo salva in memoria. Perciò, possiamo copiare un byte con la seguente coppia di istruzioni:

```
lbu x12, 0(x10) // Leggi un byte dall'indirizzo sorgente
sb  x12, 0(x11) // Scrivi il byte all'indirizzo di
                // destinazione
```

Normalmente i caratteri vengono raggruppati in stringhe, composte da un numero variabile di caratteri. Ci sono tre modi per rappresentare una stringa: (1) il primo elemento della stringa viene utilizzato per definirne la lunghezza, (2) una variabile di appoggio contiene la lunghezza della stringa (come in una struttura) oppure (3) l'ultimo elemento della stringa è individuato da un carattere particolare di fine stringa. Il linguaggio C utilizza questo terzo modo e termina le stringhe con un byte che contiene il valore 0 (carattere "null" in ASCII). In C, quindi, la stringa "Cal" viene rappresentata da 4 byte, i cui valori decimali sono rispettivamente 67, 97, 108 e 0 (come vedremo in seguito, Java invece utilizza il primo modo).

Compilazione di una procedura per copiare una stringa come esempio di utilizzo delle stringhe nel C

ESEMPIO

La procedura `copiaStringa` copia la stringa `y` nella stringa `x`, utilizzando il byte null come carattere di fine stringa come previsto nel linguaggio C:

```
void copiaStringa(char x[], char y[])
{
   size_t i;
   i = 0;
   while ((x[i] = y[i])!= '\0') /* copia & controllo del byte */
      i += 1;
}
```

Qual è il codice assembler RISC-V corrispondente?

SOLUZIONE

Il frammento di codice assembler RISC-V viene mostrato di seguito. Si assuma che gli indirizzi di base dei vettori `x` e `y` si trovino rispettivamente in `x10` e `x11`, mentre `i` sia contenuto in `x19`. `copiaStringa` aggiorna lo stack pointer e poi salva il registro da preservare `x19` nello stack:

```
copiaStringa:
     addi sp, sp, -4 // aggiorna lo stack
                     // per inserire un elemento
     sw x19, 0(sp)   // salva x19
```

Per inizializzare `i` a 0, l'istruzione successiva imposta `x19` a 0 sommando 0 con 0 e mettendo il risultato in `x19`:

```
add x19, x0, x0   // i = 0 + 0
```

(continua)

(continua)

Questo è l'inizio del ciclo. Come prima cosa, l'indirizzo di `y[i]` viene creato sommando `i` a `y[]` e ponendo il risultato in `x5`:

```
L1: add x5, x19, x11   // indirizzo di y[i] in x5
```

Si noti che non è necessario moltiplicare `i` per 4, dato che `y` è un vettore di byte e non di parole come nei casi precedenti.

Load byte (`lb`) estende il segno del byte mentre load byte unsigned (`lbu`) riempie di zeri le posizioni vuote.

Per caricare il carattere presente in `y[i]` utilizziamo l'istruzione *load byte unsigned* e mettiamo il carattere letto in `x6`:

```
lbu x6, 0(x5)   // x6 = y[i]
```

Analogamente calcoliamo l'indirizzo di `x[i]` e lo scriviamo in `x7`; poniamo quindi il carattere che si trova in `x6` nella locazione di memoria a quell'indirizzo:

```
add x7, x19, x10  // indirizzo di x[i] in x7
sb  x6, 0(x7)     // x[i] = y[i]
```

Successivamente possiamo uscire dal ciclo se il carattere letto è 0, cioè se si tratta del carattere di fine stringa:

```
beq x6, x10, L2
```

Altrimenti si incrementa `i` di 1 e si torna all'inizio del ciclo:

```
addi x19, x19, 1 // i = i + 1
jal  x0, L1      // salta a L1
```

Se non si ritorna all'inizio del ciclo vuol dire che la stringa è terminata; in questo caso viene ripristinato il valore di `x19` e dello stack pointer e si esce dalla procedura:

```
L2: lw   x19, 0(sp) // ripristina il vecchio x19
    addi sp, sp, 4  // aggiorna lo stack eliminando
                    // una parola
    jral x0, 0(x1)  // ritorna al chiamante
```

In C la copia di stringhe di solito avviene tramite puntatori e non tramite vettori per evitare di effettuare operazioni sull'indice `i`. *Vedi* il Paragrafo 2.14 per un confronto tra l'utilizzo di vettori e puntatori.

Dal momento che la procedura `copiaStringa` appena descritta è una procedura di tipo foglia, il compilatore può allocare `i` in un registro temporaneo evitando di salvare e ripristinare `x19`. Pertanto, invece di considerare questi registri soltanto come registri temporanei, è più corretto considerarli come registri che le procedure possono utilizzare a loro piacimento. Quando il compilatore incontra una procedura foglia esaurisce tutti i registri temporanei prima di utilizzare quelli di cui deve salvare il contenuto.

Caratteri e stringhe in Java

L'*Unicode* è una codifica universale degli alfabeti di gran parte delle lingue utilizzate sul nostro pianeta. La **Figura 2.16** riporta un elenco di alfabeti inclusi nell'Unicode. Come si può notare, il numero di alfabeti è quasi uguale a quello dei *simboli* nel codice ASCII. Java utilizza Unicode per rappresentare i caratteri e quindi normalmente utilizza 16 bit per rappresentare un singolo carattere.

Latino	Malayalam	Tagbanwa	Punteggiatura generale
Greco	Sinhala	Khmer	Lettere che modificano la spaziatura
Cirillico	Thailandese	Mongolo	Simboli delle monete
Armeno	Laotiano	Limbu	Combinazione di segni diacritici
Ebraico	Tibetano	Tai Le	Combinazione di segni per ottenere simboli
Arabo	Myanmar	Kangxi Radicals	Apici e pedici
Siriaco	Georgiano	Hiragana	Forme numeriche
Thaana	Hangul Jamo	Katakana	Operatori matematici
Devanagari	Etiope	Bopomofo	Simboli matematici alfanumerici
Bengalese	Cherokee	Kanbun	Sequenze Braille
Gurmukhi	Sillabico Unificato degli Aborigeni Canadesi	Shavian	Caratteri per il riconoscimento ottico
Gujarati	Ogham	Osmanya	Simboli musicali bizantini
Oriya	Runico	Sillabico Cipriota	Simboli musicali
Tamil	Tagalog	Thailandese Simbolico Xuan Jing	Frecce
Telugu	Hanunoo	Simboli Yijing Hexagram	Disegno di riquadri
Kannada	Buhid	Numeri Aegean	Forme geometriche

Figura 2.16 Esempi di alfabeti rappresentati con l'Unicode. La versione 4.0 di Unicode contiene più di 160 "blocchi", che è il nome dato a un insieme di simboli. Ogni blocco è un multiplo di 16. Per esempio, l'alfabeto greco parte dalla posizione 0370_{esa}, il cirillico da 0400_{esa}. Le prime tre colonne mostrano il contenuto di 48 blocchi che corrispondono alle lingue rappresentate dall'Unicode, seguendo approssimativamente l'ordine numerico dell'Unicode. L'ultima colonna contiene 16 blocchi, considerati multilingua e che non sono in ordine. La codifica a 16 bit, chiamata UTF-16, è quella di default, mentre una codifica a lunghezza variabile, denominata UTF-8, contiene come sottoinsieme la codifica ASCII su 8 bit e utilizza 16–32 bit per gli altri caratteri. Infine, UTF-32 utilizza 32 bit per ogni carattere. Ogni mese di Giugno viene rilasciata una nuova versione di Unicode; nel Giugno 2020 era stata rilasciata la versione 13.0. Le versioni 9.0 e 13.0 hanno aggiunto vari Emoji, mentre le versioni precedenti avevano aggiunto nuove lingue e i geroglifici. Il totale dei caratteri codificati è circa di 150 000. Per saperne di più *vedi* il sito web www.unicode.org.

L'insieme RISC-V contiene istruzioni esplicite per caricare e memorizzare questi gruppi di 16 bit, che sono detti *halfword* (mezze parole). L'istruzione *load half unsigned* (`lhu`, carica mezza parola senza segno) legge 16 bit dalla memoria e li posiziona nei bit più a destra di un registro e riempie i rimanenti 16 bit di zero. Come load byte, *load half* (`lh`) tratta le mezze parole come numeri dotati di segno e perciò estende il segno per riempire i 16 bit più significativi del registro. *Store half* (`sh`) prende i 16 bit meno significativi di un registro e li scrive in memoria. Possiamo copiare mezza parola con la seguente coppia di istruzioni:

```
lhu x19, 0(x10) // Leggi mezza parola (16 bit)
                // dall'indirizzo sorgente
sh  x19, 0(x11) // Scrivi mezza parola (16 bit)
                // all'indirizzo di destinazione
```

Le stringhe costituiscono una classe standard Java, con speciali supporti precostituiti e metodi predefiniti per la loro concatenazione, comparazione e conversione. A differenza del C, Java prevede una parola che contiene la lunghezza della stringa, in modo simile a quanto fa per i vettori.

Approfondimento. Il software RISC-V cerca di mantenere lo stack allineato agli indirizzi di quattro parole (*quad word* o parola quadrupla, 16 byte) per ottenere prestazioni migliori. Questa convenzione comporta che una variabile di tipo char nello stack

possa occupare fino a 16 byte, anche se ne servirebbero di meno. In ogni caso, le variabili di tipo stringa o i vettori di byte in C *vengono compattati* in modo tale da inserire 16 byte per ogni parola quadrupla, mentre le stringhe Java e i vettori di valori interi codificati su 16 bit (short) vengono compattati in 8 elementi di mezza parola ciascuno per ogni parola quadrupla.

Approfondimento. La maggior parte delle pagine web oggigiorno utilizza il codice Unicode invece del codice ASCII, riflettendo così la natura internazionale del web. Quindi Unicode può essere considerato oggigiorno ancora più noto del codice ASCII.

Autovalutazione

I. Quali delle seguenti affermazioni su caratteri e stringhe in C e Java sono vere?
   1. Una stringa in C occupa metà dello spazio di memoria rispetto alla stessa stringa in Java.
   2. Le stringhe sono solo un nome informale per indicare vettori di caratteri sia in C sia in Java.
   3. Le stringhe in C e in Java utilizzano il carattere null (0) per segnalare il termine della stringa.
   4. Le operazioni sulle stringhe, come il calcolo della loro lunghezza, sono più veloci in C che in Java.

II. Quali tipi di variabili in grado di memorizzare il numero $1\,000\,000\,000_{dec}$ richiedono più spazio in memoria?
   1. `int` in C
   2. `string` in C
   3. `string` in Java

QUADRO D'INSIEME

I neofiti della programmazione sono sorpresi dal fatto che il tipo di un dato non sia codificato all'interno del dato, ma sia invece codificato nel programma che opera su quel dato. Per illustrare la problematica, vi forniamo un esempio nel linguaggio naturale. Cosa significa la parola "won"? Non potete rispondere a questa domanda senza conoscere il contesto, in particolare la lingua nella quale la parola viene pronunciata. Infatti, "won":

1. In inglese è un verbo ed è il passato remoto di "to win".
2. In koreano è un nome ed è la moneta in vigore nella Corea del Sud.
3. In polacco, è un aggettivo e significa "buon odore".
4. In russo, è un aggettivo e significa "cattivo odore".

Anche un numero binario può rappresentare diversi tipi di dati. Per esempio, la sequenza di 32 bit:

```
01100010 01100001 01010000 00000000
```

può rappresentare:

1. 1 650 544 640, se la sequenza viene considerata un numero intero senza segno;
2. +1 650 544 640, se la sequenza viene considerata un numero intero con segno;
3. "baP", se il programma considera la sequenza una stringa ASCII terminata con il carattere "null";

 © 978-88-08-19966-9

QUADRO D'INSIEME *(continua)*

4. Il colore blu scuro, se il programma considera la sequenza come la codifica di una mistura dei quattro colori di base nel sistema Pantone di codifica dei colori: turchese, magenta, giallo e nero.

Nel quadro d'insieme alla fine del Paragrafo 2.5 abbiamo sottolineato come anche le istruzioni vengano rappresentate come numeri, per cui una sequenza di 32 bit può anche rappresentare un'istruzione RISC-V espressa in linguaggio macchina:

```
0000 0000 1001 1010 1000 0100 1011 0011
```

corrisponde all'istruzione assembler di somma:

```
add x9, x21, x9
```

Se per sbaglio fornite un'immagine come documento a un programma di elaborazione testi, il programma cercherà di interpretarla come testo e vedrete una strana immagine comparire sullo schermo. Succederà qualcosa di simile se fornite dei dati contenenti del testo a un programma di visualizzazione grafica. Questa mancanza di vincoli nei computer basati su programmi memorizzati è il motivo per cui i file seguono una convenzione secondo la quale l'estensione denota il tipo di file (per esempio .jpg, .pdf, .txt). Questo consente ai programmi di controllare se l'estensione del file è corretta prima di caricare il file, in modo da ridurre il verificarsi di queste situazioni imbarazzanti. ■

2.10 Indirizzamento RISC-V di un campo immediato e di un indirizzo ampio

Anche se mantenere tutte le istruzioni RISC-V a 32 bit semplifica l'hardware, ci sono casi in cui sarebbe utile avere una costante o un indirizzo a 32 bit. Questo paragrafo inizia illustrando la soluzione generale al problema delle costanti molto grandi e prosegue mostrando le ottimizzazioni adottate per comporre gli indirizzi nelle istruzioni di salto.

Operandi immediati ampi

Sebbene le costanti siano molto spesso piccole e possano quindi trovare spazio all'interno del campo di 12 bit a loro assegnato, qualche volta sono più grandi.

Per questo motivo l'insieme di istruzioni RISC-V include l'istruzione *load upper immediate* (`lui`) che permette di caricare i 20 bit più significativi di una costante nei bit da 12 a 31 di un registro, e riempie i 12 bit più a destra del registro con zero. Questa istruzione consente, per esempio, di creare una costante su 32 bit con due istruzioni. L'istruzione `lui` utilizza un nuovo formato di istruzione, tipo U, dato che gli altri formati non possono ospitare una costante così ampia.

Caricamento di una costante su 32 bit

ESEMPIO

Qual è il codice assembler RISC-V utilizzato per caricare la seguente costante a 32 bit nel registro `x19`?

```
00000000 00111101 00000101 00000000
```

(continua)

(continua)

SOLUZIONE

Per prima cosa attraverso l'istruzione `lui` si caricano i bit dalla posizione 12 alla 31, che costituiscono il numero 976 in decimale:

```
lui x19, 976 // 976 in decimale = 0000 0000 0011 1101 0000
```

A questo punto il registro `x19` contiene:

```
00000000 00111101 00000000 00000000
```

Il passo successivo consiste nell'inserire nei 12 bit meno significativi della costante, il cui valore è 1280 in notazione decimale:

```
addi x19, x19, 1280   // 1280dec = 00000101 00000000
```

Il valore finale contenuto nel registro `x19` è il valore desiderato:

```
00000000 00111101 00000101 00000000
```

Approfondimento. Nell'esempio precedente, il bit 11 della costante era uguale a 0. Se il bit 11 fosse stato uguale a 1 ci sarebbe stata una complicazione aggiuntiva: viene esteso automaticamente il bit di segno del campo immediato su 12 bit, per cui l'addendo della `addi` sarebbe risultato negativo. Questo significa che oltre a sommare gli 11 bit più a destra della costante, sottrarremmo al numero finale 2^{12}. Per correggere questo errore, è sufficiente aggiungere 1 alla costante caricata con la `lui`, dato che la costante caricata è scalata per 2^{12}.

Interfaccia hardware/software

I compilatori o gli assemblatori devono spezzare le costanti molto grandi e successivamente assemblarle di nuovo in un registro. Come ci si può aspettare, la dimensione limitata del campo riservato alle costanti nelle istruzioni di tipo immediato può rappresentare un problema, sia per gli indirizzi di memoria nelle operazioni di trasferimento dati sia per le costanti.

In conclusione, la rappresentazione simbolica delle istruzioni in linguaggio macchina RISC-V non è più limitata dall'hardware, ma da ciò che il progettista dell'assemblatore ha scelto di supportare (Paragrafo 2.12). Noi abbiamo scelto di rimanere legati all'hardware per spiegare l'architettura del calcolatore, rendendo esplicito l'eventuale utilizzo di istruzioni avanzate dell'assemblatore che non fanno però parte dell'architettura del processore.

Indirizzamento nei salti

Le istruzioni di salto condizionato RISC-V utilizzano un formato che contiene un campo immediato a 12 bit. Questo formato può rappresentare indirizzi di salto da –4096 a 4094 in multipli di 2. Per i motivi che spiegheremo nel Capitolo 4, è possibile saltare solo a indirizzi pari. L'istruzione

```
bne x10, x11, 2000  // se x10 != x11, vai
                    // all'indirizzo 2000dec = 0111 1101 0000
```

potrebbe essere codificata nel formato S (in realtà le cose sono un po' più complicate, *vedi* Paragrafo 4.4):

0011111	01011	01010	001	01000	1100111
imm[12:6]	rs2	rs1	funz3	imm[5:1]	codop

dove il codice operativo dei salti condizionati è 1100111_{due} e il codice funz3 della `bne` è 001_{due}.

L'istruzione di salto incondizionato *jump-and-link* (`jal`) utilizza un formato che prevede un codice operativo su 7 bit, un registro operando di destinazione su 5 bit e un indirizzo immediato su 20 bit. L'indirizzo ritorno, che è l'indirizzo dell'istruzione successiva alla `jal`, viene scritto in rd. Quindi l'istruzione,

```
jal  x0, 2000  // vai all'indirizzo 2000dec  = 0111 1101 0000
```

potrebbe essere codificata in linguaggio macchina nel formato U (il Paragrafo 4.4 mostrerà il formato reale dell'istruzione `jal`):

00000000001111101000	00000	1101111
imm[20:1]	rd	codop

Se gli indirizzi del programma trovassero posto in questo campo a 20 bit, risulterebbe che nessun programma potrebbe avere una dimensione superiore a 2^{20}, troppo piccola per essere utilizzata nelle applicazioni reali. Una valida alternativa consiste nello specificare un registro il cui contenuto deve essere sommato all'indirizzo del salto; l'istruzione di salto dovrebbe quindi effettuare il seguente calcolo:

Program counter = Registro + Spiazzamento del salto

Questa somma consentirebbe a un programma di indirizzare 2^{32} posizioni pur continuando a utilizzare i salti condizionati, risolvendo il problema della dimensione dell'indirizzo di salto. La domanda che dobbiamo porci è: quale registro si deve utilizzare?

La risposta proviene dall'analisi di come i salti condizionati vengono utilizzati: essi si trovano tipicamente nei cicli e nei costrutti *if* e quindi di solito eseguono salti a istruzioni vicine. Per fare un esempio, quasi la metà delle istruzioni di salto condizionato nei programmi appartenenti ai benchmark SPEC saltano a indirizzi che distano meno di 16 istruzioni.

Dal momento che il program counter (PC) contiene l'indirizzo dell'istruzione corrente, si può saltare con un salto condizionato fino a una distanza di $\pm 2^{10}$ parole rispetto all'istruzione corrente, e con un salto incondizionato fino a $\pm 2^{18}$ parole rispetto all'istruzione corrente, utilizzando il PC come registro da sommare all'indirizzo di salto. Dato che quasi tutti i cicli e i costrutti *if* hanno dimensione inferiore alle 2^{10} parole, il PC è la scelta ideale. Questo metodo di indirizzamento per i salti si chiama **indirizzamento relativo al program counter** (*PC-relative addressing*).

Indirizzamento relativo al program counter: una modalità di indirizzamento secondo la quale l'indirizzo è ottenuto mediante la somma del program counter (PC) e il campo costante contenuto nell'istruzione.

Come molti dei calcolatori più recenti, il RISC-V utilizza l'indirizzamento relativo al PC per tutti i tipi di salto condizionato e incondizionato, dato che l'indirizzo di destinazione ha un'alta probabilità di essere vicino a quello dell'istruzione di salto. D'altra parte le chiamate a procedura facilmente richiedono di saltare più di 2^{18} parole, dato che non è garantito che il chiamante si trovi vicino in memoria al chiamato. Quindi il RISC-V consente di effettuare salti molto lunghi a uno qualsiasi tra 2^{32} indirizzi utilizzando una sequenza di due istruzioni: `lui` scrive i bit da 12 a 31 in un registro temporaneo e `jalr` somma i 12 bit meno significativi all'indirizzo contenuto nel registro temporaneo e salta all'indirizzo ottenuto con la somma.

Dato che le istruzioni RISC-V sono ampie 4 byte, le istruzioni di salto RISC-V sono state progettate per ampliare il loro spazio di indirizzamento definendo l'indirizzo relativo al PC in termini di numero di *parole* tra l'istruzione corrente di salto e l'istruzione di destinazione del salto, invece che in termini di numero di byte. Tuttavia, gli architetti RISC-V hanno voluto supportare la possibilità che le istruzioni siano ampie solo 2 byte, per cui lo spiazzamento

viene definito nelle istruzioni di salto in termini di *mezze parole* che intercorrono tra l'indirizzo dell'istruzione corrente di salto e quello di destinazione del salto. Quindi il campo di indirizzi di 20 bit nell'istruzione `jal` può codificare una distanza di $\pm 2^{19}$ mezze parole, o ±1 MiB a partire dal valore attuale del PC. Analogamente, il campo di 12 bit delle istruzioni di salto condizionato è espresso anch'esso in termini di mezze parole; questo vuol dire che rappresenta un indirizzo di 13 bit in termini di byte.

Calcolo dello spiazzamento dei salti condizionati in linguaggio macchina

ESEMPIO

Il ciclo *while* riportato nel Paragrafo 2.7 è stato compilato nelle seguenti istruzioni assembler:

```
Ciclo: slli x10, x22, 2  // registro temporaneo x10 = i * 4
       add x10, x10, x25 // x10 = indirizzo di salva[i]
       lw x9, 0(x10)     // registro temporaneo x9 = salva[i]
       bne x9, x24, Esci // vai a Esci se salva[i] ≠ k
       addi x22, x22, 1  // i = i + 1
       beq x0, x0, Ciclo // vai a Ciclo
Esci:
```

Se si suppone che il ciclo sia contenuto in memoria a partire dalla locazione 80000, quale sarà il codice macchina RISC-V corrispondente?

SOLUZIONE

Le istruzioni in linguaggio macchina e gli indirizzi corrispondenti vengono riportati di seguito.

Indirizzo	Istruzione					
80000	0000000	00010	10110	001	01010	0010011
80004	0000000	11001	01010	000	01010	0110011
80008	0000000	00000	01010	011	01001	0000011
80012	0000000	11000	01001	001	01100	1100011
80016	0000000	00001	10110	000	10110	0010011
80020	1111111	00000	00000	000	01101	1100011

Si ricordi che le istruzioni RISC-V indirizzano il byte, quindi gli indirizzi di parole consecutive differiscono di quattro unità, cioè il numero di byte in una parola. L'istruzione `bne` della quarta riga somma 12 byte (e quindi 3 parole) all'indirizzo dell'istruzione; specifica quindi l'indirizzo di destinazione del salto come distanza dall'istruzione di salto (12 + 80012) e non come indirizzo assoluto dell'istruzione di arrivo (80024). L'istruzione di branch dell'ultima riga utilizza un calcolo simile per saltare all'indietro (–20 + 80020), che corrisponde all'etichetta `Ciclo`.

Approfondimento. Nei Capitoli 2 e 3, supponiamo che i salti condizionati e incondizionati utilizzino i formati S e U per ragioni pedagogiche. In realtà, queste istruzioni utilizzano i formati, SB e UJ rispettivamente, che prevedono gli stessi campi con la stessa ampiezza dei formati S e U, ma che hanno alcuni bit dell'indirizzo spostati. Il razionale dietro questa codifica diventa chiaro dopo avere capito il funzionamento dell'hardware. Come vedremo nel Capitolo 4, i formati SB e UJ semplificano l'hardware ma producono un lavoro maggiore agli assemblatori (e agli autori!). Le Figure 4.17 e 4.18 mostrano i vantaggi per l'hardware.

Interfaccia hardware/software

La quasi totalità delle istruzioni di salto condizionato ha come destinazione una locazione di memoria vicina, ma occasionalmente può accadere di dover eseguire un salto condizionato a una distanza tale che non può essere rappresentata nei 12 bit del campo indirizzo di questo tipo di istruzioni.

L'assemblatore risolve il problema nello stesso modo in cui risolveva il problema degli indirizzi o delle costanti molto grandi: inserisce un salto incondizionato all'istruzione di arrivo del salto, e inverte il test sui registri in modo tale che l'istruzione di salto condizionato decida se saltare o meno l'istruzione di salto incondizionato.

Salto a indirizzi lontani

ESEMPIO

Si consideri il seguente salto condizionato che viene preso quando il contenuto di `x10` è uguale a zero:

```
beq x10, x0, L1
```

Si sostituisca l'istruzione `beq` con una coppia di istruzioni che permetta di saltare a una distanza molto maggiore.

SOLUZIONE

Queste istruzioni sostituiscono il salto condizionato a un indirizzo vicino:

```
        bne x10, x0, L2
        jal x0, L1
L2:
```

Riassunto delle modalità di indirizzamento del RISC-V

Modalità di indirizzamento: uno dei possibili modi di indirizzamento definito dal diverso utilizzo degli operandi e/o degli indirizzi.

I diversi formati di indirizzamento sono in generale chiamati **modalità di indirizzamento**. La Figura 2.17 mostra come sono utilizzati gli operandi in ciascuna modalità di indirizzamento, che nel RISC-V sono le seguenti:

1. *Indirizzamento immediato* (*immediate addressing*), in cui l'operando è una costante contenuta nell'istruzione stessa;
2. *Indirizzamento tramite registro* (*register addressing*), in cui l'operando è un registro;
3. *Indirizzamento tramite base e spiazzamento* (*base and displacement addressing*), in cui l'operando è in una locazione di memoria individuata dalla somma del contenuto di un registro e di una costante contenuta nell'istruzione stessa;
4. *Indirizzamento relativo al program counter* (*PC-relative addressing*), in cui l'indirizzo di salto è la somma del contenuto del program counter e di una costante contenuta nell'istruzione stessa.

Come decodificare il linguaggio macchina

A volte occorre fare reverse-engineering del linguaggio macchina per creare il codice assembler originario. Uno di questi casi è quando si verifica un "core dump". La Figura 2.18 mostra la codifica RISC-V dei codici operatici del linguaggio macchina del RISC-V.

Questa figura vi aiuterà quando dovrete tradurre a mano dal codice in linguaggio macchina al codice assembler e viceversa.

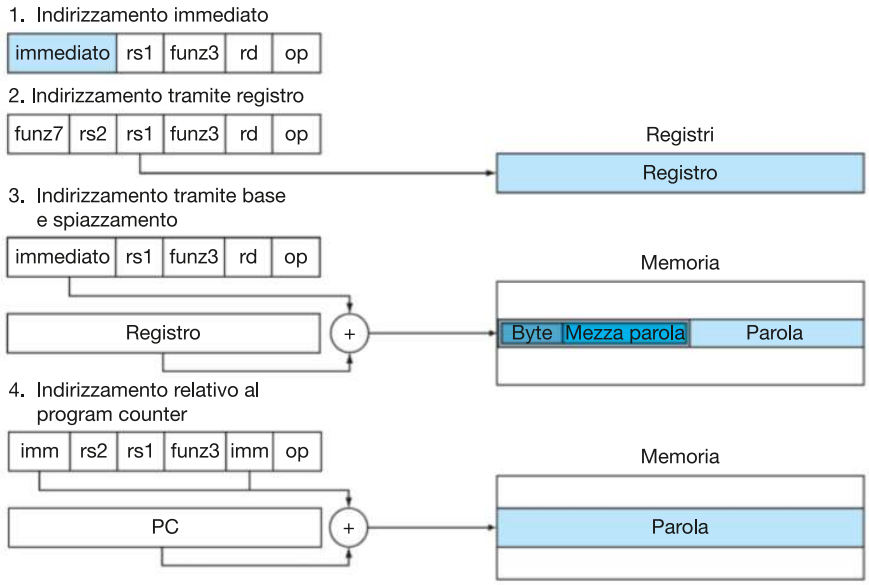


Figura 2.17 Illustrazione delle quattro modalità di indirizzamento del RISC-V. Gli operandi sono evidenziati in blu. L'operando della modalità 3 si trova in memoria, mentre quello della modalità 2 si trova in un registro. Si noti che le varianti delle istruzioni di load e store possono accedere al byte, alla mezza parola o alla parola. Nella modalità 1, l'operando è contenuto nell'istruzione stessa. La modalità 4 viene utilizzata per indirizzare le istruzioni in memoria, aggiungendo un indirizzo ampio al PC. Si noti che un'operazione può utilizzare diverse modalità di indirizzamento: la somma, per esempio, può avere sia un operando immediato (`addi`) sia tutti gli operandi nei registri (`add`).

Come decodificare le istruzioni in linguaggio macchina

ESEMPIO

Qual è l'istruzione assembler corrispondente alla seguente istruzione in linguaggio macchina?

00578833_{esa}

SOLUZIONE

Il primo passo consiste nel convertire il valore esadecimale in binario:

```
0000 0000 0101 0111 1000 1000 0011 0011
```

Per capire come interpretare i diversi bit, occorre determinare il formato dell'istruzione, e per fare ciò esaminiamo per prima cosa il codice operativo. Questo è definito dai sette bit più a destra dell'istruzione, cioè 0110011. Cercando questo numero nella Figura 2.18, vediamo che il codice operativo corrisponde alle istruzioni aritmetiche di tipo R. Possiamo quindi suddividere la stringa dell'istruzione nei campi riportati nella **Figura 2.19**.

funz7	rs2	rs1	funz3	rd	codop
0000000	00101	01111	000	10000	0110011

Decodifichiamo il resto dell'istruzione analizzzando i valori contenuti nei diversi campi. I campi funz7 e funz3 contengono entrambi zero, indicando che l'istruzione è una `add`. Il valore decimale dei registri degli operandi è 5 per il campo rs2, 15 per rs1 e 16 per rd. Questi numeri rappresentano i registri `x5`, `x15` e `x16`. Siamo ora in grado di scrivere l'istruzione assembler:

```
add x16, x15, x5
```

Formato	Istruzione	Codice Operativo	funz3	funz6/7
Tipo R	add	0110011	000	0000000
	sub	0110011	000	0100000
	sll	0110011	001	0000000
	xor	0110011	100	0000000
	srl	0110011	101	0000000
	sra	0110011	101	0000000
	or	0110011	110	0000000
	and	0110011	111	0000000
	lr.d	0110011	011	0001000
	sc.d	0110011	011	0001100
Tipo I	lb	0000011	000	n.a.
	lh	0000011	001	n.a.
	lw	0000011	010	n.a.
	ld	0000011	011	n.a.
	lbu	0000011	100	n.a.
	lhu	0000011	101	n.a.
	lwu	0000011	110	n.a.
	addi	0010011	000	n.a.
	slli	0010011	001	000000
	xori	0010011	100	n.a.
	srli	0010011	101	000000
	srai	0010011	101	010000
	ori	0010011	110	n.a.
	andi	0010011	111	n.a.
	jalr	1100111	000	n.a.
Tipo S	sb	0100011	000	n.a.
	sh	0100011	001	n.a.
	sw	0100011	010	n.a.
	sd	0100011	111	n.a.
Tipo SB	beq	1100111	000	n.a.
	bne	1100111	001	n.a.
	blt	1100111	100	n.a.
	bge	1100111	101	n.a.
	bltu	1100111	110	n.a.
	bgeu	1100111	111	n.a.
Tipo U	lui	0110111	n.a.	n.a.
Tipo UJ	jal	1101111	n.a.	n.a.

Figura 2.18 Codifica delle istruzioni RISC-V. Tutte le istruzioni hanno un codice operativo, e tutti i formati tranne il tipo U utilizzano il campo funz3. Le istruzioni di tipo R utilizzano il campo funz7, e le istruzioni di scorrimento mediante costante (`slli, srli, srai`) utilizzano il campo funz6. I formati SB e UJ verranno trattati nel Capitolo 4.

La Figura 2.19 mostra tutti i formati delle istruzioni RISC-V. Nella Figura 2.1 avevamo mostrato le istruzioni assembler RISC-V che sono descritte in questo capitolo. Il prossimo capitolo tratta le istruzioni RISC-V per moltiplicare e dividere, e l'aritmetica per i numeri decimali.

Nome (dimensione del campo)	Campi						Commenti
	7 bit	5 bit	5 bit	3 bit	5 bit	7 bit	
Tipo R	funz7	rs2	rs1	funz3	rd	codop	Istruzioni aritmetiche
Tipo I	Immediato[11:0]		rs1	funz3	rd	codop	Istruzioni di caricamento dalla memoria e aritmetica con costanti
Tipo S	immed[11:5]	rs2	rs1	funz3	immed[4:0]	codop	Istruzioni di trasferimento alla memoria (store)
Tipo U	immediato[31:12]				rd	codop	Formato caricamento stringhe di bit più significativi

Figura 2.19 Quattro dei formati delle istruzioni RISC-V. La Figura 4.16 rivelerà i due formati RISC-V mancanti: SB per i salti condizionati e UJ per i salti incondizionati. I formati SB e UJ prevedono gli stessi campi con la stessa ampiezza dei formati S e U ma che hanno alcuni bit dell'indirizzo spostati. Il razionale dietro questa codifica diventa chiaro dopo avere capito il funzionamento dell'hardware. Come vedremo nel Capitolo 4, i formati SB e UJ semplificano l'hardware ma producono un lavoro maggiore agli assemblatori (e agli autori!).

Autovalutazione

I. Qual è l'intervallo degli indirizzi raggiungibili da un salto condizionato nel RISC-V (K = 1024)?
   1. Indirizzi tra 0 e 4K – 1
   2. Indirizzi tra 0 e 8K – 1
   3. Indirizzi fino a 2K prima e fino a 2K dopo l'istruzione di salto
   4. Indirizzi fino a 4K prima e fino a 4K dopo l'istruzione di salto

II. Qual è l'intervallo degli indirizzi raggiungibili da un'istruzione di jump e di jump-and-link RISC-V (M = 1024K)?
   1. Indirizzi tra 0 e 512K – 1
   2. Indirizzi tra 0 e 1M – 1
   3. Indirizzi fino a 512K prima e fino a 512K dopo l'istruzione di salto
   4. Indirizzi fino a 1M prima e fino a 1M dopo l'istruzione di salto

2.11 Parallelismo e istruzioni: la sincronizzazione

PARALLELISMO

L'**esecuzione parallela** è più semplice quando i task sono indipendenti, ma spesso essi devono cooperare e la cooperazione di solito significa che uno dei task deve scrivere dei valori che devono essere letti da altri task. Per sapere quando un task ha terminato la scrittura, permettendo a un altro task di leggere i valori scritti, è necessaria una sincronizzazione. Se i task non fossero sincronizzati si potrebbe creare una **competizione sui dati** (*data race*), a causa della quale i risultati di un programma possono cambiare a seconda dell'ordine degli eventi.

Riprendiamo l'analogia con gli otto giornalisti che lavorano contemporaneamente a un articolo, descritta nel Paragrafo 1.6. Si supponga che uno dei giornalisti debba leggere il testo scritto dagli altri prima di potere scrivere la conclusione: dovrà sapere quando gli altri giornalisti hanno terminato di scrivere, in modo tale che queste parti non rischino di essere modificate dopo che lui le ha lette. Cioè, è opportuno che i giornalisti sincronizzino scrittura e lettura delle varie parti dell'articolo in modo tale che la conclusione sia consistente con il contenuto del resto dell'articolo.

Competizione sui dati: due accessi a memoria entrano in competizione per un dato se vengono effettuati da due thread diversi, riguardano lo stesso indirizzo di memoria, uno dei due accessi è in scrittura e si verificano uno subito dopo l'altro.

Nelle architetture i meccanismi di sincronizzazione sono tipicamente integrati in procedure software di livello utente basate su istruzioni hardware di

 © 978-88-08-19966-9

sincronizzazione. In questo paragrafo ci focalizzeremo sull'implementazione delle operazioni di sincronizzazione mediante un lucchetto virtuale che può assumere i valori *lock* (blocco) e *unlock* (sblocco). Lock e unlock possono essere utilizzate in modo semplice per creare delle regioni di dati, chiamate di *mutua esclusione*, sulle quali un solo processore alla volta può operare, ma anche per implementare meccanismi di sincronizzazione più complessi.

La funzionalità critica richiesta per implementare meccanismi di sincronizzazione nei multiprocessori è fornita da un insieme di primitive hardware in grado di leggere e modificare una locazione della memoria *atomicamente*, cioè in modo tale che non si possano compiere operazioni tra la lettura e la scrittura di quella locazione di memoria. Senza queste primitive hardware il costo di costruzione di primitive di sincronizzazione di base sarebbe elevato e crescerebbe in modo insostenibile con il numero dei processori.

Esistono diverse formulazioni possibili delle primitive hardware di base, ma tutte consentono di leggere e modificare atomicamente una locazione di memoria, e a tutte è associato un modo per riconoscere se lettura e scrittura siano state eseguite in modo veramente atomico. In generale, i progettisti non si aspettano che siano gli utenti a implementare le primitive hardware di base, ma si aspettano che le primitive vengano utilizzate dai programmatori di sistema per costruire una libreria di sincronizzazione, processo che si rivela spesso complesso e insidioso.

Iniziamo a esaminare una di queste primitive hardware e vediamo in che modo utilizzarla per costruire una primitiva di sincronizzazione di base. Una tipica operazione con cui costruire processi di sincronizzazione è lo *scambio atomico* (*atomic swap*), che scambia il contenuto di un registro con il contenuto di una parola di memoria.

Per vedere come utilizzare questa operazione, supponiamo di volere costruire un lucchetto semplice, in cui il valore 0 viene utilizzato per indicare che il lucchetto è aperto e 1 per indicare che la memoria associata al lucchetto non è disponibile (lucchetto chiuso). Un processore cerca di bloccare una locazione di memoria scambiando un 1, contenuto in un registro, con l'indirizzo di memoria che si vuole bloccare. Il valore restituito dalla funzione che esegue lo scambio sarà 1 (non disponibile) se un altro processore aveva già richiesto e ottenuto l'accesso a quella locazione di memoria, altrimenti sarà 0 (libera). In questo secondo caso, il lock verrà modificato e impostato a 1 (locazione non disponibile), in modo da evitare ulteriori competizioni con altri processori, che vedrebbero come valore del lock ancora 0 (locazione libera).

Per esempio, si considerino due processori che cercano di eseguire uno scambio simultaneamente: questa competizione sui dati (data race) viene interrotta, poiché solamente uno dei due processori potrà eseguire lo scambio per primo e la sua funzione di scambio restituirà uno 0 (libera), mentre la funzione del secondo processore restituirà 1 (non disponibile) perché non potrà completare l'operazione. La chiave per utilizzare le primitive di scambio per implementare scambi sincronizzati è che l'operazione sia atomica: lo scambio deve essere indivisibile e due scambi richiesti simultaneamente devono essere messi in sequenza dall'hardware. È impossibile che due processori possano impostare la variabile di sincronizzazione a 1.

L'implementazione di una singola operazione atomica sulla memoria introduce alcune sfide nella progettazione dei processori, dato che richiede che la lettura e la scrittura della memoria avvengano in un'unica istruzione non interrompibile.

Una possibile alternativa è avere una coppia di istruzioni nella quale la seconda restituisca un valore che indica che le due istruzioni sono state eseguite come una singola istruzione atomica. La coppia di istruzioni è effettivamente eseguita in modo atomico se si ha la certezza che tutte le altre operazioni

effettuate dai processori sono state eseguite prima di essa. In tal caso nessun altro processore può modificare il valore della memoria tra le due istruzioni.

Nel RISC-V questa coppia di istruzioni è costituita da un'istruzione di load speciale, chiamata *load-reserved word* (`lr.w`, load riservata di una parola) e da un'istruzione di store speciale chiamata *store-conditional word* (`sc.w`, store condizionata di una parola). Queste due istruzioni sono utilizzate in sequenza: se il contenuto di una locazione di memoria associata a una load riservata (linked) venisse alterato prima che la store condizionata associata abbia salvato in quella cella di memoria il dato, l'istruzione di store condizionata fallirebbe e non scriverebbe il dato in memoria.

L'istruzione di store condizionata ha quindi due funzioni: salva il contenuto di un registro (presumibilmente diverso) in memoria *e* imposta il contenuto di quel registro a 1, se la scrittura ha avuto successo, e a 0, se invece è fallita. Quindi l'istruzione `sc.w` specifica tre registri: uno che contiene l'indirizzo, uno per indicare se l'operazione atomica fallisce o ha successo e uno per il dato da scrivere in memoria se l'operazione ha avuto successo. Dato che l'istruzione load riservata restituisce il valore letto e l'istruzione store condizionata restituisce 1 solamente se la scrittura ha avuto successo, la sequenza di istruzioni seguente implementa uno scambio atomico relativo alla locazione di memoria specificata dal contenuto di `x20`:

```
tentativo: lr.w x10, (x20)         // carica il dato e
                                   // blocca la cella di memoria
           sc.w x11, x23, (x20)    // store condizionata
           bne  x11, x0, tentativo // ritorna a tentativo se la
                                   // store fallisce (= 0)
           addi x23, x10, 0        // inserisci il valore letto in x23
```

Se il processore intervenisse per modificare il contenuto della memoria in `x20` tra l'esecuzione dell'istruzione `lr.w` e della `sc.w`, l'istruzione `sc.w` scriverebbe un valore non 0 nel registro `x11`, forzando la ripetizione della sequenza di istruzioni. Al termine di questa sequenza di istruzioni il contenuto di `x23` e della locazione di memoria indicata da `x20` sono stati scambiati.

Approfondimento. Abbiamo presentato lo scambio atomico per la sincronizzazione dei sistemi multiprocessore, ma questo è utilizzato anche dal sistema operativo che gestisce più processi in esecuzione su un singolo processore. Per assicurarsi che niente interferisca con l'esecuzione sul processore, l'istruzione di store condizionata fallisce ogni volta che il processore esegue un cambio di contesto tra le due istruzioni (Capitolo 5).

Approfondimento. Un vantaggio del meccanismo della coppia load linked/store condizionata è che può essere utilizzato per costruire altre primitive di sincronizzazione, quali *atomic compare and swap* e *atomic fetch and increment*, utilizzate in alcuni modelli di programmazione parallela. Queste primitive richiedono un certo numero di istruzioni tra la `lr.w` e la `sc.w`, ma non tante.

Dato che l'istruzione di store condizionata potrà fallire sia dopo la richiesta di un altro tentativo di scrittura nell'indirizzo da cui una load linked ha letto il dato, sia dopo un'eccezione, molta attenzione deve essere posta nella scelta delle istruzioni da inserire tra questa coppia di istruzioni. In particolare, si possono inserire in modo sicuro solamente le istruzioni di aritmetica con gli interi, di salto in avanti e di salto all'indietro al di fuori del blocco compreso tra l'istruzione di load linked e store condizionata.

Viceversa, è possibile creare delle situazioni di stallo (*deadlock*) nelle quali il processore non può terminare l'esecuzione della `sc.w` perché continua a ricevere un'eccezione di page fault. Inoltre, il numero di istruzioni inserite tra la load linked e la store condizionata deve essere ridotto al minimo, in modo tale da minimizzare la probabilità che un evento scorrelato o un altro processore causino il fallimento frequente della store condizionata.

 © 978-88-08-19966-9

Approfondimento. Mentre il codice precedente eseguiva uno scambio atomico, il codice seguente blocca in modo più efficace la locazione di memoria contenuta nel registro x20, dove il valore 0 indica che la locazione è stata sbloccata, mentre 1 che il blocco è stato eseguito:

```
        addi x12, x0, 1      // copia del valore associato al blocco
ancora: lr.w x10, (x20)      // carica con il blocco per vedere se
                             // la locazione è bloccata
        bne  x10, x0, ancora // controlla se è ancora 0
        sc.w x11, x12,(x20)  // prova a memorizzare il nuovo valore
        bne  x11, x0, ancora // salta se la memorizzazione ha fallito
```

Per rilasciare il blocco è sufficiente utilizzare una normale istruzione di store per scrivere 0 nella locazione di memoria:

```
sw x0, 0(x20) // elimina il blocco scrivendo 0
```

Autovalutazione

Quando si utilizzano primitive come load linked riservata e store condizionata?

1. Quando processi cooperanti di un programma parallelo devono essere sincronizzati per ottenere un comportamento corretto nella lettura e nella scrittura di dati condivisi.
2. Quando processi cooperanti su un'architettura a singolo processore devono essere sincronizzati per leggere e scrivere dati condivisi.

2.12 Tradurre e avviare un programma

Questo paragrafo descrive i quattro passi necessari a trasformare un programma C, contenuto in un file in una memoria non volatile (disco o memoria flash), in un programma pronto per essere eseguito su un calcolatore. La Figura 2.20 mostra la sequenza delle trasformazioni; alcuni calcolatori possono raggruppare questi passi per ridurre il tempo di traduzione, ma concettualmente queste sono le quattro fasi attraverso le quali tutti i programmi devono passare. Noi seguiremo questa sequenza di trasformazioni.

Compilatore

Il compilatore trasforma il programma C in un *programma in linguaggio assembler*, cioè in una forma simbolica di ciò che il calcolatore è in grado di capire. I programmi scritti in linguaggio ad alto livello sono costituiti da un numero di righe di codice minore rispetto al linguaggio assembler e questo fa sì che la produttività di un programmatore sia più elevata.

Linguaggio assembler: un linguaggio simbolico che può essere tradotto nel linguaggio macchina binario.

Nel 1975 molti sistemi operativi e molti assemblatori erano scritti in **linguaggio assembler**, dal momento che la capacità della memoria era ridotta e i compilatori erano inefficienti. Oggi la capacità delle memorie per ogni singolo chip di DRAM è cresciuta di un milione di volte, riducendo il problema delle dimensioni dei programmi; inoltre, i compilatori ottimizzanti attuali sono in grado di produrre codice assembler quasi con la medesima qualità di quello scritto da programmatori assembler esperti e qualche volta addirittura migliore, nel caso di programmi di grandi dimensioni.

Assemblatore

Dato che il linguaggio assembler rappresenta l'interfaccia verso il software di livello più alto, l'assemblatore può anche trattare varianti delle istruzioni in

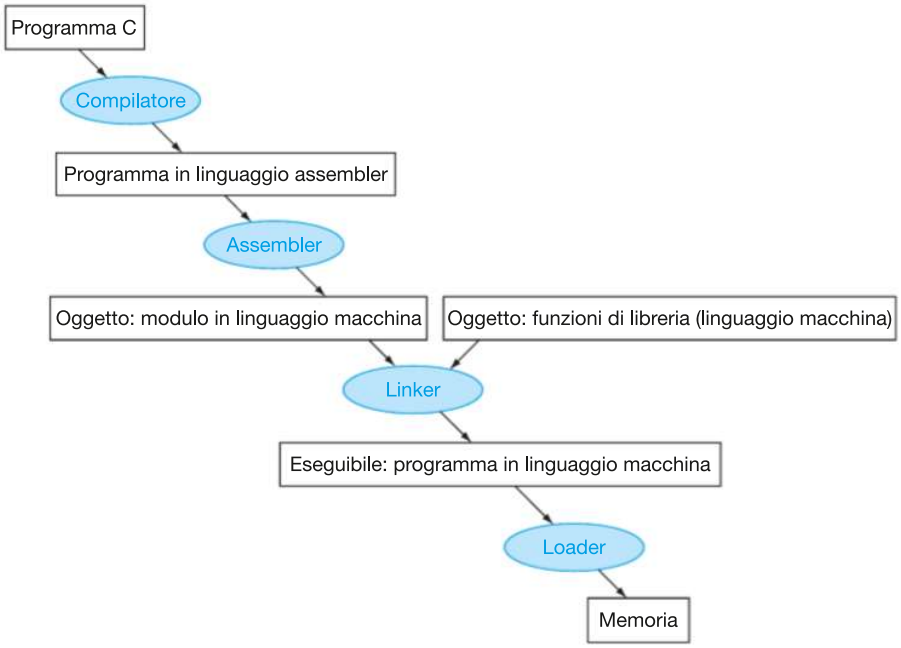


Figura 2.20 Sequenza di passi di traduzione per il C. Un programma scritto in linguaggio ad alto livello viene per prima cosa compilato in linguaggio assembler e poi assemblato per ottenere un modulo oggetto in linguaggio macchina. Il linker unisce uno o più moduli tra loro e con le procedure contenute nelle librerie, e risolve tutti i riferimenti incrociati. Il loader, infine, carica il codice macchina nell'opportuna area di memoria, in modo che possa essere eseguito dal processore. Per accelerare il processo di traduzione, alcuni passi possono essere saltati o eseguiti insieme ad altri. Alcuni compilatori producono direttamente il codice macchina mentre altri utilizzano dei linker-loader, i quali eseguono gli ultimi due passi contemporaneamente. Per identificare il tipo di file, il sistema operativo UNIX segue una precisa convenzione sull'estensione dei file: i file sorgente C vengono chiamati `x.c`, i file assembler `x.s`, i file oggetto `x.o`, le librerie collegate staticamente `x.a`, le librerie collegate dinamicamente `x.so` e i file eseguibili sono solitamente denominati `a.out`. Il sistema operativo MS-DOS utilizza come estensioni rispettivamente `.C`, `.ASM`, `.OBJ`, `.LIB`, `.DLL` e `.EXE` con lo stesso significato.

linguaggio macchina come se fossero istruzioni vere e proprie. Non è richiesto che l'hardware implementi queste istruzioni, ma esse consentono di semplificare la traduzione e la programmazione, aumentando la leggibilità del codice. Queste istruzioni sono chiamate **pseudoistruzioni**. Come detto in precedenza, l'hardware RISC-V si assicura che il registro `x0` contenga sempre il valore 0; quindi il registro `x0` fornisce il valore 0 ogni volta che viene letto e se il programmatore cerca di cambiarne il valore di `x0`, il nuovo valore viene semplicemente scartato. Il registro `x0` viene utilizzato per creare l'istruzione assembler che copia il contenuto di un registro in un altro. L'assemblatore RISC-V accetta l'istruzione seguente anche se non fa parte del linguaggio macchina RISC-V:

Pseudoistruzione: una variante di un'istruzione in linguaggio assembler frequentemente utilizzata; viene spesso trattata come una vera e propria istruzione.

```
li x9, 123    // carica il valore 123 nel registro x9
```

L'assemblatore converte questa istruzione assembler nell'istruzione equivalente in linguaggio macchina, che è:

```
addi x9, x0, 123 // il registro x9 assume il valore x0 + 123
```

L'assemblatore RISC-V converte anche la pseudoistruzione `mv` (*move*, sposta) in un'istruzione `addi`. Quindi:

```
mv x10, x11 // il registro x10 assume il valore del registro x11
```

diventa

```
addi x10, x11, 0 // Il registro x10 riceve il contenuto
                 // del registro x11 + 0
```

 © 978-88-08-19966-9

L'assemblatore accetta anche la pseudoistruzione `j Etichetta` come istruzione di salto incondizionato a `Etichetta`, che corrisponde all'istruzione `jal x0, Etichetta`. Converte anche i salti condizionati a locazioni lontane in un'istruzione di salto condizionato seguita da una di salto incondizionato. Come accennato in precedenza, l'assemblatore RISC-V consente di caricare delle costanti ampie nei registri nonostante la dimensione limitata dei campi immediati. Quindi, la pseudoistruzione *carica immediata* (`li`, *load immediate*) introdotta in precedenza può creare delle costanti più ampie di quelle che possono essere contenute nel campo immediato della `addi`; la macro *load address* (`la`) lavora in modo simile per gli indirizzi simbolici. Infine, può semplificare l'insieme delle istruzioni determinando quale variante di un'istruzione desideri il programmatore. Per esempio, l'assemblatore RISC-V non richiede che il programma specifichi la versione immediata dell'istruzione utilizzando una costante per le istruzioni aritmetiche e logiche; genera semplicemente il codice operativo adeguato. Quindi:

```
and x9, x10, 15 // registro x9 assume il valore
                // di x10 AND 15
```

diventa

```
andi x9, x10, 15 // registro x9 assume il valore
                 // di x10 AND 15
```

Abbiamo aggiunto la "`i`" nell'istruzione per ricordare al lettore che `andi` produce un codice operativo diverso con un formato di istruzione differente rispetto all'istruzione `and` che non ha operandi immediati.

Riassumendo, le pseudoistruzioni consentono all'assembler RISC-V di avere un insieme di istruzioni più ricco di quello implementato in hardware. Se dovete scrivere un programma assembler, è opportuno che utilizziate le pseudo-istruzioni per semplificarvi il lavoro. Tuttavia, per capire l'architettura RISC-V e ottenere le prestazioni migliori, occorre studiare a fondo le reali istruzioni RISC-V che trovate nelle Figure 2.1 e 2.18. Per ridurre la confusione sulla differenza tra istruzioni reali e pseudoistruzioni, i Capitoli 2 e 3 utilizzano solo istruzioni reali anche dove i programmatori assembler esperti utilizzerebbero le pseudoistruzioni.

Gli assemblatori, inoltre, accettano numeri espressi in basi diverse. Oltre alla base due e a quella decimale, solitamente accettano anche una base più concisa di quella binaria, che può essere facilmente convertita in una sequenza di bit: gli assemblatori RISC-V utilizzano la base esadecimale e ottale.

Tutte le caratteristiche dell'assemblatore appena illustrate sono molto utili, ma l'assemblatore ha come compito principale la generazione del linguaggio macchina: converte un programma assembler in un *file oggetto*, cioè una sequenza di istruzioni in linguaggio macchina, di dati e di informazioni necessarie a collocare le istruzioni in memoria nella posizione opportuna.

Per produrre la versione binaria di ogni istruzione di un programma assembler, l'assemblatore deve determinare gli indirizzi corrispondenti a tutte le etichette: esso tiene traccia di tutte le etichette utilizzate nei salti e nei trasferimenti di dati scrivendole in una tabella, detta **tabella dei simboli** (*symbol table*). Come si può immaginare, questa tabella conterrà coppie di tipo simbolo-indirizzo. I file oggetto nei sistemi UNIX tipicamente contengono sei segmenti distinti.

Tabella dei simboli: tabella che fa corrispondere i nomi delle etichette agli indirizzi delle parole di memoria contenenti le istruzioni.

- L'*intestazione del file oggetto* (*object file header*) descrive la dimensione e la posizione degli altri segmenti del file oggetto stesso.
- Il *segmento di testo* (*text segment*) contiene il codice in linguaggio macchina.
- Il *segmento di dati statici* (*static data segment*) contiene tutti i dati allocati per tutta la durata del programma. UNIX consente l'utilizzo sia di *dati statici*, che rimangono allocati per tutta la durata dell'esecuzione del

programma, sia di *dati dinamici*, che possono crescere o diminuire di dimensione a seconda delle esigenze del programma (Figura 2.13).
- Le *informazioni di rilocazione* (*relocation information*) identificano le istruzioni e i dati che, quando il programma è posto in memoria, dipendono da indirizzi assoluti.
- La *tabella dei simboli* (*symbol table*) contiene le rimanenti etichette di cui non è stata trovata una definizione, per esempio quelle che fanno riferimento a moduli esterni.
- Le *informazioni per il debugger* (*debugging information*) contengono una descrizione concisa di come sono stati compilati i moduli, in modo che il debugger (programma per il supporto della ricerca degli errori) possa associare le istruzioni in linguaggio macchina al codice sorgente C e rendere leggibili le strutture dati.

Il paragrafo seguente, relativo al linker, mostrerà come vengono collegate procedure già assemblate, per esempio quelle contenute nelle librerie.

Linker

Tutto ciò che abbiamo visto finora suggerisce che la modifica anche di una sola linea di codice richieda di ricompilare e riassemblare l'intero programma. Ma ritradurre tutto dall'inizio è un terribile spreco di risorse computazionali, soprattutto per quanto riguarda le librerie standard, che verrebbero ricompilate e riassemblate anche se, per definizione, esse non cambiano praticamente mai. Una valida alternativa consiste nel compilare e assemblare ciascuna procedura indipendentemente dalle altre, in modo che la modifica di una linea di codice renda necessario ricompilare e riassemblare solo la procedura a cui la linea di codice appartiene. Tale alternativa necessita di un nuovo programma di sistema, chiamato **link editor** o **linker**; esso prende tutti i programmi (le procedure) in codice macchina che sono stati assemblati indipendentemente e li "cuce" insieme. Il motivo per cui il linker è particolarmente utile è che risulta molto più veloce "correggere" il codice piuttosto che ricompilarlo e assemblarlo di nuovo. Il linker esegue i seguenti tre passi:

Link editor o linker: programma di sistema che unisce varie procedure in linguaggio macchina assemblate indipendentemente e risolve tutte le etichette non definite, producendo un programma eseguibile.

1. Inserisce in memoria in modo simbolico il codice e i moduli dati;
2. Determina gli indirizzi dei dati e delle etichette che compaiono nelle istruzioni;
3. Corregge i riferimenti interni ed esterni.

Il linker utilizza le informazioni di rilocazione e la tabella dei simboli di ciascun modulo oggetto per risolvere tutte le etichette non definite. Queste si trovano nelle istruzioni di salto e negli indirizzi dei dati; il compito del linker è quindi più o meno quello di un editor: trovare gli indirizzi vecchi e sostituirli con quelli nuovi. Questa attività di editing è all'origine del nome *link editor*, abbreviato in linker. Dopo che tutti i riferimenti esterni sono stati risolti, il linker determina le locazioni di memoria che ciascun modulo dovrà occupare. Nella Figura 2.13 avevamo mostrato la convenzione RISC-V per l'allocazione dei programmi e dei dati in memoria. Poiché tutti i file vengono assemblati in modo indipendente, l'assemblatore non può conoscere la posizione relativa delle istruzioni e dei dati di un modulo rispetto a un altro; perciò, quando il linker inserisce un modulo in memoria, tutti i riferimenti *assoluti*, cioè gli indirizzi di memoria definiti non in relazione a un registro, devono essere *rilocati* in modo da poter riflettere la loro reale posizione. Il linker produce un **file eseguibile** che può essere eseguito su un calcolatore. Di norma questo file ha lo stesso formato di un file oggetto, ma non contiene più riferimenti non risolti. È possibile eseguire questo passo anche in maniera parziale, come è il caso delle librerie, che contengono ancora indirizzi non risolti e quindi in realtà sono assimilabili a file oggetto.

File eseguibile: programma in grado di essere eseguito, scritto nel formato dei file oggetto ma che non contiene riferimenti non risolti. Può contenere tabelle di simboli e informazioni per il debugger. Un "eseguibile spogliato" (*stripped executable*) non contiene queste ultime informazioni. Le informazioni di rilocazione possono venire incluse per il loader.

 © 978-88-08-19966-9

Eseguire il link di due file oggetto

ESEMPIO

Si esegua il link dei due file oggetto riportati qui sotto, mostrando gli indirizzi aggiornati delle prime istruzioni del file eseguibile completo. Le istruzioni sono mostrate in linguaggio assembler solo per rendere l'esempio più leggibile, ma nella realtà le istruzioni sono rappresentate da numeri.

Si noti che nel file oggetto sono evidenziati gli indirizzi e i simboli che devono essere aggiornati nel processo di link, cioè le istruzioni che fanno riferimento agli indirizzi delle procedure `A` e `B` e quelle che fanno riferimento agli indirizzi delle parole dei dati `X` e `Y`.

Intestazione del file oggetto			
	Nome	Procedura A	
	Dimensione del testo	100_{esa}	
	Dimensione dei dati	20_{esa}	
Segmento testo	Indirizzo	Istruzione	
	0	`lw x10, 0(x3)`	
	4	`jal x1, 0`	
	…	…	
Segmento dati	0	(X)	
	…	…	
Informazioni di rilocazione	Indirizzo	Tipo di istruzione	Dipendenza
	0	`lw`	X
	4	`jal`	B
Tabella dei simboli	Etichetta	Indirizzo	
	X	–	
	B	–	
	Nome	Procedura B	
	Dimensione del testo	200_{esa}	
	Dimensione dei dati	30_{esa}	
Segmento testo	Indirizzo	Istruzione	
	0	`sw x11, 0(x3)`	
	4	`jal x1, 0`	
	…	…	
Segmento dati	0	(Y)	
	…	…	
Informazioni di rilocazione	Indirizzo	Tipo di istruzione	Dipendenza
	0	`sw`	Y
	4	`jal`	A
Tabella dei simboli	Etichetta	Indirizzo	
	Y	–	
	A	–	

(continua)

(continua)

SOLUZIONE

La procedura A deve trovare l'indirizzo della variabile etichettata X per inserirlo nell'istruzione di load e l'indirizzo della procedura B per inserirlo nell'istruzione jal. La procedura B ha bisogno dell'indirizzo della variabile etichettata Y per inserirlo nell'istruzione di store e dell'indirizzo della procedura A per la sua istruzione jal.

Dalla Figura 2.14 si ricava che il segmento di testo inizia all'indirizzo $0000\,0000\,0040\,0000_{esa}$ e che il segmento dati inizia in $0000\,0000\,1000\,0000_{esa}$. Il testo della procedura A viene inserito a partire dal primo dei due indirizzi e i suoi dati a partire dal secondo. L'intestazione del file oggetto della procedura A indica che il suo segmento testo è lungo 100_{esa} byte e che i suoi dati occupano 20_{esa} byte; quindi l'indirizzo di partenza del testo della procedura B sarà $40\,0100_{esa}$ e i suoi dati partiranno da $1000\,0020_{esa}$.

Intestazione del file eseguibile		
	Dimensione del testo	300_{esa}
	Dimensione dei dati	50_{esa}
Segmento testo	Indirizzo	Istruzione
	$0000\ 0000\ 0040\ 0000_{esa}$	`lw x10, 0(x3)`
	$0000\ 0000\ 0040\ 0004_{esa}$	`jal x1,` 252_{dec}
	...	...
	$0000\ 0000\ 0040\ 0100_{esa}$	`sw x11, 32(x3)`
	$0000\ 0000\ 0040\ 0104_{esa}$	`jal x1,` -260_{dec}
	...	...
Segmento dati	Indirizzo	
	$0000\ 0000\ 1000\ 0000_{esa}$	(X)
	...	...
	$0000\ 0000\ 1000\ 0020_{esa}$	(Y)
	...	...

A questo punto il linker aggiorna il campo indirizzo delle istruzioni, utilizzando il campo del codice operativo per riconoscere il formato dell'indirizzo. In questo caso sono presenti tre tipi di istruzioni:

1. Le istruzioni jal utilizzano l'indirizzamento relativo al PC. Il campo indirizzo dell'istruzione jal, che si trova all'indirizzo $40\,0004_{esa}$ e salta all'indirizzo $40\,0100_{esa}$ (l'indirizzo della procedura B), conterrà quindi il valore ($40\,0100_{esa} - 40\,0004_{esa}$) o 252_{dec} nel suo campo indirizzo. Analogamente, dato che $40\,0000_{esa}$ è l'indirizzo della procedura A, la jal a $40\ 0104_{esa}$ conterrà il numero negativo -260_{dec} ($40\,0000_{esa} - 40\,0104_{esa}$) nel proprio campo indirizzo.
2. Gli indirizzi delle istruzioni di trasferimento dati con la memoria sono un po' più complessi da ricavare, dal momento che sono relativi a un registro di base. In questo esempio viene utilizzato il registro x3 come registro base, che supponiamo inizializzato a $0000\,0000\,1000\,0000_{esa}$. Per ottenere l'indirizzo della parola X ($0000\,0000\,1000\,0000_{esa}$), inseriamo 0_{dec} nel campo indirizzo dell'istruzione lw all'indirizzo $40\,0000_{esa}$. Allo stesso modo inseriamo 20_{esa} nel campo indirizzo dell'istruzione sw all'indirizzo $40\,0100_{esa}$ per ottenere l'indirizzo $0000\,0000\,1000\,0020_{esa}$ (l'indirizzo della parola Y).

(continua)

(continua)

3. Gli indirizzi associati alle operazioni di store vengono gestiti esattamente come nelle operazioni di load, a eccezione del fatto che il formato delle istruzioni di tipo S rappresenta le costanti in modo differente dal formato I delle load. Inseriamo 32_{dec} nel campo indirizzo della `sw` all'indirizzo $40\,0100_{esa}$ per ottenere l'indirizzo $0000\,0000\,1000\,0020_{esa}$ (l'indirizzo della parola `Y`).

Loader

Loader: programma di sistema che carica un programma oggetto nella memoria principale in modo che sia pronto per essere eseguito.

Una volta che il file eseguibile è stato memorizzato su disco, il sistema operativo può leggerlo e trasferirlo in memoria per avviarne l'esecuzione. Nei sistemi UNIX il **loader** (programma di caricamento) esegue i seguenti passi.

1. Lettura dell'intestazione del file eseguibile per determinare la lunghezza del segmento testo e del segmento dati.
2. Creazione di uno spazio di indirizzamento sufficiente a contenere testo e dati.
3. Copia delle istruzioni e dei dati dal file eseguibile in memoria.
4. Copia nello stack degli eventuali parametri passati al programma principale.
5. Inizializzazione dei registri del calcolatore e impostazione dello stack pointer affinché punti alla prima locazione libera.
6. Salto a una procedura di *start-up* (avviamento), la quale copia i parametri nei registri argomento e chiama la procedura principale del programma. Quando la procedura principale restituisce il controllo, la procedura di startup termina il programma con una chiamata alla funzione di sistema `exit`.

Librerie a caricamento dinamico

Praticamente tutti i problemi dell'informatica possono essere risolti con un altro livello di indirizzamento.
David Wheeler

Nella prima parte di questo paragrafo abbiamo descritto l'approccio tradizionale, detto *statico*, al collegamento delle librerie che vengono mappate prima che il programma sia eseguito. Nonostante l'approccio statico sia il modo più veloce per chiamare le funzioni di libreria, ci sono alcuni svantaggi:

- Le funzioni di libreria diventano parte del codice eseguibile. Se viene rilasciata una nuova versione della libreria, che per esempio può contenere la correzione di alcuni errori o supportare nuovi dispositivi hardware, un programma che carica staticamente le librerie continua a utilizzare la vecchia versione.
- Vengono caricate tutte le funzioni di libreria utilizzate dal programma, qualsiasi sia il punto del programma nel quale le funzioni vengono chiamate, comprese quelle funzioni che non vengono utilizzate. La libreria può essere molto più grande del programma; per esempio la libreria standard C di un sistema RISC-V su cui è montato il sistema operativo Linux è di 1,5 MiB.

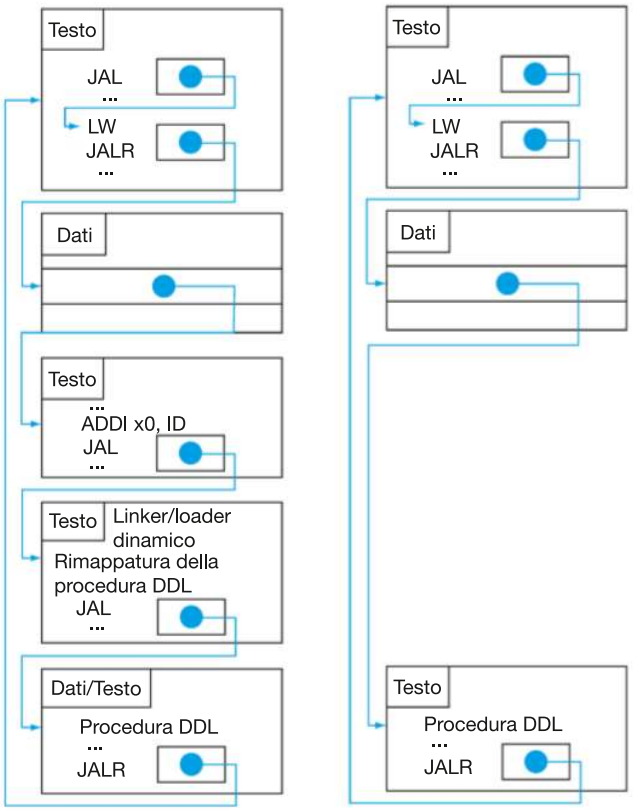
Librerie a caricamento dinamico (DLL): procedure di libreria che vengono collegate a un programma durante la sua esecuzione.

Questi svantaggi hanno portato allo sviluppo delle **librerie a caricamento dinamico** (**DLL**, *Dynamically Linked Libraries*), per cui le funzioni di libreria non vengono collegate e caricate finché non inizia l'esecuzione del programma. Sia il programma sia le librerie contengono ulteriori informazioni riguardanti la posizione delle procedure non locali e il loro nome. Nella versione originale delle DLL, il loader lanciava un linker dinamico, utilizzando queste ulteriori informazioni presenti nel file per cercare le librerie appropriate e per aggiornare tutti i riferimenti esterni. L'inconveniente era che anche in questo

caso venivano caricate tutte le funzioni di libreria disponibili, invece di caricare solamente quelle utilizzate effettivamente durante l'esecuzione del programma. Questa osservazione ha portato alla versione delle DLL con collegamento *lazy* (pigro), in cui ogni procedura viene caricata solo *dopo* la sua chiamata.

Come per molti altri casi nel nostro campo, questo trucco è basato sulla rimappatura, come mostrato nella **Figura 2.21**. Ciascuna procedura non locale inizia chiamando una procedura fasulla (*dummy*), inserita subito dopo la fine del codice del programma; questa procedura contiene un'istruzione di salto indiretto: si inserisce quindi una procedura fasulla diversa per ognuna delle procedure non locali.

La prima volta che la funzione di libreria viene chiamata, il programma salta all'istruzione fasulla a essa associata ed esegue il salto indiretto indicato. L'istruzione di salto porta a una porzione di codice che inserisce un numero in un registro, il quale identifica la funzione di libreria desiderata, e quindi salta a sua volta al linker-loader dinamico. Il linker-loader dinamico trova la funzione di libreria che si sta cercando, la ri-mappa in memoria e cambia l'indirizzo contenuto nell'istruzione di salto indiretto, in modo che la procedura fasulla salti ora alla prima istruzione della funzione richiesta. A questo punto la funzione è disponibile e si ritornerà al punto originale di chiamata una volta completata la sua esecuzione. In seguito la chiamata alla funzione



(a) Prima chiamata a una procedura di una DDL

(b) Chiamate successive alla procedura della DDL

Figura 2.21 Procedura per il caricamento dinamico delle librerie a collegamento *lazy*. (a) Passi seguiti per la prima chiamata a una procedura di una DLL. **(b)** I passi associati alla ricerca della procedura, alla sua rimappatura e al suo collegamento vengono saltati nelle chiamate successive. Come vedremo nel Capitolo 5, il sistema operativo può evitare di copiare la funzione desiderata, rimappandola mediante l'utilizzo della memoria virtuale.

di libreria prevederà un salto indiretto unico alla sua prima istruzione, senza salti addizionali.

Riassumendo, le DLL richiedono spazio aggiuntivo per memorizzare le informazioni necessarie per il collegamento dinamico, ma non richiedono di caricare o collegare l'intera libreria. Queste librerie pagano un costo computazionale significativo la prima volta che una funzione viene chiamata, ma solamente il costo di un salto indiretto in tutte le chiamate successive. Si noti che il ritorno da queste funzioni di libreria non richiede alcun costo aggiuntivo. Il sistema operativo Windows di Microsoft è basato pesantemente sulle librerie a caricamento dinamico, che costituiscono anche lo standard negli attuali sistemi UNIX.

Come avviare un programma Java

La discussione precedente ha riguardato il modello tradizionale di esecuzione di un programma, dove si dà importanza alla velocità di esecuzione per una specifica architettura dell'insieme di istruzioni o per una sua specifica implementazione. Anche se è possibile eseguire un programma Java come se fosse un programma C, Java è stato creato con obiettivi differenti: uno dei più importanti è quello di poter eseguire correttamente una applicazione su qualsiasi calcolatore, eventualmente con velocità di esecuzione diverse.

La **Figura 2.22** mostra i passaggi tipici della traduzione ed esecuzione di un programma Java. Diversamente dal caso in cui le applicazioni vengono compilate traducendole nel linguaggio assembler del calcolatore, i programmi Java vengono tradotti, in prima istanza, in un insieme di istruzioni universali e semplici da interpretare, il **bytecode Java** (Paragrafo 2.15). Questo insieme di istruzioni è stato progettato per essere simile al linguaggio Java, in modo da rendere molto semplice la fase di compilazione; in pratica non vengono effettuate ottimizzazioni sul codice. Come per i compilatori C, anche il compilatore Java controlla i tipi di dato e genera le operazioni necessarie per ogni tipo. I programmi Java vengono distribuiti in forma binaria, come bytecode.

Bytecode Java: istruzione appartenente all'insieme di istruzioni progettato per interpretare i programmi Java.

Un interprete software, chiamato **Java Virtual Machine** (**JVM**), può eseguire i bytecode Java. Un interprete è un programma che simula una certa architettura di un insieme di istruzioni; per esempio, il simulatore RISC-V a cui si fa riferimento in questo libro è un interprete. Non c'è bisogno di una fase separata per la traduzione del codice in linguaggio macchina (assemblaggio),

Java Virtual Machine (JVM): programma che interpreta i bytecode Java.

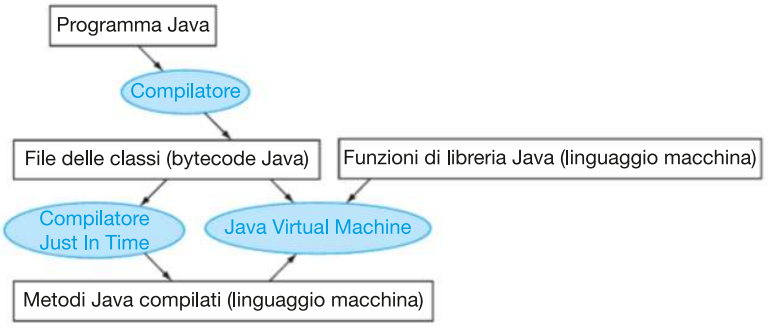


Figura 2.22 Gerarchia della traduzione del codice Java. Il primo passo per un programma Java è la compilazione in una versione binaria, detta bytecode Java, nella quale tutti gli indirizzi vengono definiti dal compilatore. A questo punto il programma Java è pronto per essere eseguito da un interprete, chiamato *Java Virtual Machine* (JVM) che collega i metodi desiderati della libreria Java durante l'esecuzione stessa del programma. Per ottenere prestazioni migliori, la Java Virtual Machine può invocare il compilatore Just In Time (JIT), che selettivamente compila i metodi del linguaggio Java nel linguaggio macchina del calcolatore sul quale è in esecuzione.

dato che questa viene effettuata direttamente dal compilatore o dalla stessa JVM durante l'esecuzione del codice.

Il principale vantaggio nell'uso di un interprete è la portabilità. La disponibilità della Java Virtual Machine per diverse piattaforme ha comportato che gran parte dei programmatori abbia potuto scrivere ed eseguire i programmi Java subito dopo la creazione del linguaggio. Oggi la Java Virtual Machine si può trovare su centinaia di milioni di dispositivi, dai cellulari ai browser Internet.

Lo svantaggio nell'uso di un interprete è dato dalle basse prestazioni. L'incredibile miglioramento dell'hardware in termini di prestazioni tra gli anni '80 e gli anni '90 è riuscito a rendere sensato l'utilizzo di un interprete per molte applicazioni importanti; tuttavia, quando si confrontano i programmi C compilati tradizionalmente con quelli Java interpretati, si trova spesso un fattore 10 di differenza nelle prestazioni, il che rende i programmi interpretati poco interessanti per alcune tipologie di applicazioni.

Per mantenere la portabilità e migliorare la velocità di esecuzione, la successiva fase dello sviluppo di Java ha visto la nascita di compilatori che traducono il codice interpretato in codice macchina del calcolatore *durante* l'esecuzione del programma stesso. Questi programmi, chiamati **compilatori Just In Time** (**JIT**), tracciano un profilo dell'applicazione durante la sua esecuzione in modo da sapere quali siano i metodi "caldi" (quelli che durante l'esecuzione consumano più tempo CPU), e quindi li compilano, traducendoli in istruzioni nel linguaggio macchina proprio del calcolatore. La porzione compilata viene salvata su disco per la successiva esecuzione del programma, ottenendo così che lo stesso programma possa essere eseguito più velocemente in tutte le esecuzioni successive. Il rapporto tra codice compilato e interpretato evolve quindi nel tempo, e i programmi Java eseguiti molto di frequente soffrono solo di un piccolo costo addizionale di interpretazione.

Compilatore Just In Time (JIT): il nome comunemente dato a un compilatore che opera nel corso dell'esecuzione di un programma, traducendo il codice interpretato nel codice macchina del calcolatore su cui il programma è in esecuzione.

Dato che l'incremento di prestazioni dei calcolatori permette di creare compilatori sempre più potenti e che i ricercatori sviluppano tecniche sempre più efficienti per compilare il codice Java durante l'esecuzione, la differenza di prestazioni tra Java e C/C++ sta assottigliandosi sempre più. Il Paragrafo 2.15 approfondisce maggiormente la descrizione di Java, del bytecode Java, della JVM e dei compilatori JIT.

Autovalutazione

Quale dei seguenti vantaggi di un interprete rispetto a un compilatore è più importante per un programmatore Java?

1. La facilità di scrivere un interprete.
2. I messaggi di errore più chiari.
3. Il codice oggetto più piccolo.
4. L'indipendenza dal calcolatore.

2.13 Un esempio riassuntivo in linguaggio C

Il rischio che si corre nel mostrare solo piccoli frammenti di codice assembler è di non dare un'idea di quale aspetto abbia un programma completo. Per ovviare a ciò, in questo paragrafo presentiamo il codice RISC-V corrispondente a due procedure scritte in linguaggio C: la prima inverte la posizione di due elementi di un vettore e la seconda li ordina.

```
void scambia(int v[], int k)
{
    int temp;
    temp = v[k];
    v[k] = v[k + 1];
    v[k+1] = temp;
}
```

Figura 2.23 Procedura C che scambia il contenuto di due locazioni di memoria. Questa procedura sarà utilizzata all'interno della procedura di ordinamento descritta in seguito.

Procedura `scambia`

Iniziamo esaminando il codice C della procedura `scambia`, descritta nella Figura 2.23, che scambia semplicemente il contenuto di due locazioni di memoria. Per tradurre da C ad assembler occorre seguire i seguenti passaggi:

1. Vengono allocati i registri per le variabili del programma;
2. Si scrive il codice relativo al corpo della procedura;
3. Viene salvato il contenuto dei registri dalle modifiche che possono avvenire nel corso della procedura.

Le tre parti della procedura `scambia` verranno di seguito descritte prima separatamente e poi ricomposte.

Allocazione dei registri per la procedura `scambia`

Come già illustrato, per quanto riguarda il passaggio dei parametri prevede l'utilizzo dei registri da `x10` a `x17`. Dal momento che la procedura `scambia` ha solo due parametri la convenzione RISC-V, `v` e `k`, questi si troveranno nei registri `x10` e `x11`. L'unica variabile della procedura è `temp`, che associamo al registro `x5`. Questo è possibile perché `scambia` è una procedura foglia. Lo schema di allocazione dei registri corrisponde alla dichiarazione delle variabili contenuta nella prima riga della procedura scambia della Figura 2.23.

Codice del corpo della procedura `scambia`

Le restanti righe di codice C della procedura `scambia` sono:

```
temp   = v[k];
v[k]   = v[k + 1];
v[k+1] = temp;
```

Ricordando che gli indirizzi di memoria delle istruzioni RISC-V fanno riferimento al byte e che quindi due parole consecutive differiscono tra loro di 4 byte, occorre moltiplicare l'indice `k` per 4 prima di sommarlo all'indirizzo base del vettore. *Dimenticare che gli indirizzi di due parole consecutive differiscono di 4 byte, invece che di 1, è un errore molto comune nella programmazione assembler.* Perciò, il primo passo di esecuzione consiste nell'ottenere l'indirizzo di `v[k]` moltiplicando `k` per 4 attraverso un'operazione di shift a sinistra di 2 posizioni:

```
slli  x6, x11, 2    // registro x6 = k * 4
add   x6, x10, x6   // registro x6 = v + (k * 4)
```

Ora si può caricare `v[k]` utilizzando `x6` e, successivamente, `v[k+1]` aggiungendo 4 a `x6`:

```
lw x5, 0(x6)   // registro x5 (temp) = v[k]
lw x7, 4(x6)   // registro x7 = v[k + 1]
               // si riferisce all'elemento di v successivo
```

Il passo successivo consiste nel memorizzare x9 e x11 scambiando la loro posizione in memoria:

```
sw x7, 0(x6)   // v[k] = registro x7
sw x5, 4(x6)   // v[k + 1] = registro x5 (temp)
```

A questo punto abbiamo allocato i registri e scritto il corpo di scambia; non resta che aggiungere il codice per salvare il contenuto dei registri eventualmente utilizzati dalla procedura scambia. Dato che non vengono utilizzati registri che devono essere preservati ed essendo scambia una procedura foglia, non dobbiamo salvare il contenuto di alcun registro.

Procedura scambia completa

Siamo ora pronti per la procedura scambia completa, che comprende l'etichetta della procedura e l'istruzione di ritorno.

```
scambia:
   slli   x6, x11, 2   // reg x6 = k * 4
   add    x6, x10, x6  // reg x6 = v + (k * 4)
   lw     x5, 0(x6)    // reg x5 (temp) = v[k]
   lw     x7, 4(x6)    // reg x7 = v[k + 1]
   sw     x7, 0(x6)    // v[k] = reg x7
   sw     x5, 4(x6)    // v[k+1] = reg x5 (temp)
   jalr   x0, 0(x1)    // ritorno alla procedura chiamante
```

Procedura ordina

Per capire meglio il rigore richiesto dalla programmazione in linguaggio assembler, descriveremo un secondo esempio, un po' più lungo del primo. Scriveremo una procedura che richiama la procedura scambia, il cui scopo è quello di ordinare gli elementi di un vettore di numeri interi. Utilizzeremo l'algoritmo *Bubble Sort*, uno degli algoritmi di ordinamento più semplici e veloci. La Figura 2.24 mostra la versione C del programma. Anche in questo caso la procedura verrà costruita attraverso passi successivi fino ad arrivare al codice assembler completo.

Allocazione dei registri per la procedura ordina

I due parametri della procedura ordina, v e n, saranno contenuti nei registri dei parametri x10 e x11; assegniamo i registri x19 e x20 rispettivamente alle variabili i e j.

Codice del corpo della procedura ordina

Il corpo della procedura consiste in due cicli *for* annidati e nella chiamata alla procedura scambia, a cui vengono passati i parametri opportuni. Svilupperemo

```
void scambia (int v[], size_t int n)
{
   size_t i, j;
   for (i = 0; i < n; i += 1) {
        for (j = i - 1; j >= 0 && v[j] > v[j + 1]; j -= 1) {
            swap(v,j);
        }
   }
}
```

Figura 2.24 Procedura C che opera un ordinamento degli elementi di un vettore v.

 © 978-88-08-19966-9

il codice procedendo dall'esterno all'interno. Il primo passo consiste nella traduzione del ciclo *for* più esterno:

```
for (i = 0; i < n; i += 1) {
```

Si ricordi che il ciclo *for* in C è composto da tre parti: inizializzazione, test di fine ciclo e incremento associato a ogni iterazione. È sufficiente una sola istruzione per inizializzare `i` a 0, contenuta nella prima parte dell'istruzione *for*:

```
addi x19, x0, 0
```

Si ricordi che `li` è una pseudoistruzione fornita dall'assemblatore per comodità di programmazione (Paragrafo 2.12.). Viene richiesta una seconda istruzione per incrementare il contatore `i`, traducendo così l'ultima parte del ciclo *for*:

```
addi x19, x19, 1 // i += 1
```

Il ciclo termina se la condizione `i < n` *non* è vera o, in altre parole, se `i ≥ n` il ciclo termina. Questo confronto richiede solamente una istruzione:

```
for1test: bge x19, x11, esci1  // vai a esci1 se
                               // x19 ≥ x1 (i ≥ n)
```

L'ultima istruzione del ciclo sarà un salto incondizionato all'inizio del ciclo stesso:

```
          j for1test  // Torna al test contenuto
                      // nel ciclo più esterno
esci1:
```

Lo scheletro del codice per il ciclo *for* più esterno sarà quindi:

```
    addi x19, x0, 0      // i = 0
for1test:
    bge x19, x11, esci1   // vai a esci1 se
                          // x19 ≥ x1 (i ≥ n)
        ...
    (corpo del ciclo for più esterno)
        ...
        addi x19, x19, 1  // i += 1
        j for1test        // torna al test del
                          // ciclo più esterno
esci1:
```

Ecco fatto. Negli esercizi potrete sperimentare come scrivere una procedura che esegua più velocemente cicli di questo tipo.

Il ciclo *for* più interno ha la seguente forma in linguaggio C:

```
for (j = i - 1; j >= 0 && v[j] > v[j + 1]; j -= 1) {
```

La parte di inizializzazione è anche in questo caso composta da una istruzione:

```
addi x20, x19, -1    // j = i - 1
```

Anche il decremento di `j` alla fine del ciclo può essere tradotto in una singola istruzione:

```
addi x20, x20, -1    // j -= 1
```

Il test di fine ciclo è composto da due parti. Si esce dal ciclo se almeno una delle condizioni non è verificata; il primo test deve quindi provocare l'uscita dal ciclo se $j < 0$:

```
for2test:
     blt x20, x0, esci2  // vai a esci2 se
                         // x20 < 0 (j < 0)
```

L'ultima istruzione fa sì che si salti il secondo test se non risulta `j ≥ 0`.

Il secondo test impone che il ciclo termini se `v[j] > v[j + 1]` *non* è vera o, in altre parole, se `v[j] ≤ v[j + 1]`. Per prima cosa occorre ottenere l'indirizzo dell'elemento del vettore, moltiplicando `j` per 4 (dato che gli indirizzi sono sempre espressi in byte) e sommarlo all'indirizzo base di `v`:

```
slli x5, x20, 2   // registro x5 = j * 4
add  x5, x10, x5  // registro x5 = v + (j * 4)
```

Ora è possibile caricare `v[j]`:

```
lw x6, 0(x5)      // registro x6 = v[j]
```

Dal momento che l'elemento successivo del vettore è semplicemente la parola successiva in memoria, basta sommare 4 all'indirizzo contenuto in `x5` per ottenere l'indirizzo di `v[j + 1]`:

```
lw x7, 4(x5)     // registro x7 = v[j + 1]
```

Verifichiamo se `v[j] ≤ v[j + 1]` per uscire dal ciclo:

```
ble x6, x7, esci2   // vai a esci2 se x6 ≤ x7
```

L'ultima istruzione del ciclo sarà un salto incondizionato all'inizio del ciclo interno stesso:

```
jal x0, for2test    // torna al test di ingresso del ciclo più interno
```

Unendo i vari frammenti di codice, lo scheletro del secondo ciclo *for* diventa:

```
          addi x20, x19, -1  // j = i - 1
for2test: blt x20, x0, esci2 // vai a esci2 se
                             // x20 < 0 (j < 0)
          slli x5, x20, 2    // registro x5 = j * 4
          add  x5, x10, x5   // registro x5 = v + (j * 4)
          lw   x6, 0(x5)     // registro x6 = v[j]
          lw   x7, 4(x5)     // registro x7 = v[j + 1]
          ble  x6, x7, esci2 // vai a esci2 se x6 ≤ x7
            ...
            (corpo del ciclo for più interno)
            ...
          addi x20, x20, -1  // j -= 1
          jal x0, for2test   // salta al test del ciclo
                             // più interno
esci2:
```

Chiamata alla procedura contenuta in `ordina`

Il passo successivo consiste nel tradurre il corpo del ciclo *for* più interno:

```
scambia(v,j);
```

La chiamata alla procedura `scambia` è molto semplice:

```
jal x1, scambia
```

Passaggio dei parametri in `ordina`

Il problema nasce dal fatto che la procedura `ordina` ha bisogno di conservare i propri parametri, contenuti nei registri `x10` e `x11`, mentre la

procedura `scambia` ha bisogno di inserire i propri parametri in questi stessi due registri.

Una possibile soluzione consiste nel copiare i parametri della procedura `ordina` in altri registri nella fase iniziale della procedura stessa, in modo da rendere disponibili per la procedura `scambia` i registri `x10` e `x11` (la copia in un registro è più veloce del salvataggio e della lettura dei dati dallo stack). Quindi all'inizio della procedura si copia il contenuto di `x10` e `x11` rispettivamente in `x21` e `x22`:

```
addi x21, x10, 0    // copia del parametro x10 in x21
addi x22, x11, 0    // copia del parametro x11 in x22
```

A questo punto si possono passare i parametri alla procedura `scambia` con le due istruzioni seguenti:

```
addi x10, x21, 0    // il primo parametro
                    // di scambia è v
addi x11, x20, 0    // il secondo parametro
                    // di scambia è j
```

Salvataggio dei registri in `ordina`

Rimane ancora da scrivere la parte di codice relativa al salvataggio e al ripristino dei registri. Chiaramente è necessario salvare l'indirizzo di ritorno, contenuto nel registro `x1`, dal momento che `ordina` è una procedura e quindi verrà chiamata a sua volta da un'altra procedura. Inoltre `ordina` utilizza i registri `x19`, `x20`, `x21` e `x22` che devono quindi essere salvati. La parte iniziale della procedura sarà quindi:

```
addi sp,  sp, -20   // crea spazio nello stack per 5 registri
sw   x1,  16(sp)    // salva x1 in stack
sw   x22, 12(sp)    // salva x22 in stack
sw   x21, 8(sp)     // salva x21 in stack
sw   x20, 4(sp)     // salva x20 in stack
sw   x19, 0(sp)     // salva x19 in stack
```

La parte finale della procedura semplicemente inverte tutte queste istruzioni e le conclude con una `jalr` all'indirizzo di ritorno.

Procedura `ordina` completa

Possiamo ora mettere assieme tutti i diversi frammenti della procedura e ottenere il codice mostrato nella **Figura 2.25**; occorre avere cura di sostituire i riferimenti ai registri `x10` e `x11` all'interno dei cicli *for* con i riferimenti ai registri `x21` e `x22`.

Per rendere il codice più leggibile, a ciascun blocco di codice viene associato il suo ruolo. In questo esempio le 9 linee di codice C della procedura `ordina` sono diventate 34 righe in codice assembler RISC-V.

Approfondimento. Un tipo di ottimizzazione che in questo caso può dare dei buoni risultati è l'*inserimento in linea* (*in-lining*) *della procedura*. Invece di passare gli argomenti sotto forma di parametri e di richiamare il codice della procedura `scambia` con un'istruzione `jal`, il compilatore copia il codice del corpo della procedura `scambia` nel punto in cui compare la sua chiamata. In questo esempio l'inserimento in linea del codice eliminerebbe quattro istruzioni. Lo svantaggio di questa ottimizzazione è che il codice compilato sarà di dimensioni maggiori quando la procedura inserita in linea viene chiamata da più posizioni. Questa espansione del codice potrebbe *peggiorare le prestazioni* se aumentasse la frequenza di miss della cache (Capitolo 5).

Salvataggio dei registri			
	ordina:	addi sp, sp, -20	// crea spazio per 5 registri in stack
		sw x1, 16(sp)	// salva l'indirizzo di ritorno in stack
		sw x22, 12(sp)	// salva x22 in stack
		sw x21, 8(sp)	// salva x21 in stack
		sw x20, 4(sp)	// salva x20 in stack
		sw x19, 0(sp)	// salva x19 in stack
Corpo della procedura			
Spostamento dei parametri		addi x21, x10, 0	// copia il parametro x10 in x21
		addi x22, x11, 0	// copia il parametro x11 in x22
Ciclo esterno		addi x19, x0, 0	// i = 0
	for1test:	bge x19, x22, esci1	// vai a esci1 se i >= n
Ciclo interno		addi x20, x19, -1	// j = i - 1
	for2test:	blt x20, x0, esci2	// vai a esci2 se j < 0
		slli x5, x20, 2	// x5 = j * 4
		add x5, x21, x5	// x5 = v + (j * 4)
		lw x6, 0(x5)	// x6 = v[j]
		lw x7, 8(x5)	// x7 = v[j + 1]
		ble x6, x7, esci2	// vai a esci2 se x6 < x7
Passaggio dei parametri e chiamata		addi x10, x21, 0	// il primo parametro di ordina è v
		addi x11, x20, 0	// il secondo parametro di ordina è j
		jal x1, scambia	// chiama scambia
Ciclo interno		addi x20, x20, -1	// j-=1
		j for2test	// vai a for2test
Ciclo esterno	esci2:	addi x19, x19, 1	// i += 1
		j for1test	// vai a for1test
Ripristino dei registri			
	esci1:	lw x19, 0(sp)	// ripristina x19 dallo stack
		lw x20, 4(sp)	// ripristina x20 dallo stack
		lw x21, 8(sp)	// ripristina x21 dallo stack
		lw x22, 12(sp)	// ripristina x22 dallo stack
		lw x1, 16(sp)	// ripristina l'indirizzo di ritorno dallo stack
		addi sp, sp, 20	// ripristina lo stack pointer
Ritorno alla procedura			
		jalr x0, 0(x1)	// ritorno alla procedura chiamante

Figura 2.25 Versione in linguaggio assembler RISC-V della procedura `ordina` della Figura 2.24.

Capire le prestazioni dei programmi

La Figura 2.26 mostra l'impatto delle ottimizzazioni effettuate dal compilatore sulle prestazioni di un programma di ordinamento in termini di tempo di compilazione, cicli di clock, numero di istruzioni e CPI. Si può notare come il codice non ottimizzato abbia il CPI migliore, l'ottimizzazione di tipo O1 produca il minor numero di istruzioni, ma l'ottimizzazione O3 sia la più veloce. Si ricordi che, in ultima analisi, il tempo è l'unica misura accurata delle prestazioni di un programma.

La Figura 2.27 confronta l'impatto dei linguaggi di programmazione, dell'utilizzo di codice compilato o interpretato, e dell'algoritmo sulle prestazioni del programma di ordinamento. Le quattro colonne mostrano che il programma non ottimizzato C è 8,3 volte più veloce di quello scritto in codice Java interpretato per quanto riguarda l'algoritmo Bubble Sort. Utilizzando il compilatore JIT Java, il codice Java diventa 2,1 volte più veloce del codice C non ottimizzato e risulta più lento solamente di un fattore 1,13 rispetto al codice C con il massimo livello di ottimizzazione. (Il Paragrafo 2.15 fornisce maggiori dettagli sul confronto tra

(continua)

(continua) compilazione e interpretazione di Java, e riporta il codice Java della procedura di ordinamento tramite Bubble Sort.) Le prestazioni non sono altrettanto vicine quando si considera l'algoritmo di ordinamento Quicksort (colonna 5), probabilmente perché non si riesce ad ammortizzare il costo della compilazione JIT con un tempo di esecuzione così breve. L'ultima colonna mostra l'impatto sulle prestazioni di un algoritmo migliore: si può ottenere un aumento della velocità di esecuzione addirittura di tre ordini di grandezza per l'ordinamento di 100 000 elementi. Anche confrontando le prestazioni del codice Java interpretato per Quicksort, riportate nella colonna 5, con quelle del codice C compilato per Bubble Sort riportate nella colonna 4, l'algoritmo Quicksort batte Bubble Sort di un fattore 50 (0,05 × 2468, ovvero il codice è 123 volte più veloce del codice C non ottimizzato, contro un aumento di velocità di 2,41 volte).

Ottimizzazione gcc	Prestazioni relative	Cicli di clock (milioni)	Numero di istruzioni (milioni)	CPI
Nessuna	1,00	158 615	114 938	1,38
O1 (media)	2,37	66 990	37 470	1,79
O2 (completa)	2,38	66 521	39 993	1,66
O3 (integrazione delle procedure)	2,41	65 747	44 993	1,46

Figura 2.26 Confronto delle prestazioni in termini di numero di istruzioni e di CPI, utilizzando diverse ottimizzazioni del compilatore per l'implementazione dell'algoritmo Bubble Sort. Il programma ordina 100 000 numeri, codificati su una parola, contenuti in un vettore inizializzato con valori casuali. I programmi sono stati eseguiti su un Pentium 4 con una frequenza di clock di 3,06 GHz, con bus di sistema a 533 MHz e con 2 GB di memoria SDRAM (PC2100 DDR). È stato utilizzato il kernel di Linux 2.4.20.

Linguaggio	Metodo esecuzione	Ottimizzazione	Prestazioni relative per Bubble Sort	Prestazioni relative per Quicksort	Speedup di Quicksort rispetto a Bubble Sort
C	Compilato	Nessuna	1,00	1,00	2468
	Compilato	O1	2,37	1,50	1562
	Compilato	O2	2,38	1,50	1555
	Compilato	O3	2,41	1,91	1955
Java	Interpretato	–	0,12	0,05	1050
	Compilato Just In Time	–	2,13	0,29	338

Figura 2.27 Confronto delle prestazioni di due programmi di ordinamento ottenute eseguendo un codice C ottenuto da un compilatore con diversi livelli di ottimizzazione e interpretando il corrispondente codice Java. Le prestazioni sono riferite a quelle ottenute con codice C non ottimizzato. Nell'ultima colonna è riportato l'incremento di prestazioni di Quicksort rispetto a Bubble Sort per ciascuna delle versioni di codice. I programmi sono stati eseguiti sullo stesso sistema con cui sono stati ottenuti i dati riportati nella Figura 2.26. La JVM è la versione 1.3.1 della Sun e il compilatore JIT è l'Hotspot, versione 1.3.1, della Sun.

2.14 Confronto tra vettori e puntatori

La comprensione dei puntatori è una delle maggiori sfide per chi inizia a programmare in C. Il confronto tra un programma assembler che utilizza vettori e indici e un programma che utilizza i puntatori aiuta a capire meglio il funzionamento di questi ultimi. Questo paragrafo mostra la versione C e assembler di due procedure che azzerano il contenuto di una sequenza di parole in

```
azzera1(int vettore[], size_t dim)
{
    size_t i;
    for (i = 0; i < dim; i += 1)
        vett[i] = 0;
}

azzera2(int *vett, size_t dim)
{
    int *p;
    for (p = &vett[0]; p < &vett[dim]; p = p + 1)
        *p = 0;
}
```

Figura 2.28 Due procedure C che azzerano tutti gli elementi di un vettore. `Azzera1` utilizza un indice, mentre `azzera2` utilizza un puntatore. La seconda procedura richiede qualche spiegazione per chi non ha familiarità con il linguaggio C. L'indirizzo di una variabile viene indicato con &, mentre l'oggetto puntato da un puntatore viene indicato con *. Dalla dichiarazione delle variabili si può vedere che sia `vett` sia `p` sono puntatori a interi. La prima parte dell'istruzione *for* della procedura `azzera2` assegna l'indirizzo del primo elemento di `vett` al puntatore `p`, mentre la seconda parte effettua un controllo per vedere se il puntatore ha superato l'ultimo elemento di `vett`. L'incremento di un'unità del puntatore nell'ultima parte dell'istruzione *for* equivale a spostare il puntatore all'elemento successivo, incrementando l'indirizzo di un numero di byte pari a quello dichiarato: dato che `p` è un puntatore a interi, il compilatore genera delle istruzioni RISC-V che incrementano `p` di 4, che corrisponde al numero di byte che costituiscono un intero nei RISC-V. L'istruzione di assegnamento scrive uno 0 nell'oggetto puntato da `p`.

memoria: la prima utilizza l'indice di un vettore, la seconda i puntatori. La versione C delle due procedure è mostrata nella **Figura 2.28**.

Lo scopo del paragrafo è mostrare come i puntatori si possano tradurre in istruzioni RISC-V e non di raccomandare uno stile di programmazione oramai datato. Al termine vedremo l'impatto delle ottimizzazioni dei moderni compilatori su queste due procedure.

Versione della procedura `azzera` che utilizza vettore e indice

Iniziamo dalla prima versione della procedura, `azzera1`, che utilizza un vettore, concentrandoci sul corpo del ciclo e ignorando il codice di collegamento. Supponiamo che i due parametri `vett` e `dim` si trovino nei registri `x10` e `x11` e che `i` sia allocato nel registro `x5`.

L'inizializzazione di `i`, cioè la prima parte dell'istruzione *for*, è elementare:

```
addi    x5, x0, 0   // i = 0 (registro x5 = 0)
```

Per impostare `vett[i]` a 0 occorre per prima cosa determinare il suo indirizzo; per fare ciò si moltiplica `i` per 4 e si ottiene l'indirizzo in byte:

```
ciclo1: slli x6, x5, 2   // x6 = i * 4
```

Dal momento che l'indirizzo di partenza del vettore è in un registro, lo si deve sommare al valore dell'indice per ottenere `vett[i]`:

```
add x7, x10, x6  // x7 = indirizzo di vett[i]
```

Infine, possiamo scrivere 0 a questo indirizzo:

```
sw x0, 0(x7)    // vett[i] = 0
```

Questa istruzione si trova alla fine del corpo del ciclo *for*, quindi il passo successivo consiste nell'incrementare `i`:

```
addi x5, x5, 1    // i = i + 1
```

Il test di fine ciclo verifica se `i` è minore della dimensione del vettore:

```
blt x5, x11, ciclo1// se (i < dim) vai a ciclo1
```

Abbiamo così visto tutte le parti della procedura. Il codice RISC-V completo che azzera tutti gli elementi del vettore (utilizzando un indice) sarà quindi il seguente:

```
        addi x5, x0, 0       // i = 0
ciclo1: slli x6, x5, 2       // x6 = i * 4
        add  x7, x10, x6     // x7 = indirizzo di vett[i]
        sw   x0, 0(x7)       // vett[i] = 0
        addi x5, x5, 1       // i = i + 1
        blt  x5, x11, ciclo1 // se (i < dim) vai a ciclo1
```

Si noti che questo frammento di codice funziona solo se `dim` è maggiore di 0; il C ANSI richiede che venga effettuato un test su `dim` prima del ciclo, ma in questo esempio non considereremo questo vincolo.

Versione della procedura `azzera` che utilizza i puntatori

La seconda procedura, che utilizza i puntatori, alloca i due parametri `vett` e `dim` nei registri `x10` e `x11` e il puntatore `p` in `x5`. Il codice di questa seconda procedura inizia con l'assegnazione dell'indirizzo del primo elemento del vettore al puntatore `p`:

```
addi x5, x10, 0    // p = indirizzo di vett[0]
```

La parte successiva è relativa al codice del corpo del ciclo *for*, che semplicemente scrive uno 0 nella posizione puntata da `p`:

```
ciclo2: sw x0, 0(x5)     // Memoria[p] = 0
```

Questa istruzione costituisce il corpo del ciclo, mentre l'istruzione seguente aggiorna `p` per fare sì che punti alla parola successiva:

```
addi x5, x5, 4    // p = p + 4
```

In C incrementare un puntatore di 1 significa farlo puntare all'oggetto successivo della sequenza. Dato che `p` è un puntatore a interi, ciascuno dei quali occupa 4 byte, il compilatore incrementa `p` di 4 unità.

La fase seguente consiste nel test di fine ciclo. Per prima cosa occorre calcolare l'indirizzo dell'ultimo elemento del vettore, moltiplicando `dim` per 4 al fine di ottenere l'indirizzo in byte:

```
slli x6, x11, 2   // x6 = dim * 4
```

Il prodotto ottenuto viene quindi sommato all'indirizzo di partenza del vettore, per ottenere l'indirizzo della prima parola che *segue* il vettore:

```
add x7, x10, x6   // x7 = indirizzo di vett[dim]
```

Il test di fine ciclo controlla semplicemente che `p` sia minore dell'ultimo elemento di `vett`:

```
bltu x5, x7 ciclo2   // se (p<&vett[dim]) vai a ciclo2
```

Abbiamo così completato tutte le parti del programma. Possiamo ora vedere la versione completa della procedura che azzera un vettore, basata sui puntatori:

```
        addi x5, x10, x0    // p = indirizzo di vett[0]
ciclo2: sw   x0, 0(x5)      // memoria[p] = 0
        addi x5, x5, 4      // p = p + 4
        slli x6, x11, 2     // x6 = dim * 4
        add  x7, x10, x6    // x7 = indirizzo di vett[dim]
        bltu x5, x7 ciclo2  // se (p<&vett[dim]) vai a ciclo2
```

Come nel caso precedente, si supponga che `dim` sia maggiore di 0.

Si noti che questa versione del programma calcola l'indirizzo corrispondente alla fine del vettore durante ogni iterazione del ciclo, anche se esso non cambia.

Spostando questo calcolo fuori dal ciclo si ottiene una versione più veloce:

```
        addi x5, x10, 0    // p = indirizzo di vett[0]
        slli x6, x11, 2    // x6 = dim * 4
        add  x7, x10, x6   // x7 = indirizzo di vett[dim]
ciclo2: sw   x0, 0(x5)     // memoria[p] = 0
        addi x5, x5, 4     // p = p + 4
        bltu x5, x7 ciclo2 // se (p<&vett[dim]) vai a ciclo2
```

Confronto tra le due versioni di `azzera`

Il confronto delle due versioni di codice, riportate una a fianco dell'altra, consente di evidenziare le differenze fra indici e puntatori (i cambiamenti introdotti dall'uso dei puntatori sono discussi nel testo):

```
       addi x5, x0, 0       // i = 0
ciclo1: slli x6, x5, 2      // x6 = i * 4
       add  x7, x10, x6     // x7 = indirizzo di vett[i]
       sw   x0, 0(x7)       // vett[i] = 0
       addi x5, x5, 1       // i = i + 1
       blt  x5, x11, ciclo1 // if (i < size) go to ciclo1
```

```
       addi x5, x10, 0     // p = indirizzo di vett[0]
       slli x6, x11, 2     // x6 = dim * 4
       add  x7, x10, x6    // x7 = indirizzo di vett[dim]
ciclo2: sw   x0, 0(x5)     // memoria[p] = 0
       addi x5, x5, 4      // p = p + 4
       bltu x5, x7, ciclo2 // if (p < &vett[dim]) vai a ciclo2
```

La versione di sinistra deve necessariamente contenere le operazioni di moltiplicazione e somma all'interno del ciclo, dato che `i` viene incrementato e quindi l'indirizzo di ciascun elemento del vettore deve essere calcolato a partire dal nuovo valore dell'indice, mentre la versione di destra incrementa direttamente il puntatore alla memoria `p`.

La versione con i puntatori sposta le istruzioni che determinano la posizione in memoria del primo e dell'ultimo elemento del vettore fuori dal ciclo, riducendo così da cinque a tre il numero di istruzioni eseguite a ogni ciclo.

Questa ottimizzazione effettuata manualmente corrisponde a due ottimizzazioni effettuate automaticamente dai compilatori: riduzione del peso delle istruzioni (operazione di shift invece che di moltiplicazione) ed eliminazione di variabili per induzione (eliminazione del calcolo dell'indirizzo degli elementi del vettore all'interno del ciclo). Il Paragrafo 2.15 descrive queste e altre ottimizzazioni.

Approfondimento. Come abbiamo detto in precedenza, un compilatore C aggiungerebbe un test per assicurarsi che `dim` sia maggiore di 0; un modo per farlo sarebbe saltare all'istruzione dopo il ciclo mediante `blt x0, 11, dopoCiclo`.

Capire le prestazioni dei programmi

Tradizionalmente veniva raccomandato agli aspiranti programmatori di utilizzare i puntatori in C per ottenere un codice più efficiente di quello ottenibile utilizzando indici e vettori. I compilatori ottimizzanti moderni sono in grado di produrre, partendo da un codice scritto con i vettori, un codice altrettanto efficiente. Molti programmatori attualmente preferiscono che sia il compilatore a effettuare le ristrutturazioni pesanti del codice.

 © 978-88-08-19966-9

2.15 Approfondimento: compilazione del C e interpretazione di Java

Questo paragrafo contiene una breve introduzione sul funzionamento di un compilatore C e su come viene eseguito un programma Java. Dato che il compilatore ha un grande impatto sulle prestazioni, comprendere la tecnologia dei compilatori moderni è un elemento critico per valutarle. La descrizione di come si costruisce un compilatore è oggetto di corsi della durata di uno o due semestri interi, per cui in questa introduzione possiamo solamente descrivere i concetti fondamentali.

Linguaggio orientato agli oggetti: un linguaggio di programmazione che è orientato agli oggetti piuttosto che alle azioni, in altre parole che è orientato ai dati piuttosto che alla logica di funzionamento.

La seconda parte del paragrafo è rivolta ai lettori interessati a comprendere come un **linguaggio orientato agli oggetti**, quale Java, venga eseguito su un'architettura RISC-V. Vengono mostrati i bytecode Java utilizzati per l'interpretazione del codice e il RISC-V che si ottiene dalla versione Java di alcuni frammenti di procedura riportati nei paragrafi precedenti, tra i quali quello relativo all'algoritmo di Bubble Sort. Viene descritto, inoltre, il funzionamento sia della Java Virtual Machine sia dei compilatori Just In Time. Troverete il testo di questo paragrafo online .

2.16 Un caso reale: le istruzioni dell'architettura MIPS

L'insieme di istruzioni più vicino al RISC-V è quello del MIPS. Anch'esso è nato in ambito accademico, ma è ora proprietà della Wave Computing. I MIPS e i RISC-V condividono la stessa filosofia di progetto, anche se il MIPS ha 25 anni più del RISC-V. La buona notizia è che se conoscete il RISC-V, è veramente facile conoscere anche il MIPS. Per illustrare il loro grado di somiglianza, la **Figura 2.29** compara i formati delle istruzioni dei RISC-V e dei MIPS e confrontati con i formati di un'altra architettura molto utilizzata, l'architettura ARMv7, che condivide la stessa filosofia.

L'ISA dei MIPS ha una versione a 32 bit e una a 64 bit, saggiamente denominate MIPS-32 e MIPS-64. Questi insiemi di istruzioni sono virtualmente identici, a eccezione della dimensione maggiore dello spazio di indirizzamento che richiede registri a 64 bit invece che a 32 bit. Riportiamo ora le caratteristiche comuni a RISC-V e MIPS.

- Tutte le istruzioni sono ampie 32 bit in entrambe le architetture.
- Entrambe le architetture hanno 32 registri a uso generale, con un registro impostato fisso a 0.
- L'unico modo per accedere alla memoria è attraverso le istruzioni di load e store in entrambe le architetture.
- A differenza di alcune architetture, non ci sono istruzioni nei MIPS e nei RISC-V che possano trasferire alla o dalla memoria più di un registro.
- Entrambe le architetture hanno istruzioni che possono saltare se il contenuto di un registro è uguale a zero o è diverso da zero.
- Le modalità di indirizzamento in entrambe le architetture possono trasferire dati di tutte le dimensioni.

Una delle differenze principali tra i RISC-V e i MIPS è sui salti condizionati basati su condizioni diverse dall'uguaglianza o disuguaglianza. Mentre il RISC-V fornisce semplicemente istruzioni di salto condizionato che confrontano il contenuto di due registri, i MIPS si devono basare su un'istruzione di comparazione che imposta il contenuto di un registro a 0 o a 1 a seconda che il risultato del confronto sia vero o falso. I programmatori devono fare seguire questa istruzione di comparazione con un'istruzione di saldo condizionato

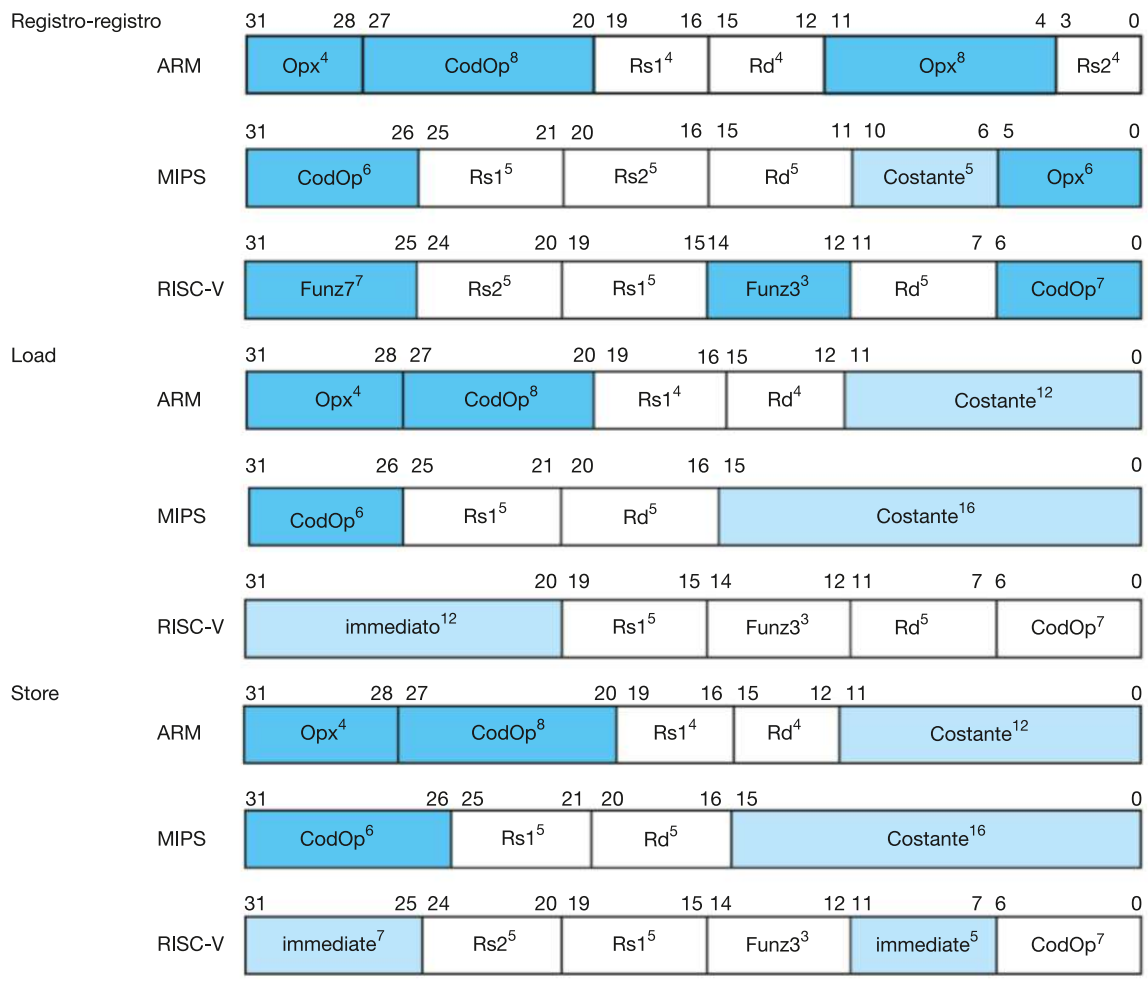


Figura 2.29 Formati delle istruzioni ARM, RISC-V e MIPS. Le differenze sono dovute al fatto che l'ARM è un'architettura a 16 registri, mentre il MIPS e il RISC-V ne hanno 32.

basato su uguaglianza o disuguaglianza, a seconda del risultato desiderato della comparazione. Obbedendo alla loro filosofia minimalista, i MIPS eseguono soltanto il confronto di tipo "minore di", lasciando al programmatore l'onere di scambiare l'ordine degli operandi o la condizione che deve verificare il salto condizionato per ottenere l'uscita desiderata. I MIPS hanno sia la versione con segno che senza segno dell'istruzione di imposta se minore: `slt` e `sltu`.

Guardando oltre il nucleo delle istruzioni che vengono utilizzate più comunemente, l'altra differenza principale è che il MIPS completo ha un insieme di istruzioni molto più ampio del RISC-V, come vedremo nel Paragrafo 2.20.

2.17 Un caso reale: le istruzioni dell'architettura ARMv7 (32 bit)

L'ARM è l'architettura di insiemi di istruzioni più diffusa tra i sistemi embedded, dato che la utilizzavano già più di 100 miliardi di dispositivi a fine 2016. ARM originariamente era l'acronimo di Acorn RISC Machine (Macchina RISC Acorn), successivamente modificato in Advanced RISC Machine

(Macchina RISC evoluta). Esso fu lanciato sul mercato lo stesso anno in cui furono introdotti i MIPS ed era stato progettato seguendo la stessa filosofia (le somiglianze tra le due architetture sono riportate nella Figura 2.30). La differenza principale è che il RISC V ha più registri mentre l'ARM ha più modalità di indirizzamento.

Esiste un nucleo di istruzioni per le operazioni aritmetico-logiche e per il trasferimento dati molto simile per le architetture RISC-V e ARM, come mostrato nella Figura 2.31.

	ARM	RISC-V
Data	1985	1985
Dimensione delle istruzioni (bit)	32	32
Spazio di indirizzamento (dimensione, modello)	32 bit, flat	32 bit, flat
Allineamento dei dati	Allineati	Allineato
Modalità di indirizzamento	9	3
Registri interi (numeri, modello, dimensione)	16 registri generici a 32 bit	32 registri generici a 32 bit
I/O	Mappato in memoria	Mappato in memoria

Figura 2.30 Somiglianze tra l'insieme delle istruzioni ARM e RISC-V.

	Nome dell'istruzione	ARM	RISC-V
Operazioni registro-registro	Addizione	add	add, addw
	Addizione (trap dell'overflow)	adds, swivs	add, addw
	Sottrazione	sub	sub, subw
	Sottrazione (trap dell'overflow)	subs, swivs	sub
	Moltiplicazione	mul	mul, mulw
	Divisione	–	div, divw
	And	and	and
	Or	orr	or
	Xor	eor	xor
	Caricamento della parte alta di un registro	–	lui
	Scorrimento a sinistra, logico	lsl[1]	sll, sllw
	Scorrimento a destra, logico	lsr[1]	srl, srlw
	Scorrimento a destra, aritmetico	asr[1]	sra, sraw
	Comparazione	cmp, cmn, tst, teq	slt / slti / sltu
Lettura / scrittura in memoria	Lettura di un byte con segno	ldsrb	lb
	Lettura di un byte senza segno	ldrb	lbu
	Lettura di una mezza parola dotata di segno	ldrsh	lh
	Lettura di una mezza parola senza segno	ldrh	lhu
	Lettura di una parola	ldr	lw
	Scrittura in memoria di un byte	strb	sb
	Scrittura in memoria di una mezza parola	strh	sh
	Scrittura in memoria di una parola	str	sw
	Lettura, scrittura di registri speciali	mrs, msr	–
	Scambio atomico	swp, swpb	lr.w, sc.w

Figura 2.31 Istruzioni ARM che trasferiscono dati tra i registri e con la memoria e le istruzioni equivalenti RISC-V. Il trattino indica che l'istruzione non è disponibile nell'architettura corrispondente o che deve essere tradotta con un numero elevato di istruzioni elementari. Le istruzioni equivalenti sono separate dalla virgola. Nell'architettura ARM le operazioni di scorrimento sono parte integrante di tutte le operazioni sui dati, perciò le istruzioni di scorrimento contrassegnate con 1 in apice (per esempio lsr[1]) non sono altro che varianti dell'istruzione `move`. Si noti che l'architettura ARM non contiene operazioni di divisione.

Modalità di indirizzamento	ARM	RISC-V
Operandi nei registri	X	X
Operandi immediati	X	X
Registro + offset (base + spiazzamento)	X	X
Registro + registro (indicizzazione)	X	–
Registro + registro scalato (scalato)	X	–
Registro + offset e registro di aggiornamento	X	–
Registro + registro generico e registro di aggiornamento	X	–
Incremento/decremento automatico	X	–
Dati relativi al PC	X	X

Figura 2.32 Riassunto delle modalità di indirizzamento. Gli ARM consentono di indirizzare i dati sia in modo indiretto, tramite registro, sia attraverso la somma "contenuto di un registro + offset"; la prima modalità non è ottenuta semplicemente dalla seconda modalità inserendo uno 0 nel campo offset. Negli ARM, per ottenere uno spazio di indirizzamento più ampio, l'offset viene fatto scorrere a destra di 1 o 2 bit a seconda che il dato letto sia codificato rispettivamente su una mezza parola o su una parola intera.

Modalità di indirizzamento

La Figura 2.32 mostra le diverse modalità di indirizzamento supportate dall'ARM. A differenza del RISC-V, l'ARM non riserva un registro al valore 0. Mentre il RISC-V ha solamente tre semplici modalità di indirizzamento dei dati (Figura 2.18), l'ARM ne ha nove, di cui alcune ottenute tramite calcoli assai complessi. Per esempio, uno dei modi di indirizzamento nell'ARM prevede che si possa spostare il contenuto di un registro di un numero di posizioni arbitrarie, per poi sommarlo a quello di un altro registro, per formare un indirizzo, e aggiornare infine il contenuto di un terzo registro con questo indirizzo.

Comparazioni e salti condizionati

Il RISC-V utilizza il contenuto di due registri per valutare il risultato del confronto associato ai salti condizionati; l'ARM, invece, utilizza la rappresentazione tradizionale, che prevede quattro possibili risultati di un confronto: negativo, zero, riporto e overflow, codificati nella parola di stato del programma.

Questi risultati possono essere impostati in seguito all'esecuzione di una qualsiasi istruzione aritmetica o logica, ma, a differenza delle architetture precedenti, l'impostazione dei relativi bit della parola di stato è facoltativa. La specifica esplicita di queste condizioni crea minori problemi nell'implementazione su pipeline.

Le istruzioni di salto condizionato dell'ARM controllano i bit associati ai risultati del confronto; in questo modo possono determinare il risultato di una qualsiasi operazione di confronto tra due numeri, sia con segno che senza segno.

L'istruzione di comparazione (CMP) sottrae un operando a un altro e il risultato della differenza viene utilizzato per impostare i bit di stato in base al risultato del confronto. Lo stesso avviene per la comparazione di minoranza (CMN, *Compare Negative*), che somma i due operandi e imposta i bit di stato conseguentemente. L'istruzione TST esegue un AND logico di due operandi e l'istruzione TEQ esegue l'OR esclusivo; entrambe in base al risultato dell'operazione, impostano i bit di stato tranne quello utilizzato per l'overflow.

Una delle caratteristiche peculiari dell'ARM è che ciascuna istruzione può essere eseguita in modo condizionato, cioè dipendente dal risultato del confronto: ciascuna istruzione inizia con un campo di 4 bit che consente di determinare se l'istruzione debba essere eseguita come un'operazione vuota (nop) oppure come un'istruzione effettiva, a seconda del valore assunto dai bit contenenti il risultato della comparazione. Perciò, i salti condizionati possono essere considerati più propriamente come delle istruzioni di salto incondizionato eseguite in modo condizionato. L'esecuzione condizionata evita all'istruzione di salto condizionato di saltare un'istruzione singola; inoltre, richiede meno istruzioni e meno tempo.

La Figura 2.29 mostra i formati delle istruzioni ARM e RISC-V. La differenza principale risiede nel campo di 4 bit dell'ARM, riservato all'esecuzione condizionata contenuto in ogni istruzione, e nella dimensione ridotta dei campi dei registri, dovuta al fatto che l'ARM ha meno registri.

Caratteristiche uniche dell'ARM

Nella Figura 2.33 sono riportate alcune istruzioni aritmetico-logiche non presenti nel MIPS. Dato che l'ARM non ha un registro dedicato a contenere il valore 0, esistono dei codici operativi separati per eseguire quelle operazioni che il RISC-V effettua con il registro $zero. Inoltre, l'ARM supporta le operazioni aritmetiche su più parole.

Il campo immediato su 12 bit dell'ARM ha un utilizzo originale. Gli 8 bit meno significativi sono estesi a 32 inserendo degli zeri, vengono quindi ruotati verso destra un numero di volte pari a quanto specificato nei primi 4 bit del campo, e il risultato è moltiplicato poi per 2. Un vantaggio di questo schema è che esso può rappresentare tutte le potenze di 2 presenti in una parola a 32 bit. Sarebbe interessante studiare se questo schema consente di memorizzare più costanti immediate del semplice campo a 12 bit, senza trasformazioni.

Lo shift degli operandi non è limitato solo alle costanti immediate. Il contenuto del secondo registro di tutte le operazioni aritmetiche e logiche può essere fatto scorrere a destra o a sinistra, prima di essere utilizzato. Le operazioni di shift consentite sono: shift logico a sinistra, shift logico e aritmetico a destra, rotazione a destra.

Nome	Definizione	ARM	RISC-V
Caricamento di una costante	Rd = imm	mov	`addi x10, x0, costante`
Not	Rd = ~(Rs1)	mvn	–
Spostamento (move)	Rd = Rs1	mov	`addi x10, x9, 0`
Rotazione a destra	Rd = Rs i >> i	ror	
	$Rd_{0\ldots i-1}$ = Rs		
And not	Rd = Rs1 & ~(Rs2)	bic	
Sottrazione inversa	Rd = Rs2 – Rs1	rsb, rsc	
Supporto della somma intera su più parole	Riporto, Rd = Rd + Rs1 + RiportoUscitaPrecedente	adcs	–
Supporto per la sottrazione intera su più parole	Riporto, Rd = Rd – Rs1 + RiportoUscitaPrecedente	sbcs	–

Figura 2.33 Le istruzioni aritmetico-logiche degli ARM che non sono presenti nei RISC-V.

L'ARM possiede anche delle istruzioni per salvare un gruppo di registri, chiamate block load e store (lettura e scrittura a blocchi). A seconda del valore assunto da una maschera di 16 bit contenuta nell'istruzione stessa, un insieme qualsiasi dei 16 registri ARM può essere letto o copiato in memoria con una sola istruzione. Queste istruzioni possono salvare e ripristinare il contenuto dei registri all'atto della chiamata di una procedura e al termine della procedura stessa. Esse possono anche essere utilizzate per copiare blocchi di memoria; attualmente si tratta dell'utilizzo più importante di queste istruzioni.

2.18 Un caso reale: le istruzioni dell'architettura ARMv8 (64 bit)

Tra i problemi potenziali di un'architettura di un insieme di istruzioni, la dimensione troppo piccola dello spazio di indirizzamento della memoria è praticamente impossibile da risolvere. Mentre l'x86 è stato esteso con successo prima a 32 e poi a 64 bit, molti dei suoi compagni sono stati lasciati indietro. Per esempio, il MOStek 6502 a 16 bit equipaggiava l'AppleII, il primo personal computer di successo. Nonostante il grande successo iniziale, il numero limitato di bit di indirizzamento lo ha condannato a essere dimenticato dalla storia.

I progettisti degli ARM, non appena hanno percepito l'insufficienza dei 32 bit di indirizzamento del loro calcolatore, hanno iniziato a progettare una versione del processore ARM a 64 bit, già a partire dal 2007; questa fu poi immessa sul mercato nel 2013. Ma questa non è una versione frutto di una semplice cosmesi per rendere tutti i registri ampi 64 bit, come è successo per l'x-86, ma una completa ristrutturazione. La buona notizia è che se conoscete il MIPS, non avrete difficoltà a comprendere l'ARMv8 che è il nome dato alla versione a 64 bit dell'ARM.

Per prima cosa, l'ARM ha abbandonato tutte quelle caratteristiche della versione 7, molto diverse dal MIPS:

- non c'è più il campo di esecuzione condizionata, presente in quasi tutte le istruzioni della versione 7;
- il campo immediato è una costante a 12 bit, e non più l'input a una funzione che restituiva la costante come era nella versione 7;
- l'ARM ha abbandonato le istruzioni di load e store multipli;
- il PC non è più uno dei registri, cosa che produceva dei salti condizionati inattesi se si scriveva nel PC.

In secondo luogo, l'ARM ha aggiunto alcune caratteristiche che sono già sfruttate dai MIPS:

- la versione 8 ha registri general-purpose a 32 bit, cosa che chi costruisce i compilatori apprezza sicuramente; come nel MIPS, uno dei registri contiene 0, anche se nelle istruzioni di load e store, il registro contiene lo stack pointer;
- le modalità di indirizzamento dell'ARMv8 funzionano per parole di tutte le dimensioni di parola, a differenza dell'ARMv7;
- comprende un'istruzione di divisione, che non era presente nell'ARMv7;
- contiene anche la coppia di istruzioni di salto condizionato equivalenti alle istruzioni di "branch-if-equal" e "branch-if-not-equal" del MIPS.

Dato che la filosofia delle istruzioni dell'ARMv8 è molto più simile alla filosofia del MIPS di quella dell'ARMv7, possiamo concludere che la somiglianza maggiore tra ARMv7 e ARMv8 è nel nome.

2.19 Un caso reale: le istruzioni dell'architettura x86

La bellezza è tutta negli occhi di chi guarda.
Margaret Wolfe Hungerford, *Molly Bawn*, 1877

I progettisti di insiemi di istruzioni talvolta forniscono istruzioni più potenti di quelle che si trovano nel RISC-V e nel MIPS. Lo scopo è di ridurre il numero di istruzioni eseguite da un programma, ma il rischio è che tale riduzione vada a scapito della semplicità aumentando il tempo di esecuzione dei programmi, perché le istruzioni diventano più complesse. Ciò può derivare da un periodo del ciclo di clock più lungo o dalla necessità di un maggior numero di cicli rispetto alle istruzioni più semplici.

La strada che porta alla creazione di istruzioni più complesse è dunque irta di pericoli. Alcuni trabocchetti legati alla complessità delle istruzioni sono descritti nel Paragrafo 2.21.

Evoluzione dell'Intel x86

RISC-V e MIPS nacquero dalla visione di singoli gruppi di ricercatori che lavorarono nello stesso periodo; i diversi blocchi di queste architetture si adattano bene l'uno all'altro e l'intera architettura può essere descritta in modo conciso. Questo non è il caso delle architetture Intel x86, che sono il risultato del lavoro indipendente di molti gruppi di progettisti che hanno continuato a sviluppare l'architettura nel corso di quasi 40 anni, raggiungendo progressivamente nuove caratteristiche all'insieme di istruzioni originario. Le pietre miliari dell'evoluzione degli x86 sono le seguenti.

Registro di utilizzo generale (GPR): un registro che può essere utilizzato praticamente da tutte le istruzioni, per contenere indirizzi o dati.

- **1978**: l'architettura Intel 8086 è introdotta come estensione, compatibile a livello di linguaggio assembler, dell'allora popolarissimo Intel 8080, un microprocessore a 8 bit. L'8086 è un'architettura a 16 bit con tutti i registri interni a 16 bit. A differenza del MIPS, i registri erano dedicati a usi specifici, quindi l'8086 non può essere considerata un'architettura con **registri di utilizzo generale** (**GPR**, *General-Purpose Register*).
- **1980**: viene annunciato il coprocessore matematico Intel 8087 che estende l'architettura 8086 con 60 nuove istruzioni in virgola mobile. Invece di utilizzare registri, si serve di uno stack per le proprie operazioni (*vedi* Paragrafi 2.24 e 3.7).
- **1982**: l'80286 estende l'architettura 8086 allargando lo spazio di indirizzamento a 24 bit mediante la creazione di un complesso sistema di traduzione e protezione degli indirizzi di memoria (Capitolo 5). Vengono anche aggiunte alcune istruzioni per completare l'insieme di istruzioni e per gestire il modello di protezione.
- **1985**: l'80386 estende a 32 bit l'architettura 80286. Oltre a fornire un'architettura a 32 bit, con registri da 32 bit e uno spazio di indirizzamento di 32 bit, l'80386 contiene nuove modalità di indirizzamento e nuove operazioni. Le istruzioni aggiunte rendono l'80386 praticamente un calcolatore a registri di utilizzo generale. L'80386 fornisce un supporto alla paginazione, oltre che alla segmentazione della memoria (Capitolo 5). Come l'80286, anche l'80386 possiede una modalità che permette di eseguire i programmi scritti per l'8086 senza apportare modifiche.
- **1989-1995**: i processori successivi, l'80486 del 1989, il Pentium del 1992 e il Pentium Pro del 1995, mirano a ottenere prestazioni sempre più elevate; vengono aggiunte solamente quattro nuove istruzioni all'insieme delle istruzioni visibile all'utente: tre per la gestione del *multiprocessing* (Capitolo 6) e una per il trasferimento dati condizionato.
- **1997**: dopo l'introduzione del Pentium e del Pentium Pro, l'Intel annuncia l'espansione di queste architetture con l'insieme di istruzioni MMX (*Multi Media Extensions*). Questo nuovo insieme di 57 istruzioni utilizza lo stack

dei dati in virgola mobile per accelerare le applicazioni legate alla multimedialità e alla comunicazione. Le istruzioni MMX tipicamente operano su più elementi alla volta, ciascuno di piccola dimensione, secondo la tradizione delle architetture SIMD (*Single Instruction Multiple Data*; Capitolo 6). Il Pentium II non aggiunge nuove istruzioni.

- **1999**: Intel aggiunge altre 70 istruzioni, chiamate SSE (*Streaming SIMD Extensions*), come parte del Pentium III. Le modifiche principali consistono nell'introdurre otto nuovi registri separati, nel raddoppiarne la dimensione a 128 bit e nell'introdurre come tipo di dato i numeri in virgola mobile in singola precisione. Ciò permette di effettuare fino a quattro operazioni in virgola mobile su 32 bit in parallelo. Per migliorare le prestazioni della memoria, l'insieme SSE comprende istruzioni per il precaricamento della cache, oltre a istruzioni di memorizzazione dei dati in streaming che evitano il passaggio dalla cache e scrivono direttamente in memoria (Capitolo 5).
- **2001**: Intel estende l'insieme di istruzioni con altre 144 istruzioni, chiamate questa volta SSE2. Vengono introdotti i numeri in virgola mobile in doppia precisione, il che permette di effettuare due operazioni in virgola mobile su 64 bit in parallelo. Quasi tutte le 144 istruzioni sono estensioni di istruzioni presenti negli insiemi MMX e SSE, che operano in parallelo su dati a 64 bit. Questo cambiamento non solo permette più operazioni multimediali, ma dà anche la possibilità al compilatore di scegliere un registro destinazione differente per le operazioni in virgola mobile rispetto all'architettura a singolo stack: infatti, il compilatore può scegliere di utilizzare gli otto registri SSE come registri in virgola mobile, come negli altri calcolatori. Queste modifiche hanno aumentato molto le prestazioni sulle operazioni in virgola mobile del Pentium 4, il primo microprocessore a includere le istruzioni SSE2.
- **2003**: questa volta è un'azienda diversa da Intel a migliorare l'architettura x86. AMD annuncia una serie di estensioni per aumentare lo spazio di indirizzamento da 32 a 64 bit. In maniera del tutto simile alla transizione dello spazio di indirizzamento da 16 a 32 bit avvenuta nel 1985 con l'80386, l'AMD64 allarga tutti i registri a 64 bit. Inoltre sale a 16 unità sia il numero di registri di tipo generale sia il numero dei registri a 128 bit di tipo SSE. Il principale cambiamento nell'ISA è l'aggiunta di una nuova modalità, chiamata *long mode* (modalità lunga), che ridefinisce l'esecuzione di tutte le istruzioni x86 con indirizzi e dati a 64 bit. Inoltre, per poter indirizzare un maggior numero di registri, viene aggiunto alle istruzioni un nuovo prefisso. Questa modalità lunga aggiunge da 4 a 10 nuove istruzioni, a seconda di come viene effettuato il conto, ed elimina 27 delle istruzioni introdotte in precedenza. Un'ulteriore estensione riguarda l'indirizzamento relativo al PC. L'AMD64 mantiene ancora una modalità di funzionamento identica a quella dell'x86 (*legacy mode*, modalità ereditaria), ma la affianca a una modalità che vincola i programmi utente a utilizzare le caratteristiche dell'x86, e permette anche al sistema operativo di utilizzare tutte le caratteristiche dell'AMD64 (*compatibility mode*, modalità compatibile). Queste modalità hanno permesso una transizione all'indirizzamento a 64 bit più morbida delle architetture HP/Intel IA-64.
- **2004**: Intel capitola e adotta la soluzione AMD64, rinominandola *tecnologia di estensione della memoria a 64 bit* (*EM64T, Extended Memory 64 Technology*). La differenza principale tra le due architetture è che Intel ha aggiunto un'operazione atomica di confronto e scambio a 128 bit che probabilmente avrebbe dovuto essere inclusa anche nell'AMD64. Allo stesso tempo Intel annuncia un'altra generazione di estensioni multimediali: SSE3, che aggiunge 13 istruzioni per supportare operazioni aritmetiche con numeri complessi, operazioni grafiche su vettori di strutture, codifica video,

conversione in virgola mobile e sincronizzazione di thread (Paragrafo 2.11). AMD ha aggiunto l'estensione SSE3 e l'operazione atomica di swap che mancava nell'AMD64 per poter mantenere la compatibilità binaria con Intel.

- **2006**: Intel annuncia 54 nuove istruzioni come parte dell'estensione dell'insieme di istruzioni SSE4. Questa estensione consente di eseguire funzioni particolari come la somma di differenze, prese in valore assoluto, i prodotti vettore su vettori di strutture, l'aggiunta di zeri o l'estensione del segno di dati codificati su un numero ridotto di bit, il conteggio degli elementi di una popolazione e così via. L'SSE4 fornisce anche il supporto alle macchine virtuali (Capitolo 5).
- **2007**: AMD annuncia 170 istruzioni come parte dell'SSE5, comprendenti 46 istruzioni con tre operandi che completano l'insieme di base, come nel RISC-V.
- **2011**: Intel rilascia l'estensione *Advanced Vector Extension*, che espande la larghezza dei registri da 128 a 256 bit, ridefinendo di conseguenza circa 250 istruzioni e aggiungendone 128 nuove.
- **2015**: Intel rilascia l'estensione AVX-512, che espande la larghezza dei registri e delle operazioni da 256 a 512 bit e, ancora una volta, ridefinisce centinaia di istruzioni e ne aggiunge ancora di più.

Come si vede da questa cronologia, la compatibilità ha influito come una "gabbia dorata" sull'architettura x86: il software già in circolazione era troppo importante perché si potesse metterlo a rischio con modifiche architetturali significative.

Qualunque siano i difetti architetturali dell'x86, si ricordi che lo sviluppo dei PC è largamente basato su questo insieme di istruzioni, che domina ancora nei sistemi cloud dell'era post-PC. La produzione annuale di 250 milioni di chip x86 può sembrare piccola rispetto ai miliardi di chip ARM, ma molte società sarebbero felici di avere il controllo di questo mercato, dato che i chip sono molto più costosi. Tuttavia questo mosaico di modelli diversi ha prodotto un'architettura difficile da spiegare e impossibile da apprezzare.

Un consiglio al lettore su quanto segue: non cerchi di leggere il prossimo paragrafo con l'attenzione necessaria a scrivere programmi assembler per l'x86: lo scopo qui è mostrare la forza e la debolezza dell'architettura per calcolatori desktop più diffusa al mondo.

Invece di mostrare l'intero insieme di istruzioni a 16, 32 e 64 bit, in questo paragrafo ci concentreremo sul sottoinsieme delle istruzioni a 32 bit nato con l'80386. Inizieremo la trattazione con i registri e le modalità di indirizzamento, per passare poi alle operazioni sui numeri interi e concludere con l'analisi della codifica delle istruzioni.

Registri e modalità di indirizzamento dell'x86

L'evoluzione dell'insieme di istruzioni si nota chiaramente osservando il formato dei registri dell'80386 (**Figura 2.34**): questo processore estende tutti i registri, precedentemente a 16 bit, a 32 bit (a eccezione dei registri di segmento). La nuova versione dei registri viene indicata anteponendo al loro nome il prefisso *E*. I registri, nel loro complesso, saranno qui considerati a utilizzo generale (GPR). L'80386 ha solo 8 registri GPR; questo significa che un programma RISC-V e MIPS ne può utilizzare il quadruplo.

Le istruzioni aritmetiche, logiche e di trasferimento dati hanno due operandi e le combinazioni consentite sono mostrate nella **Figura 2.35**. Ci sono due importanti differenze rispetto al RISC-V e al MIPS. La prima è che nell'x86 le istruzioni aritmetico-logiche hanno sempre un operando che funge sia da sorgente sia da destinazione, mentre il RISC-V e il MIPS consentono di definire

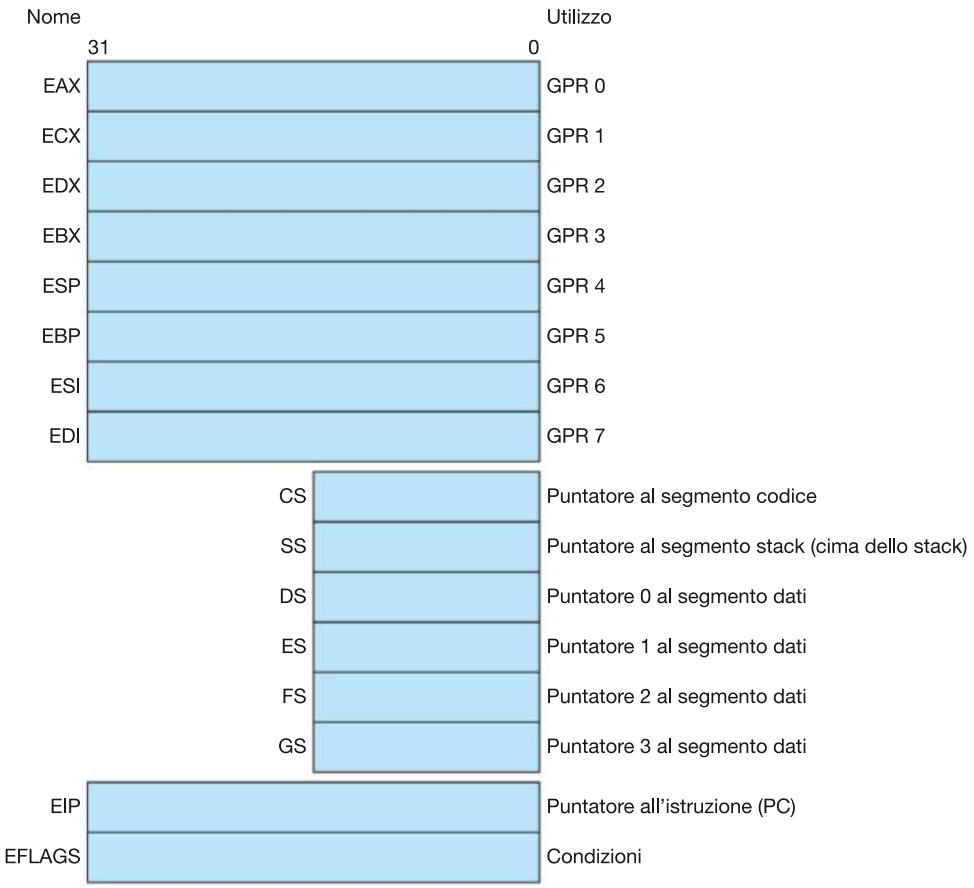


Figura 2.34 Insieme di registri dell'80386. A partire dall'80386, gli 8 registri in alto sono stati estesi a 32 bit e possono essere impiegati come registri a utilizzo generale.

Tipo di operando sorgente/destinazione	Secondo operando sorgente
Registro	Registro
Registro	Immediato
Registro	Memoria
Memoria	Registro
Memoria	Immediato

Figura 2.35 Combinazioni di operandi per le istruzioni aritmetiche, logiche e di trasferimento dati. L'x86 permette le combinazioni di operandi mostrate in figura: l'unica restrizione riguarda l'assenza di una modalità memoria-memoria. Gli operandi immediati possono avere una lunghezza di 8, 16 o 32 bit; il termine registro indica uno dei 14 registri principali della Figura 2.30 (tutti a eccezione di EIP e EFLAGS).

registri distinti per operandi sorgente e operando destinazione. Questa restrizione rende più complesso il problema del numero limitato di registri, dato che il contenuto di uno dei registri sorgente viene necessariamente modificato. La seconda importante differenza consiste nel fatto che uno degli operandi può essere contenuto in memoria; perciò praticamente tutte le istruzioni possono prendere uno dei due operandi dalla memoria, a differenza del RISC-V e del MIPS.

Modalità	Descrizione	Restrizione sui registri	Codice RISC-V equivalente
Indiretta tramite registro	L'indirizzo è in un registro	No ESP o EBP	`lw x10, 0(x11)`
Base più spiazzamento a 8 o 32 bit	L'indirizzo è il contenuto del registro base più lo spiazzamento	No ESP	`lw x10, 40(x11)`

Base più indice scalato	L'indirizzo è Base + (2Scala × Indice), dove Scala può valere 0, 1, 2 o 3	Base: qualsiasi GPR Indice: no ESP	`slli x12, x12, 3`
			`add x11, x11, x12`
			`lw x10, 0(x11)`

Base più indice scalato con spiazzamento a 8 o 32 bit	L'indirizzo è Base + (2^{Scala} × Indice) + spiazzamento, dove Scala può valere 0, 1, 2 o 3	Base: qualsiasi GPR Indice: no ESP	`slli x12, x12, 2`
			`add x11, x11, x12`
			`lw x10, 40(x11)`

Figura 2.36 Modalità di indirizzamento a 32 bit dell'x86, con restrizioni sui registri e codice equivalente RISC-V. La modalità di indirizzamento base più indice scalato, che non si trova né nel RISC-V né nel MIPS, è stata inserita per evitare la moltiplicazione per quattro (costante di scorrimento pari a 2) necessaria a trasformare l'indice contenuto in un registro in un indirizzo in byte (Figure 2.26 e 2.28). Una costante di scorrimento pari a 1 viene utilizzata per i dati a 16 bit, mentre una costante di scorrimento pari a 2 per quelli a 32 bit. Una costante di scorrimento pari a 0 significa che l'indirizzo non viene modificato. Se nella seconda o nella quarta modalità di indirizzamento lo spiazzamento fosse maggiore di 12 bit, allora il codice RISC-V equivalente richiederebbe più istruzioni: una `lui` per caricare i bit dello spiazzamento da 12 a 31 in un registro seguita da una `add` per sommare questi bit con l'indirizzo base. L'Intel dà due nomi differenti alla modalità di indirizzamento tramite registro base, cioè indirizzamento tramite base (Based) o indicizzato (Indexed) ma, dato che sono sostanzialmente identici, in questa figura sono riportati assieme.

I modi di indirizzamento dei dati in memoria, che descriveremo in dettaglio fra poco, consentono di inserire nella stessa istruzione indirizzi di due dimensioni diverse: il cosiddetto *spiazzamento* può essere di 8 o 32 bit.

Nonostante per un operando in memoria si possa usare qualunque modo di indirizzamento, ci sono alcune restrizioni sui *registri* utilizzabili nei vari modi. La Figura 2.36 mostra le modalità di indirizzamento dell'x86 e i registri GPR che non possono essere utilizzati nelle diverse modalità. Sono riportate anche le istruzioni RISC-V che consentono di ottenere lo stesso risultato.

Operazioni su numeri interi dell'x86

L'8086 consente di utilizzare sia dati a 8 bit (*byte*) sia a 16 bit (*parola*); come abbiamo visto l'80386 aggiunge indirizzi e dati a 32 bit (*doppia parola*) all'architettura x86. L'AMD64 prevede anche la possibilità di utilizzare indirizzi e dati a 64 bit, chiamati *quad word* (*parola quadrupla*), ma in questo paragrafo ci limiteremo all'analisi dell'80386. La differenza tra i diversi tipi di dati si applica sia alle operazioni sui registri sia agli accessi alla memoria.

Quasi tutte le istruzioni possono operare sia su dati a 8 bit che su dati di dimensione più ampia. Questa viene definita nel campo modalità (*mode*) e può essere di 16 o 32 bit.

Chiaramente alcuni programmi devono operare su dati di tutte e tre le dimensioni, quindi l'80386 fornisce un modo pratico per specificare la diversa dimensione dei dati senza estendere le dimensioni del codice in modo significativo. I progettisti pensarono che la maggior parte dei programmi avrebbe usato prevalentemente dati a 16 o 32 bit e che quindi aveva senso impostare come default una di queste due dimensioni. La dimensione di default viene specificata in uno dei bit del registro di segmento del codice; per sovrascriverla, occorre precedere l'istruzione con un *prefisso* di 8 bit, che specifica al calcolatore di utilizzare l'altra dimensione grande dei dati (16 o 32 bit) per quella istruzione.

La soluzione di adottare un prefisso fu presa in prestito dall'8086, che prevedeva prefissi diversi per modificare il comportamento delle istruzioni. I tre prefissi originari servivano per modificare i registri di segmento predefiniti,

per impedire l'accesso al bus permettendo la sincronizzazione dei processi (Paragrafo. 2.11) o per ripetere l'istruzione successiva fino a quando il registro contatore, ECX, non raggiungeva il valore 0. Quest'ultimo prefisso era stato pensato per essere utilizzato con un'istruzione di move e aveva lo scopo di trasferire un numero variabile di byte. L'80386 prevede anche un altro prefisso da utilizzare per modificare la dimensione di default degli indirizzi.

Le operazioni sui numeri interi dell'x86 si possono suddividere in quattro classi principali:

1. Le istruzioni per lo spostamento dei dati, che comprendono move, push e pop;
2. Le istruzioni aritmetico-logiche, comprese quelle di confronto, e le operazioni aritmetiche su numeri interi e decimali;
3. Le istruzioni che controllano il flusso del programma e che comprendono i salti condizionati e incondizionati, le chiamate a procedura e il ritorno da procedura;
4. L e istruzioni per operazioni su stringhe, che comprendono lo spostamento e il confronto tra due stringhe.

Le prime due classi non meritano particolare attenzione, a eccezione del fatto che le istruzioni aritmetico-logiche prevedono che la destinazione sia un registro o una locazione di memoria. La **Figura 2.37** mostra alcune istruzioni tipiche dell'x86 e il loro funzionamento.

Nell'x86 i salti condizionati si basano sui bit del risultato dei test, detti *condition codes* o *flags*. Questi bit vengono posti a 1 come conseguenza di un'operazione: la maggior parte di essi viene utilizzata per confrontare un certo risultato con 0. I salti condizionati, quindi, effettuano un controllo su un determinato flag. L'indirizzo di salto relativo ai salti condizionati, come spiazzamento rispetto al PC, viene specificato in numero di byte, dato che le istruzioni di un 80386 non sono tutte lunghe 4 byte come nel RISC-V e nel MIPS. Le istruzioni che operano su stringhe risalgono all'antenato 8080 e nella maggior parte dei programmi non vengono utilizzate; in generale, infatti, sono più lente delle procedure software equivalenti (Paragrafo 2.2: "Errori e trabocchetti").

Alcune istruzioni su numeri interi dell'x86 sono riportate nella **Figura 2.38**; molte di esse sono disponibili sia per dati in formato word sia in formato byte.

Istruzione	Funzione
`je` nome	`se uguale(codice di condizione){EIP=nome}; EIP-128 ≤ nome < EIP+128`
`jmp` nome	`EIP=nome`
`call` nome	`SP=SP-4; M[SP]=EIP+5; EIP=nome;`
`movw EBX,[EDI+45]`	`EBX=M[EDI+45]`
`push ESI`	`SP=SP-4; M[SP]=ESI`
`pop EDI`	`EDI=M[SP]; SP=SP+4`
`add EAX,#6765`	`EAX= EAX+6765`
`test EDX,#42`	Imposta i flag con il risultato dell'operazione con EDX e 42
`movsl`	`M[EDI]=M[ESI]; EDI=EDI+4; ESI=ESI+4`

Figura 2.37 Alcune tipiche istruzioni dell'x86 con relativa funzione. Un elenco delle operazioni utilizzate più di frequente compare nella Figura 2.34. L'istruzione CALL salva nello stack il puntatore (EIP) all'istruzione successiva (EIP è il PC dell'Intel).

 © 978-88-08-19966-9

Istruzione	Significato
Controllo	**Salti condizionati e incondizionati**
`jnz, jz`	Salto condizionato a EIP + offset su 8 bit; nomi alternativi sono `JNE` (per `JNZ`) e `JE` (per `JZ`)
`jmp`	Salto incondizionato; offset su 8 o 16 bit
`call`	Chiamata a procedura, con offset di 16 bit; l'indirizzo di ritorno viene inserito nello stack
`ret`	Prende dallo stack l'indirizzo di ritorno ed esegue il salto a quell'indirizzo
`loop`	Salto all'inizio di un ciclo: decrementa ECX e salta a EIP + offset su 8 bit, se ECX è diverso da 0
Trasferimento dati	**Spostamento dati fra registri o fra registri e memoria**
`move`	Sposta un dato da un registro a un altro o da registro a memoria
`push, pop`	Mette nello stack un operando, recupera dalla cima dello stack un operando e lo mette in un registro
`les`	Carica nel registro ES o in uno dei GPR il contenuto della memoria
Operazioni aritmetico-logiche	**Operazioni aritmetico-logiche su dati nei registri o in memoria**
`add, sub`	Somma il sorgente alla destinazione; sottrae il sorgente dalla destinazione; formato registro-memoria
`cmp`	Confronta il sorgente con la destinazione; formato registro-memoria
`shl, shr, rcr`	Scorrimento a sinistra, a destra e rotazione verso destra con impostazione del flag di riporto
`cbw`	Converte il byte che si trova negli 8 bit meno significativi di EAX in una parola che occupa i 16 bit meno significativi di EAX
`test`	Mette i valori opportuni nel registro dei flag facendo l'AND logico fra sorgente e destinazione
`inc, dec`	Incrementa, decrementa il registro destinazione
`or, xor`	OR logico, OR esclusivo; formato registro-memoria
Stringhe	**Trasferimento del contenuto di stringhe; lunghezza fornita nel prefisso di ripetizione**
`movs`	Copia una stringa sorgente in una stringa destinazione incrementando ESI ed EDI: può essere ripetuta
`lods`	Carica nel registro EAX un byte, una parola singola o doppia contenenti una stringa

Figura 2.38 Alcune tipiche operazioni dell'x86. Molte operazioni utilizzano il formato registro-memoria, dove uno dei due operandi (sorgente o destinazione) si trova in memoria e l'altro può essere contenuto in un registro o essere un operando immediato.

Codifica delle istruzioni x86

La codifica delle istruzioni dell'80386 è assai complessa, essendoci per le istruzioni molti formati diversi che possono variare da 1 byte quando non ci sono operandi fino a 15 byte.

La Figura 2.39 mostra il formato di molte istruzioni campione presentate nella Figura 2.37. Il byte del codice operativo solitamente contiene un bit che dice se l'operando è a 8 o 32 bit. Il codice operativo di alcune istruzioni può comprendere la modalità di indirizzamento e il registro; questo si verifica in molte istruzioni che hanno la forma "registro = registro operazione-operando immediato". Altre istruzioni utilizzano un "post-byte" o byte aggiuntivo per il codice operativo, chiamato "mod, reg, r/m", che contiene le informazioni relative alla modalità di indirizzamento; questo byte aggiuntivo è presente in molte istruzioni che accedono alla memoria. La modalità di indirizzamento "base più indice scalato" utilizza un secondo byte aggiuntivo, che può assumere i valori "sc, indice, base".

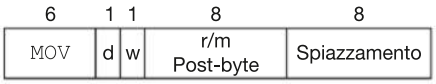
La Figura 2.40 mostra la codifica dei due post-byte per la specifica degli indirizzi, sia per la modalità a 16 bit sia per quella a 32 bit.

Sfortunatamente, per capire a fondo quali registri e quali modalità di indirizzamento siano disponibili, occorre considerare la codifica di tutte le modalità di indirizzamento e a volte persino quella delle singole istruzioni.

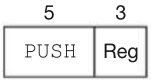
a. JE EIP + spiazzamento

b. CALL

c. MOV EBX, [EDI + 45]



d. PUSH ESI



e. ADD EAX, #6765

f. TEST EDX, #42

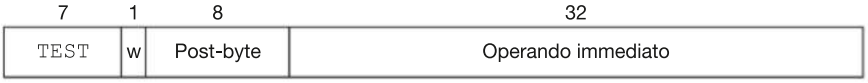


Figura 2.39 Formati tipici delle istruzioni x86. La codifica del post-byte è mostrata nella Figura 2.40. Molte istruzioni contengono il campo *w*, su un bit, che specifica se l'operazione agisce su un byte o su una parola doppia. Il campo *d* all'interno della MOV viene utilizzato nelle operazioni di trasferimento da e verso la memoria per specificare la direzione del trasferimento. L'istruzione ADD prevede 32 bit per il campo immediato, dato che nella modalità a 32 bit gli operandi immediati possono essere a 8 o 32 bit. Il campo immediato dell'istruzione TEST è lungo 32 bit, perché nella modalità a 32 bit non si possono utilizzare operandi immediati a 8 bit. Complessivamente la lunghezza di un'istruzione può variare da 1 a 15 byte. Le istruzioni più lunghe devono la loro dimensione alla presenza dei byte di prefisso, associati a un operando immediato su 4 byte e a un indirizzo con spiazzamento su 4 byte, di un codice operativo su 2 byte e di un byte che specifica quale base più indice scalato costituisca la modalità di indirizzamento utilizzata.

reg	w = 0	w = 1		r/m	mod = 0		mod = 1		mod = 2		mod = 3
		16b	32b		16b	32b	16b	32b	16b	32b	
0	AL	AX	EAX	0	ind=BX+SI	=EAX	*stesso ind di mod=0 + disp8*	*stesso ind di mod=0 + disp8*	*stesso ind di mod=0 + disp16*	*stesso ind di mod=0 + disp32*	*come il campo reg*
1	CL	CX	ECX	1	ind=BX+DI	=ECX					
2	DL	DX	EDX	2	ind=BP+SI	=EDX					
3	BL	BX	EBX	3	ind=BP+SI	=EBX					
4	AH	SP	ESP	4	ind=SI	=(*sib*)	SI+disp8	(*sib*)+disp8	SI+disp8	(*sib*)+disp32	”
5	CH	BP	EBP	5	ind=DI	=disp32	DI+disp8	EBP+disp8	DI+disp16	EBP+disp32	”
6	DH	SI	ESI	6	ind=disp16	=ESI	BP+disp8	ESI+disp8	BP+disp16	ESI+disp32	”
7	BH	DI	EDI	7	ind=BX	=EDI	BX+disp8	EDI+disp8	BX+disp16	EDI+disp32	”

Figura 2.40 La codifica del primo campo utilizzato per la specifica degli indirizzi negli x86: "mod, reg, r/m". Le prime quattro colonne mostrano la codifica su 3 bit del campo *reg* (registro); il nome del registro dipende dal bit *w* del codice operativo e dal fatto che il calcolatore si trovi nella modalità a 16 bit (8086) o in quella a 32 bit (80386). Le restanti colonne illustrano i campi *mod* e *r/m*. Il significato del campo di 3 bit *r/m* dipende dal valore contenuto nel campo da 2 bit, *mod*, e dalle dimensioni dell'indirizzo. In pratica i registri utilizzati nel calcolo dell'indirizzo sono elencati nella sesta e settima colonna sotto *mod* = 0. Viene aggiunto uno spiazzamento (disp) su 8 bit se *mod* = 1, e uno spiazzamento su 16 bit o 32 bit, secondo la modalità di indirizzamento corrente, se *mod* = 2. Le seguenti tre combinazioni costituiscono una eccezione: 1) *r/m* = 6 e *mod* = 1 oppure *mod* = 2; in questa configurazione, se la CPU si trova nella modalità a 16 bit, viene selezionato il registro BP a cui viene aggiunto lo piazzamento; 2) *r/m* = 5 e *mod* = 1 oppure *mod* = 2; in questa configurazione, se la CPU si trova nella modalità a 32 bit, viene selezionato il registro EBP a cui viene aggiunto lo spiazzamento; 3) *r/m* = 4 e *mod* ≠ 3; in questa configurazione, se la CPU si trova nella modalità a 32 bit, viene utilizzata la modalità di indirizzamento a indice scalato (sib) mostrata nella Figura 2.36. Quando *mod* = 3, il campo *r/m* indica un registro, seguendo lo stesso tipo di codifica ottenuta combinando il campo reg con il bit *w*.

Conclusioni sull'x86

L'Intel ha costruito un microprocessore a 16 bit due anni prima di quelli, più eleganti, costruiti dai suoi concorrenti, come il Motorola 68000, e questo ha fatto sì che l'8086 fosse scelto come CPU per il PC IBM. In generale gli ingegneri dell'Intel riconoscono che è più difficile costruire un x86 piuttosto che un MIPS o un RISC-V, ma l'ampiezza del mercato delle loro CPU nell'era dei PC ha consentito a Intel e ad AMD di avere più risorse da investire per superare i problemi posti dalla crescente complessità. Ciò che all'x86 manca in stile è compensato dalla sua diffusione sul mercato, rendendolo apprezzabile se visto nella giusta prospettiva.

Inoltre, i componenti architetturali dell'x86 utilizzati più di frequente non sono troppo difficili da implementare, come Intel e AMD hanno dimostrato migliorando rapidamente le prestazioni dei programmi che lavorano sui numeri interi, sin dal 1978. Per ottenere queste prestazioni i compilatori devono evitare le parti dell'architettura difficili da implementare in modo veloce.

Ma nell'era post-PC, nonostante la grande esperienza nell'architettura e nella sua produzione, l'x86 non è ancora diventato competitivo nel campo dei dispositivi personali mobili (PMD).

2.20 Un caso reale: le altre istruzioni dell'architettura RISC-V

Con l'obiettivo di realizzare un'architettura di un insieme di istruzioni adatto a una grande varietà di calcolatori, i progettisti del RISC-V hanno suddiviso l'insieme delle istruzioni in un'*architettura di base* e diverse *estensioni*. Ciascuna di esse è denominata con le lettere dell'alfabeto e l'architettura di base è chiamata `I` che sta per *interi*. L'architettura di base contiene poche istruzioni rispetto ad altri insiemi di istruzioni popolari oggigiorno, e questo capitolo le ha già introdotte quasi tutte. Questo paragrafo completa le istruzioni dell'architettura base e descrive le cinque estensioni standard.

La Figura 2.41 riporta le istruzioni non ancora trattate dell'architettura di base RISC-V. La prima istruzione, `auipc`, viene utilizzata per l'indirizzamento della memoria relativo al PC. Come l'istruzione `lui`, contiene una costante su 20 bit nei bit da 12 a 31 di un intero. L'effetto di `auipc` è quello di aggiungere questo numero al PC e scrivere la somma in un registro. Combinata con un'istruzione come `addi`, è possibile indirizzare un qualsiasi byte della memoria entro 4 GiB dall'indirizzo corrente nel PC. Questa caratteristica è utile per il *codice indipendente dalla posizione*, che viene così eseguito correttamente indipendentemente dalla posizione della memoria in cui viene caricato. Viene utilizzata soprattutto nelle librerie a collegamento dinamico.

Istruzione	Nome	Formato	Descrizione
Somma i bit più significativi al PC	`auipc`	U	Somma i 20 bit più significativi al PC; scrivi la somma in registro
Imposta a 1 se minore	`slt`	R	Confronta 2 registri; scrivi il risultato come booleano in registro
Imposta a 1 se minore, senza segno	`sltu`	R	Confronta 2 registri; scrivi il risultato come booleano in registro
Imposta a 1 se minore, immediato	`slti`	I	Confronta 1 registro e 1 immediato; scrivi il risultato come booleano in registro
Imposta a 1 se minore, immediato, senza segno	`sltiu`	I	Confronta 1 registro e 1 immediato; scrivi il risultato come booleano in registro

Figura 2.41 Le restanti 15 istruzioni dell'architettura dell'insieme delle istruzioni di base del RISC-V.

Nome simbolico	Descrizione	Numero di istruzioni
I	Architettura di base	51
M	Moltiplicazione/divisione intera	13
A	Operazioni atomiche	22
F	Virgola mobile in precisione singola	30
D	Virgola mobile in doppia precisione	32
C	Istruzioni compresse	36

Figura 2.42 L'architettura dell'insieme delle istruzioni RISC-V è suddivisa in ISA di base, denominata I, e in cinque estensioni M, A, F, D e C.

Le quattro istruzioni successive confrontano due interi e scrivono il risultato come variabile booleana in un registro. `slt` e `sltu` confrontano il contenuto di due registri come numeri con e senza segno, rispettivamente, e scrivono 1 in un registro se il primo valore è minore del secondo, o 0 nel caso opposto. `slti` e `sltiu` eseguono lo stesso confronto, ma con un valore immediato come secondo operando. E questo è tutto per l'architettura di base!

La Figura 2.42 riporta le cinque estensioni standard. La prima, M, aggiunge le istruzioni per moltiplicare e dividere i numeri interi. Il Capitolo 3 introdurrà diverse istruzioni dell'estensione M. La seconda estensione, A, supporta le operazioni atomiche sulla memoria per la sincronizzazione dei multiprocessori. Le istruzioni di load riservata (*load-reserved*, `lr.w`) e store condizionata (*store conditioned*, `sc.w`), introdotte nel Paragrafo 2.11, fanno parte dell'estensione A. Le restanti 18 istruzioni sono ottimizzazioni di schemi tipici di sincronizzazione, quali scambio atomico e somma atomica, ma non aggiungono funzionalità alla load riservata e alla store condizionata.

La terza e quarta estensione, F e D, forniscono operazioni su numeri in virgola mobile, che sono descritte nel Capitolo 3. L'ultima estensione, C, non fornisce nuove funzionalità, ma prende le istruzioni più comuni del RISC-V, come la `addi`, e fornisce le istruzioni equivalenti che sono lunghe solo 16 bit, invece di 32. Consente così di esprimere i programmi con meno byte, cosa che può ridurre i costi e, come vedremo nel Capitolo 5, può migliorare le prestazioni. Per potere essere contenute in 16 bit, queste nuove istruzioni hanno alcune restrizioni sugli operandi: per esempio, alcune istruzioni possono accedere solamente ad alcuni dei 32 registri, e i campi immediati sono più stretti.

Considerando tutte le istruzioni, il RISC-V di base e le sue estensioni hanno 184 istruzioni, più 13 istruzioni di sistema che verranno introdotte alla fine del Capitolo 5.

2.21 Come andare più veloci: la moltiplicazione di matrici in C

Iniziamo riscrivendo il programma Python del Paragrafo 1.10. La Figura 2.43 mostra una versione C del programma di moltiplicazione tra due matrici. Questo programma viene chiamato di solito DGEMM, che sta per *Double precision GEneral Matrix Multiply* (moltiplicazione generale tra matrici in doppia precisione). Poiché passiamo la dimensione delle matrici, `n`, come parametro questa versione di DGEMM utilizza versioni monodimensionali delle matrici `C`, `A` e `B` e l'aritmetica degli indirizzi per ottenere prestazioni migliori invece di utilizzare la struttura bidimensionale, più intuitiva, che abbiamo visto in Python. I commenti riportati nella figura fanno riferimento a questa notazione più intuitiva.

La Figura 2.44 mostra il codice assembler risultante che è associato al ciclo interno della Figura 2.43. Le cinque istruzioni in virgola mobile iniziano

```
1.  void dgemm (int n, double* A, double* B, double* C)
2.  {
3.     for (int i = 0; i < n; i++)
4.       for (int j = 0; j < n; j++)
5.       {
6.         double cij = C[i+j*n]; /* cij = C[i][j] */
7.         for (int k = 0; k < n; k++ )
8.           cij += A[i+k*n] * B[k+j*n]; /* cij += A[i][k]*B[k][j] */
9.         C[i+j*n] = cij; /* C[i][j] = cij */
10.      }
11. }
```

Figura 2.43 Versione C di un programma di moltiplicazione di due matrici in doppia precisione, noto come DGEMM che sta per Double precision General Matrix Multiply (moltiplicazione generale tra matrici in doppia precisione).

```
1.  vmovsd (%r10),%xmm0             # Carica in %xmm0 un elemento di C
2.  mov    %rsi,%rcx                # Copia %rsi nel registro %rcx
3.  xor    %eax,%eax                # Registra %eax = 0
4.  vmovsd (%rcx),%xmm1             # Registra %rcx = %rcx + %r9
5.  add    %r9,%rcx                 # Registra %rcx = %rcx + %r9
6.  vmulsd (%r8,%rax,8),%xmm1,%xmm1 # Moltiplica %xmm1 con un elemento di A
7.  add    $0x1,%rax                # Registra %rax = %rax + 1
8.  cmp    %eax,%edi                # Compara %eax con %edi
9.  vaddsd %xmm1,%xmm0,%xmm0        # Somma %xmm1 con %xmm0
10. jg     30 <dgemm+0x30>          # Salta se %eax > %edi
11. add    $0x1,%r11                # Registra %r11 = %r11 + 1
12. vmovsd %xmm0,(%r10)             # Salva %xmm0 in un elemento di C
```

Figura 2.44 Il codice assembler x86 associato al corpo dei cicli annidati ottenuti compilando il codice C non ottimizzato della Figura 2.43 utilizzando il compilatore gcc con i flag di ottimizzazione –O3.

con una v e contengono sd nel nome, che sta per *scalar double precision* (scalare in doppia precisione).

La Figura 2.45 mostra le prestazioni del programma C al variare del parametro di ottimizzazione da confrontare con le prestazioni del programma Python. Anche il programma C non ottimizzato risulta drammaticamente più veloce e all'aumentare del livello di ottimizzazione, diventa ancora più veloce. Il prezzo da pagare è un tempo di compilazione più lungo. Le ragioni di questo speed-up risiedono essenzialmente nell'utilizzo di un compilatore invece di un interprete e nel fatto che la dichiarazione dei tipi in C consente al compilatore di produrre un codice molto più efficiente.

–O0 (tempo di compilazione minore)	–O1	–O2	–O3 (tempo di compilazione maggiore)
77	208	212	212

Figura 2.45 Le prestazioni del programma C riportato nella Figura 2.43 rispetto al programma Python del Paragrafo 1.10 all'aumentare del livello di ottimizzazione del compilatore C, GCC.

2.22 Errori e trabocchetti

***Errore**: più le istruzioni sono potenti, migliori sono le prestazioni.*

Parte della potenza dell'architettura Intel x86 consiste nell'utilizzo dei prefissi che possono modificare l'esecuzione dell'istruzione. Per esempio, esiste un prefisso che causa la ripetizione dell'istruzione fino a che un contatore non raggiunge lo 0; quindi per eseguire trasferimenti sequenziali di dati da 32 bit da una porzione di memoria a un'altra, può sembrare che la soluzione migliore sia un'istruzione di move, preceduta dal prefisso di ripetizione.

Un metodo alternativo, che utilizza le istruzioni standard presenti in tutti i calcolatori, consiste nel caricare i dati nei registri (load) per poi memorizzare il contenuto dei registri in memoria (store). Questa seconda versione del programma, con il codice ripetuto in modo da evitare il costo addizionale dei cicli, è 1,5 più veloce. Una terza versione, che utilizza i registri in virgola mobile invece dei registri interi dell'x86, trasferisce i dati 2 volte più velocemente dell'istruzione complessa.

***Errore**: programmare in linguaggio assembler permette di ottenere le prestazioni migliori.*

Una volta i compilatori producevano sequenze di istruzioni poco efficienti, ma il loro crescente miglioramento rende la distanza fra il codice generato dal compilatore e quello scritto a mano sempre minore. In effetti, per essere in grado di competere con gli attuali compilatori, un programmatore assembler dovrebbe avere perfettamente chiari i concetti descritti nei Capitoli 4 e 5 (pipeline del processore e gerarchia delle memorie).

La battaglia fra compilatori e programmatori assembler è in una fase in cui l'uomo perde terreno ogni giorno. Per esempio, il C offre al programmatore la possibilità di suggerire al compilatore quali variabili mantenere nei registri e quali copiare in memoria. Quando i compilatori non erano molto efficienti nell'allocazione dei registri, questi suggerimenti avevano un'importanza vitale, e alcuni vecchi libri di testo sul C dedicavano un buon numero di pagine a esempi sul modo di dare suggerimenti efficaci al compilatore. Attualmente i compilatori C di solito ignorano questi suggerimenti, dato che sono in grado di allocare i registri meglio di quanto sappia fare il programmatore.

Anche se scrivere direttamente in assembler consentisse di ottenere un codice più efficiente, gli svantaggi sarebbero la quantità maggiore di tempo impiegata nella codifica e nella ricerca degli errori, la mancanza di portabilità e la difficoltà di manutenzione del codice.

Uno dei pochi assiomi dell'ingegneria del software ampiamente accettati è che il tempo di codifica è proporzionale al numero di linee di codice e, ovviamente, un programma assembler contiene molte più linee di codice di un programma C o Java. Una volta terminato, inoltre, un programma può diventare assai diffuso: la vita delle applicazioni è sempre più lunga del previsto e ciò significa che qualcun altro dovrà necessariamente aggiornare il codice a distanza di anni, facendolo funzionare su nuove versioni del sistema operativo o su nuovi modelli di calcolatore. Scrivere in linguaggio ad alto livello non solo permette ai compilatori futuri di ottimizzare il codice per i calcolatori che man mano entrano in commercio, ma rende anche più semplice la manutenzione del software e garantisce una maggiore portabilità su architetture diverse.

***Errore:** l'importanza commerciale della compatibilità binaria implica che gli insiemi di istruzioni che hanno successo non cambino.*

 © 978-88-08-19966-9

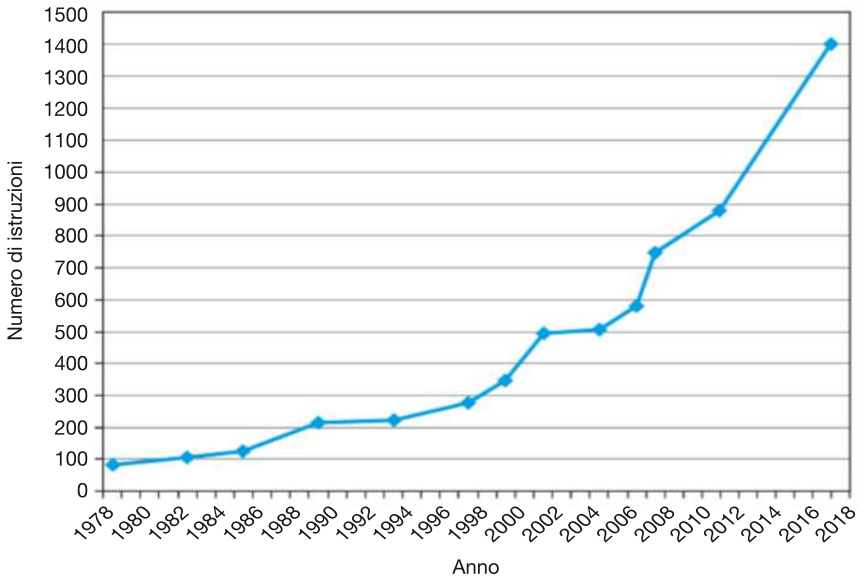


Figura 2.46 Crescita negli anni dell'insieme di istruzioni dell'x86. Sebbene alcune estensioni dell'insieme di istruzioni abbiano un evidente contenuto tecnico, il rapido incremento dell'insieme di istruzioni rende difficile per altre società cercare di costruire processori compatibili.

Anche se la compatibilità dei nuovi sistemi rispetto ai precedenti è sacrosanta, la Figura 2.46 mostra come l'architettura dell'x86 sia cresciuta enormemente, con un tasso di crescita medio di un'istruzione al mese per 40 anni!

> ***Trabocchetto***: *dimenticare che gli indirizzi di parole consecutive nei calcolatori con indirizzamento al byte non differiscono di una unità.*

Molti programmatori assembler hanno faticato a lungo per trovare errori commessi supponendo che l'indirizzo della parola successiva fosse ottenibile incrementando di una unità l'indirizzo contenuto in un registro, invece che della dimensione della parola in byte. Uomo avvisato, mezzo salvato!

> ***Trabocchetto***: *utilizzare un puntatore a una variabile locale al di fuori della procedura in cui la variabile è definita.*

Un errore comune, quando si ha a che fare con i puntatori, consiste nel passare come risultato di una procedura un puntatore a un vettore definito localmente nella procedura stessa. Secondo le regole di utilizzo dello stack descritte nella Figura 2.12, la parte di memoria che contiene il vettore locale verrà riutilizzata non appena la procedura sarà terminata e l'esecuzione tornerà alla procedura chiamante. I puntatori a variabili locali possono generare il caos.

2.23 Note conclusive

Less is more (Meno è meglio).
Robert Browning,
Andrea del Sarto, 1855

I calcolatori a *programma memorizzato* sono basati su due assunti fondamentali: le istruzioni sono indistinguibili dai numeri e la memoria in cui sono scritti i programmi può essere modificata. Questi princìpi fanno sì che lo stesso calcolatore possa essere utile a scienziati che si occupano di ambiente, a operatori finanziari e a romanzieri. La scelta dell'insieme di istruzioni

che il calcolatore è in grado di eseguire è il risultato di un delicato compromesso fra il numero di istruzioni necessarie per eseguire un programma, il numero di cicli di clock per istruzione e il periodo di clock. Come è stato illustrato in questo capitolo, sono tre i criteri che guidano i progettisti degli insiemi di istruzioni nel delicato compito di trovare il bilanciamento giusto: vediamoli.

1. *La semplicità favorisce la regolarità*. La ricerca della regolarità è alla base di molte caratteristiche dell'insieme di istruzioni RISC-V: stessa dimensione per tutte le istruzioni, richiesta di operandi di tipo registro in tutte le istruzioni aritmetiche, mantenimento dei campi registro nella stessa posizione all'interno di tutti i formati delle istruzioni.
2. *Minori sono le dimensioni, maggiore è la velocità*. La velocità di esecuzione è il motivo per cui il RISC-V ha 32 registri anziché molti di più.
3. *Un buon progetto richiede buoni compromessi*. Un esempio relativo al RISC-V è il compromesso raggiunto tra fornire indirizzi e costanti di dimensioni maggiori e mantenere la stessa lunghezza per tutte le istruzioni.

Un'altra delle grandi idee illustrate in questo capitolo è che i numeri non hanno un tipo intrinsecamente associato a essi, ma una sequenza di bit può rappresentare un numero intero, una stringa, un colore o perfino un'istruzione. È il programma a determinare il tipo di dato.

SITUAZIONI COMUNI

Abbiamo anche visto all'opera la grande idea introdotta nel primo capitolo di **rendere veloci le situazioni più comuni** applicata agli insiemi di istruzioni e all'architettura del calcolatore. Esempi di questo tipo che riguardano il RISC-V sono la modalità di indirizzamento relativo al PC per i salti condizionati e l'indirizzamento immediato per gli operandi costanti di dimensioni più ampie. La **Figura 2.47** riporta le istruzioni RISC-V trattate fino a ora.

ASTRAZIONE

Al di sopra del linguaggio macchina c'è il linguaggio assembler, leggibile anche dagli uomini, che viene tradotto in numeri binari comprensibili al calcolatore dall'assemblatore. Questo può addirittura "estendere" l'insieme di istruzioni creando istruzioni simboliche non implementate a livello hardware. Per esempio, le costanti o gli indirizzi troppo grandi vengono suddivisi in frammenti di dimensione adeguata; le varianti più comuni di determinate istruzioni prendono un nome che li contraddistingue e così via. Nascondere i dettagli delle istruzioni ai livelli superiori è un altro esempio della grande idea dell'**astrazione**.

I diversi tipi di istruzioni RISC-V sono associati a costrutti contenuti nei linguaggi di programmazione ad alto livello:

- le istruzioni aritmetiche corrispondono alle operazioni che si possono trovare negli assegnamenti;
- le istruzioni di trasferimento dati sono più frequenti quando si utilizzano strutture dati, come i vettori o le strutture;
- le istruzioni di salto condizionato vengono utilizzate nei costrutti *if* e nei cicli;
- le istruzioni di salto incondizionato vengono utilizzate nelle chiamate a procedura, nel ritorno da procedura e nei costrutti *case/switch*.

Non tutte le istruzioni sono uguali: alcune sono utilizzate con più frequenza rispetto ad altre. Per esempio, la **Figura 2.48** mostra la frequenza di utilizzo di ciascun tipo di istruzione nei programmi dello SPEC2000. Questa differenza gioca un ruolo importante nella progettazione, come verrà discusso nei capitoli che riguardano unità di elaborazione, unità di controllo e pipeline.

Dopo avere illustrato l'aritmetica dei calcolatori, a cui è dedicato il Capitolo 3, presenteremo le altre istruzioni dell'architettura dell'insieme di istruzioni RISC-V.

Istruzione RISC-V	Nome	Formato
Somma	`add`	R
Sottrazione	`sub`	R
Somma immediata	`addi`	I
Lettura parola	`lw`	I
Lettura parola senza segno	`lwu`	I
Memorizzazione parola	`sw`	S
Lettura mezza parola	`lh`	I
Lettura mezza parola senza segno	`lhu`	I
Memorizzazione mezza parola	`sh`	S
Lettura byte	`lb`	I
Lettura byte senza segno	`lbu`	I
Memorizzazione byte	`sb`	S
Lettura riservata	`lr.d`	R
Memorizzazione condizionata	`sc.d`	R
Lettura dei bit superiori	`lui`	U
And	`and`	R
Or inclusivo	`or`	R
Or esclusivo	`xor`	R
And immediato	`andi`	I
Or inclusivo immediato	`ori`	I
Or esclusivo immediato	`xori`	I
Scorrimento logico sinistra	`sll`	R
Scorrimento logico destra	`srl`	R
Scorrimento aritmetico destra	`sra`	R
Scorrimento logico sinistra immediato	`slli`	I
Scorrimento logico destra immediato	`srli`	I
Scorrimento aritmetico destra immediato	`srai`	I
Salta se uguale	`beq`	SB
Salta se non uguale	`bne`	SB
Salta se minore	`blt`	SB
Salta se maggiore o uguale	`bge`	SB
Salta se minore senza segno	`bltu`	SB
Salta se maggiore o uguale senza segno	`bgeu`	SB
Jump and link	`jal`	UJ
Jump and link a registro	`jalr`	I

Figura 2.47 Insieme delle istruzioni RISC-V esaminate finora. La Figura 2.1 mostra ulteriori dettagli sulla parte di architettura RISC-V presentata in questo capitolo. Si possono trovare queste informazioni anche nella prima e seconda colonna della scheda tecnica riassuntiva del RISC-V in fondo al libro.

Tipi di istruzioni	Esempi RISC-V	Corrispondenza con linguaggi ad alto livello	Frequenza	
			Interi	Virgola mobile
Aritmetiche	`add, sub, addi`	Operazioni in istruzioni di assegnamento	16%	48%
Trasferimento dati	`ld, sd, lw, sw, lh, sh, lb, sb, lui`	Riferimento a strutture dati, per esempio a vettori	35%	36%
Logiche	`and, or, xor, sll, srl, sra`	Operazioni in istruzioni di assegnamento	12%	4%
Salti condizionati	`beq, bne, blt, bge, bltu, bgeu`	Costrutti *if* e *cicli*	34%	8%
Salti incondizionati	`jal, jalr`	Chiamate a procedura, ritorno da procedura e costrutti *switch*	2%	0%

Figura 2.48 Tipi di istruzioni RISC-V, esempi, corrispondenza con i costrutti dei linguaggi di programmazione ad alto livello e percentuale di istruzioni RISC-V di ognuno dei diversi tipi, eseguite in media nei programmi di benchmark dello SPEC CPU2006. Nella Figura 3.24 del Capitolo 3 verranno mostrate le percentuali relative alle singole istruzioni RISC-V eseguite.

2.24 Inquadramento storico e approfondimenti

Questo paragrafo contiene una rassegna dell'evoluzione delle *architetture degli insiemi di istruzioni* (ISA), assieme a una breve storia dei linguaggi di programmazione e dei compilatori.

Le architetture di insiemi di istruzioni comprendono architetture ad accumulatore, architetture a registri a utilizzo generale, architetture a stack e una breve storia dell'architettura x86 e ARM a 32 bit, ARM v7. Verrà anche esaminata la tematica controversa del confronto tra architetture di calcolatori basate su linguaggi ad alto livello e architetture di calcolatori basate su un insieme ridotto di istruzioni. La rassegna dei linguaggi di programmazione tratterà Fortran, Lisp, Algol, C, Cobol, Pascal, Simula, Smalltalk, C++ e Java, mentre quella dei compilatori comprenderà le pietre miliari del loro sviluppo e i pionieri che hanno permesso di raggiungerle. Il testo del Paragrafo 2.24 è disponibile online .

2.25 Aiuto allo studio

Istruzioni come numeri. Dato il seguente numero binario intero:

$$\mathbf{00000001010010110100100000100011}_{\text{due}}$$

Qual è la sua rappresentazione esadecimale? Supponendo che sia un numero senza segno, quale sarebbe la sua codifica decimale? Cambia il valore se consideriamo il numero dotato di segno? Quale istruzione assembler viene rappresentata dalla stringa di bit?

Istruzioni come numeri e sicurezza. Anche se i programmi sono essenzialmente numeri in memoria, il Capitolo 5 mostra come dire al computer come proteggere i programmi da alterazioni, marcando a sola lettura una porzione dello spazio di indirizzamento. I pirati informatici più ingegnosi sfruttano i bachi dei programmi C per inserire comunque il loro codice durante l'esecuzione di un programma nonostante quel programma sia protetto.

Questo è un semplice programma di copia di una stringa: copia quello che digita l'utente in una variabile locale nello stack.

```
#include <string.h>
void copiaInput(char *input)
{
   char copiaLoc[10];
   strcpy(copiaLoc,input); // non viene controllato il fine stringa
                           in strcpy;
}
int main(int argc, char **argv)
{
   copiaInput(argv[1]);
   return(0);
}
```

Cosa succede se l'utente digita in ingresso più di 10 caratteri? Quale saranno le conseguenze sull'esecuzione del programma? Come questo può consentire a un pirata informatico di prendete il controllo sull'esecuzione del programma?

Ciclo "while" più veloce. Questo è il codice MIPS associato al ciclo C di pagina 87:

```
Ciclo:  slli x10,x22,2     // Registro temp x10 = i*4
        add  x10,x10,x25   // $t1 = indirizzo di salva[i]
        lw   x9,0(x10)     // Registro temp x9=salva[i]
        bne  x9,x24,Esci   // Vai a Esci se salva[i] ≠ k
        addi x22,x22,1     // i=i+1
        j    Ciclo         // vai a Ciclo
Esci:
```

Si supponga che il ciclo venga eseguito 10 volte. Si può rendere il ciclo più veloce eseguendo in media una istruzione di salto condizionato per ciclo invece della coppia di istruzioni: salto condizionato e salto incondizionato.

Il compilatore. Questa è una porzione di codice assembler RISC-V con le prime cinque istruzioni commentate.

```
slli  x5,x18,2            // x5 = f*4
add  x5,x23,x5            // x5 = &A[f]
slli  x6,x19,2            // x6 = g*4
add  x6,x24,x6            // x6 = &B[g]
lw   x18,0(x5)            // f = A[f]
addi x7,x5,4              //
lw   x0,0(x7)             //
add  x5,x5,x18            //
sw   x5,0(x6)             //
```

Si supponga che le variabili `f` e `g` siano assegnate ai registri `x18` e `x19` rispettivamente. Si supponga che l'indirizzo di base dei vettori `A` e `B` sia contenuto nei registri `x23` e `x24` rispettivamente. Inserire i commenti per le ultime quattro istruzioni.

Risposte ai quesiti

Istruzioni come numeri

Binario: **00000001010010110100100000100011**$_{due}$
Esadecimale: **014B4823**$_{esa}$
Decimale: **21702691**$_{dec}$

Dato che il primo bit contiene 0, il valore intero codificato è lo stesso se il numero è considerato con segno o senza segno.

Linguaggio assembler: `sw x20, 16(x22)`
Linguaggio macchina:

31 25	24 20	19 15	14 # 12	11 7	6 0
immediato[11:5]	rs2	rs1	funz3	immediato[4:0]	codop
0000000	10100	10110	010	10000	0100011
7	5	5	3	5	7

Istruzioni come numeri e sicurezza. La variabile locale `copiaLoc` può contenere in modo sicuro fino a nove caratteri seguiti dal carattere null che funge da terminatore della stringa. I caratteri in eccesso vanno a sovrascrivere altre celle di memorie in stack. Man mano che lo stack cresce verso il basso, i valori al di sotto dello stack conterranno i frame di stack, costruiti dalle chiamate a procedura precedenti, che contengono anche l'indirizzo di ritorno di quella procedura. Un pirata informatico particolarmente abile non solo può inserire del codice

all'interno dello stack, ma può così alterare l'indirizzo di ritorno memorizzato in stack in modo tale che il programma in esecuzione possa andare a utilizzare l'indirizzo di ritorno inserito dal pirata per iniziare l'esecuzione di codice inserito in stack dopo che le procedure sono ritornate al loro programma chiamante.

Ciclo "while" più veloce. Il trucco è invertire il salto condizionato per farlo saltare all'inizio del ciclo invece di utilizzare il salto incondizionato al termine del ciclo. Per mantenere la semantica del ciclo while, il codice deve controllare se `salva [i] == k` prima di incrementare `i`.

```
        slli x10,x22,2        // Registro temp x10 = i*4
        add  x10,x10,x25      // $t1 = indirizzo di salva[i]
        lw   x9,0(x10)        // Registro temp x9=salva[i]
        bne  x9,x24,Esci      // Vai a Esci se salva[i] ≠ k
Ciclo:  addi  x22,x22,1       // i=i+1
        slli x10,x22,2        // Registro temp x10 = i*4
        add  x10,x10,x25      // $t1 = indirizzo di salva[i]
        lw   x9,0(x10)        // Registro temp x9=salva[i]
        beq  x9,x24,Ciclo     // Vai a Ciclo se salva[i] = k
Esci:
```

Il compilatore

```
slli  x5,x18,2          // x5 = f*4
add   x5,x23,x5         // x5 = &A[f]
slli  x6,x19,2          // x6 = g*4
add   x6,x24,x6         // x6 = &B[g]
lw    x18,0(x5)         // f = A[f]
addi x7,x5,4            // x7=x5+4 -> x5 punta ora A[f+1]
lw    x5,0(x7)          // x5=A[f+1]
add   x5,x5,x18         // x5 = x5 + x18 -> x5 ora vale A[f] + A[f+1]
sw    x5,0(x6)          // scrivi il risultato in B[g]
```

L'istruzione C equivalente è:

```
B[g] = A[f] + A[f+1];
```

2.26 Esercizi

2.1 [5] <2.2> Qual è il codice assembler RISC-V corrispondente alla seguente istruzione C? Si supponga che le variabili `f`, `g` e `h` siano già state memorizzate nei registri `x5`, `x6` e `x7` rispettivamente. Si utilizzi il numero minimo di istruzioni assembler.

```
f = g + (h - 5);
```

2.2 [5] <2.2> Quali sono le istruzioni C che corrispondono alle seguenti due istruzioni in assembler RISC-V?

```
add f, g, h
add f, i, f
```

2.3 [5] <2.2, 2.3> Qual è il codice assembler RISC-V corrispondente alla seguente istruzione C? Si supponga che le variabili `f`, `g`, `h`, `i` e `j` siano assegnate rispettivamente ai registri `x5`, `x6`, `x7`, `x28` e `x29`. Si assuma che l'indirizzo base dei vettori `A` e `B` sia contenuto rispettivamente nei registri `x10` e `x11`.

```
B[8] = A[i - j];
```

2.4 [10] <2.2, 2.3> Qual è il codice C assembler RISC-V corrispondente alle seguenti istruzioni assembler RISC-V? Si supponga che le variabili `f`, `g`, `h`, `i` e `j` siano assegnate rispettivamente ai registri `x5`, `x6`, `x7`, `x28` e `x29`. Si assuma che l'indirizzo base dei vettori `A` e `B` sia contenuto rispettivamente nei registri `x10` e `x11`.

```
slli x30, x5, 2     // x30 = f * 4
add  x30, x10, x30  // x30 = &A[f]
slli x31, x6, 2     // x31 = g * 4
add  x31, x11, x31  // x31 = &B[g]
lw   x5,  0(x30)    // f = A[f]
```

```
addi x12, x30, 4
lw   x30, 0(x12)
add  x30, x30, x5
sw   x30, 0(x31)
```

2.5 [5] <2.3> Mostrare come il numero esadecimale 0xabcdef12 viene disposto in memoria in una macchina con codifica little-endian e in una con codifica big-endian. Si supponga che i dati vengano memorizzati a partire dall'indirizzo 0 e che la dimensione della parola sia di 4 byte.

2.6 [5] <2.4> Tradurre 0xabcdef12 in decimale.

2.7 [5] <2.2, 2.3> Tradurre la seguente istruzione C in assembler RISC-V. Si supponga che le variabili f, g, h, i e j siano assegnate rispettivamente ai registri x5, x6, x7, x28 e x29. Si assuma che l'indirizzo base dei vettori A e B sia contenuto nei registri x10 e x11 rispettivamente. Si assuma che gli elementi dei vettori A e B siano parole su 8 byte:

```
B[8] = A[i] + A[j];
```

2.8 [10] <2.2, 2.3> Tradurre il seguente spezzone di codice assembler RISC-V in C. Si supponga che le variabili f, g, h, i e j siano assegnate rispettivamente ai registri x5, x6, x7, x28 e x29. Si assuma che l'indirizzo base dei vettori A e B sia contenuto nei registri x10 e x11 rispettivamente.

```
addi x30, x10, 8
addi x31, x10, 0
sw   x31, 0(x30)
lw   x30, 0(x30)
add  x5,  x30, x31
```

2.9 [20] <2.2, 2.5> Per ciascuna istruzione RISC-V dell'Esercizio 2.8, definire il valore del codice operativo (OP – opcode) e assegnare un valore ai campi registro sorgente (rs1) e registro destinazione (rd). Per le istruzioni di tipo I, assegnare un valore al contenuto del campo immediato e, per le istruzioni di tipo R, al contenuto del campo del registro del secondo operando (rs2). Per le istruzioni non di tipo U o UJ, mostrare il contenuto del campo funz3, e per le istruzioni di tipo R e S, mostrare il contenuto anche del campo funz7.

2.10 Supponiamo che i registri x5 e x6 contengano i numeri 0x80000000 e 0xD0000000 rispettivamente.

2.10.1 [5] <2.4> Determinare quale sarà il contenuto di x30 dopo l'esecuzione di questa istruzione assembler:

```
add x30, x5, x6
```

2.10.2 [5] <2.4> Il contenuto di x30 è corretto, o si è verificato un overflow?

2.10.3 [5] <2.4> Si supponga che i registri x5 e x6 contengano i valori riportati sopra. Determinare quale sarà il contenuto di x30 dopo l'esecuzione di questa istruzione assembler:

```
sub x30, x5, x6
```

2.10.4 [5] <2.4> Il contenuto di x30 è corretto o si è verificato un overflow?

2.10.5 [5] <2.4> Si supponga che i registri x5 e x6 contengano i valori riportati sopra. Determinare quale sarà il contenuto di x30 dopo l'esecuzione di queste due istruzioni assembler:

```
add x30, x5, x6
add x30, x30, x5
```

2.10.6 [5] <2.4> Il contenuto di x30 è corretto o si è verificato un overflow?

2.11 Si supponga che x5 contenga il valore 128_{dec}.

2.11.1 [5] <2.4> Determinare qual è l'intervallo dei valori di x6 che produrrebbero un overflow quando viene eseguita l'istruzione `add x30, x5, x6`.

2.11.2 [5] <2.4> Determinare qual è l'intervallo dei valori di x6 che produrrebbero un overflow quando viene eseguita l'istruzione `sub x30, x5, x6`.

2.11.3 [5] <2.4> Determinare qual è l'intervallo dei valori di x6 che produrrebbero un overflow quando viene eseguita l'istruzione `sub x30, x6, x5`.

2.12 [5] <2.2, 2.5> Determinare quale istruzione assembler corrisponde alla seguente stringa binaria:

0000 0000 0001 0000 1000 0000 10111 0011_{due}

Suggerimento: la Figura 2.20 può essere utile.

2.13 [5] <2.2, 2.5> Definire il tipo di istruzione e la rappresentazione esadecimale della seguente istruzione assembler:

```
sw x5, 32(x30)
```

2.14 [5] <2.5> Definire il tipo di istruzione, l'istruzione in linguaggio assembler e la rappresentazione binaria dell'istruzione descritta dai seguenti campi RISC-V:

```
codop = 0x33, funz3 = 0x0, funz7 = 0x20, rs2 = 5, rs1 = 7, rd = 6
```

2.15 [5] <2.5> Definire il tipo di istruzione, l'istruzione in linguaggio assembler e la rappresentazione binaria dell'istruzione descritta dai seguenti campi RISC-V:

```
codop = 0x3, funz3 = 0x3, rs1 = 27, rd = 3, imm = 0x4
```

2.16 Si supponga di voler espandere il register file del RISC-V a 128 registri ed espandere l'insieme delle istruzioni in modo che contenga quattro volte il numero di istruzioni.

2.16.1 [5] <2.5> Quali saranno le dimensioni risultanti, in numero di bit, dei diversi campi di un'istruzione di tipo R?

2.16.2 [5] <2.5> Quali saranno le dimensioni risultanti, in numero di bit, dei diversi campi di un'istruzione di tipo I?

2.16.3 [5] <2.5, 2.8, 2.10> In che modo le due modifiche proposte sopra potrebbero consentire di diminuire le dimensioni dei programmi assembler RISC-V? In che modo le modifiche suggerite sopra potrebbero fare aumentare le dimensioni dei programmi assembler RISC-V?

2.17 Si supponga che i registri seguenti contengano i valori:

```
x5 = 0x0000AAAA, x6 = 0x12345678
```

2.17.1 [5] <2.6> Supponendo che i registri `x5` e `x6` contengano i valori riportati sopra, determinare il contenuto di `x7` dopo l'esecuzione delle seguenti istruzioni:

```
slli x7, x5, 4
or   x7, x7, x6
```

2.17.2 [5] <2.6> Supponendo che i registri `x5` e `x6` contengano i valori riportati sopra, determinare il contenuto di `x7` dopo l'esecuzione delle seguenti istruzioni:

```
slli  x7, x6, 4
```

2.17.3 [5] <2.6> Supponendo che i registri `x5` e `x6` contengano i valori riportati sopra, determinare il contenuto di `x7` dopo l'esecuzione delle seguenti istruzioni:

```
srli  x7, x5, 3
andi  x7, x7, 0xFEF
```

2.18 [10] <2.6> Determinare la sequenza più corta di istruzioni RISC-V che consente di estrarre i bit da 11 a 16 dal registro `x5` e li sostituisce ai bit da 26 a 31 del registro `x6` senza modificare gli altri bit del registro `x5` e `x6`. (Assicuratevi di verificare il vostro codice utilizzando `x5 = 0` e `x6 = 0xffffffff`. Questo potrebbe rivelare una svista comune.)

2.19 [5] <2.6> Determinare la sequenza più corta di istruzioni RISC-V che si possono utilizzare per implementare la pseudoistruzione seguente:

```
not x5, x6   // inversione bit a bit
```

2.20 [5] <2.6> Determinare la sequenza più corta di istruzioni RISC-V che si possono utilizzare per tradurre la seguente istruzione C. Si supponga `x6 = A` e che `x17` contenga l'indirizzo di base di C.

```
A = C[0] << 4;
```

2.21 [5] <2.7> Si supponga che `x5` contenga il valore `0x01010000`. Determinare il contenuto di `x6` dopo l'esecuzione delle seguenti istruzioni:

```
        bge x5, x0, ELSE
        jal x0, FINE
ELSE:   ori x6, x0, 2
FINE:
```

2.22 Si supponga che il program counter (PC) sia impostato a `0x20000000`.

2.22.1 [5] <2.10> Qual è l'intervallo degli indirizzi a cui si può saltare utilizzando l'istruzione RISC-V di *jump-and-link* (`jal`)? (In altre parole, qual è l'insieme dei possibili valori che può assumere il PC dopo l'esecuzione dell'istruzione `jal`?).

2.22.2 [5] <2.10> Qual è l'intervallo degli indirizzi a cui si può saltare utilizzando l'istruzione RISC-V di *branch se uguale* (`beq`)? (In altre parole, qual è l'insieme dei possibili valori che può assumere il PC dopo l'esecuzione dell'istruzione `beq`?).

2.23 Si consideri la proposta di una nuova istruzione chiamata `rpt`. Questa istruzione combina in una sola istruzione il controllo di una condizione di fine ciclo e il decremento dell'indice di ciclo. Per esempio `rpt x29, ciclo` avrebbe questo effetto:

```
if (x29 > 0){
        x29 = x29 - 1;
        goto ciclo
    }
```

2.23.1 [5] <2.7, 2.10> Quale sarebbe il formato di istruzione più appropriato per implementare l'istruzione riportata sopra nell'insieme di istruzioni RISC-V?

2.23.2 [5] <2.7> Determinare la sequenza minima di istruzioni RISC-V che consenta di implementare l'istruzione riportata sopra.

2.24 Si consideri il seguente ciclo in assembler RISC-V:

```
CICLO: beq  x6, x0, FINE
       addi x6, x6, -1
       addi x5, x5, 2
       jal  x0, CICLO
FINE:
```

2.24.1 [5] <2.7> Si supponga che il registro x6 venga inizializzato al valore 10. Quale sarà il contenuto finale di x5 supponendo che x5 venga inizializzato a 0?

2.24.2 [5] <2.7> Scrivere la procedura C equivalente al ciclo assembler RISC-V riportato sopra. Si supponga che i registri x5 e x6 contengano rispettivamente le variabili intere acc e i rispettivamente.

2.24.3 [5] <2.7> Si supponga che il registro x6 sia inizializzato con il valore numerico N. Quante istruzioni RISC-V verranno eseguite dal ciclo scritto in linguaggio assembler RISC-V e riportato sopra?

2.24.4 [5] <2.7> Sostituire l'istruzione "beq x6, x0, FINE" con l'istruzione "blt x6, x0, FINE" nel ciclo scritto in linguaggio assembler RISC-V e riportato sopra. Scrivere l'equivalente codice C.

2.25 [10] <2.7> Tradurre il seguente frammento di codice C in codice assembler RISC-V. Si utilizzi il minor numero possibile di istruzioni. Si supponga che le variabili a, b, i e j siano contenute rispettivamente nei registri x5, x6, x7 e x29. Si supponga anche che il registro x10 contenga l'indirizzo base del vettore D.

```
for (i=0; i<a; i++)
    for (j=0; j<b; j++)
         D[4*j] = i+j;
```

2.26 [5] <2.7> Quante istruzioni RISC-V sono necessarie per implementare il frammento di codice C dell'Esercizio 2.25? Supponendo che le variabili a e b vengano inizializzate a 10 e 1, e che tutti gli elementi di D contengano 0 all'inizio del ciclo, quante istruzioni RISC-V verranno eseguite per completare il ciclo?

2.27 [5] <2.7> Tradurre il seguente ciclo in C. Si supponga che il registro x5 contenga la variabile intera i, x6 contenga la variabile intera Ris e x10 l'indirizzo base del vettore MemVett.

```
        addi x6, x6, 0
        addi x29, x0, 100
CICLO:  lw   x7, 0(x10)
        add  x5, x5, x7
        addi x10, x10, 8
        addi x6, x6, 1
        blt  x6, x29, CICLO
```

2.28 [10] <2.7> Riscrivere il ciclo in assembler RISC-V dell'Esercizio 2.27 in modo da ridurre il numero totale di istruzioni RISC-V eseguite. Suggerimento: si noti che la variabile i viene utilizzata solo per il controllo del ciclo.

2.29 [30] <2.8> Tradurre il seguente frammento di codice C in codice assembler RISC-V. Suggerimento: ricordate che lo stack pointer deve rimanere allineato a valori multipli di 16.

```
int fib(int n) {
    if (n==0)
       return 0;
    else if (n==1)
       return 1;
    else
       return(fib(n-1) + fib(n-2);
}
```

2.30 [20] <2.8> Mostrare il contenuto dello stack dopo ciascuna chiamata di funzione dell'Esercizio 2.29. Si supponga che lo stack pointer contenga inizialmente l'indirizzo 0x7ffffffc. Seguire le convenzioni sull'utilizzo dei registri specificate nella Figura 2.11.

2.31 [20] <2.8> Tradurre la funzione f in codice assembler RISC-V. Si supponga che la dichiarazione della funzione funz sia int funz(int a, int b). Il codice della funzione f è il seguente:

```
int f(int a, int b, int c, int d) {
  return funz(funz(a,b),c+d);
}
```

2.32 [5] <2.8> È possibile utilizzare l'ottimizzazione relativa alle chiamate ad anello (*tail call*) in questa funzione? Se la risposta fosse negativa, spiegare perché; se fosse affermativa, calcolare la differenza nel numero di istruzioni eseguite per calcolare f con e senza l'ottimizzazione.

2.33 [5] <2.8> Un attimo prima che la funzione f dell'Esercizio 2.31 ritorni alla funzione chiamante, che cosa si sa sul contenuto dei registri x10-x14, x8, x1 e sp? Si ricordi che conosciamo l'intera funzione f, ma della funzione funz abbiamo solamente la dichiarazione.

2.34 [30] <2.9> Scrivere un programma in linguaggio assembler RISC-V che converta un numero positivo o negativo rappresentato come sequenza di cifre codificate come caratteri ASCII in un numero intero. Il programma dovrà contenere nel registro x10 l'indirizzo della stringa, terminata con il carattere *null*. La stringa conterrà una sequenza di cifre comprese tra 0 e 9 eventualmente preceduta dal segno: "–" o "+". Il programma dovrà determinare il numero associato a questa sequenza di cifre e scrivere questo numero nel registro x10. Se all'interno della stringa comparisse un carattere non numerico, in una qualunque posizione, il programma dovrà terminare e restituire il valore –1 nel registro x10. Per esempio, se il registro x10 puntasse alla sequenza di tre byte

costituita dai numeri 50_{dec}, 52_{dec}, 0_{dec} (corrispondenti alla stringa "24" seguita dal carattere terminatore null), il registro `x10` dovrà contenere il numero 24_{dec} al termine del programma. L'istruzione RISC-V `mul` prende due registri in ingresso. Non esiste l'istruzione "`muli`". Quindi, memorizzare semplicemente la costante 10 in un registro.

2.35 Si consideri il seguente frammento di codice:

```
lb x6, 0(x7)
sw x6, 8(x7)
```

Si supponga che il registro `x7` contenga l'indirizzo `0x10000000` e che l'indirizzo dei dati sia `0x12345678`.

2.35.1 [5] <2.3., 2.9> Quale valore è contenuto all'indirizzo `0x10000008` in una macchina con codifica "big-endian"?

2.35.2 [5] <2.3., 2.9> Quale valore è contenuto all'indirizzo `0x10000008` in una macchina con codifica "little-endian"?

2.36 [5] <2.10> Scrivere il frammento di codice assembler RISC-V che crea la costante su 32 bit: `0x12345678` e la memorizza nel registro `x10`.

2.37 [10] <2.11> Scrivere il codice assembler RISC-V che implementa il seguente frammento di codice C come operazione atomica di "imposta il massimo" utilizzando la coppia di istruzioni `lr.d/sc.d`. In questo caso l'argomento `shvar` contiene l'indirizzo di una variabile condivisa che dovrebbe essere sostituita da `x` se `x` è maggiore del valore a cui punta:

```
void setmax(int *shvar, int x= {
  // Inizia la sezione critica
  if (x > *shvar)
      *shvar = x;
  // Fine della sezione critica
}
```

2.38 [5] <2.11> Utilizzando il codice sviluppato per risolvere l'Esercizio 2.37 come esempio, spiegare cosa succede quando due processori iniziano a eseguire questa sezione critica contemporaneamente, ipotizzando che ciascun processore esegua esattamente una sola istruzione per ciclo di clock.

2.39 Si supponga che per un certo processore, il CPI delle istruzioni aritmetiche sia 1, il CPI delle istruzioni di load/store sia 10 e il CPI delle istruzioni di salto condizionato (branch) sia 3. Si supponga che il programma sia costituito dal seguente numero di istruzioni: 500 milioni di istruzioni aritmetiche, 300 milioni di istruzioni di load/store e 100 milioni di istruzioni di salto condizionato.

2.39.1 [5] <1.6, 2.13> Si supponga che vengano aggiunte nuove istruzioni aritmetiche, più potenti, all'insieme delle istruzioni. Utilizzando queste nuove istruzioni, in media si può ridurre il numero di istruzioni aritmetiche richieste per eseguire un programma del 25%, con un aumento della durata del ciclo di clock del 10%. È una buona scelta di progetto? Perché?

2.39.2 [5] <1.6, 2.13> Si supponga che si trovi un modo per raddoppiare le prestazioni delle istruzioni aritmetiche. Quale sarebbe lo speedup complessivo del calcolatore? Quale sarebbe lo speedup se si migliorassero le prestazioni delle istruzioni aritmetiche di 10 volte?

2.40 Si supponga che per un certo programma il 70% delle istruzioni eseguite siano istruzioni aritmetiche, il 10% sia load/store e il 20% sia salti condizionati.

2.40.1 [5] <1.6, 2.13> Data la distribuzione delle istruzioni riportata in precedenza e assumendo che un'istruzione aritmetica richieda 2 cicli di clock, un'istruzione di lettura/scrittura richieda 6 cicli e un'istruzione di salto condizionato richieda 3 cicli, determinare il CPI medio.

2.40.2 [5] <1.6, 2.13> Quanti cicli di clock deve durare un'istruzione aritmetica, in media, per ottenere un miglioramento delle prestazioni del 25%, se le istruzioni di lettura/scrittura e di salto condizionato non vengono rese più veloci?

2.40.3 [5] <1.6, 2.13> Quanti cicli di clock deve durare un'istruzione aritmetica, in media, per ottenere un miglioramento delle prestazioni del 50%, se le istruzioni di lettura/scrittura e di salto condizionato non vengono velocizzate?

2.41 [10] <2.22> Si supponga che l'ISA del RISC-V comprenda una modalità di indirizzamento mediante offset scalato simile a quella dell'x86 descritta nel Paragrafo 2.19 (Figura 2.39). Descrivere come si potrebbe utilizzare l'offset scalato nelle istruzioni di load per ridurre ulteriormente il numero di istruzioni assembler necessarie per eseguire la funzione dell'Esercizio 2.4.

2.42 [10] <2.22> Si supponga che l'ISA del RISC-V comprenda una modalità di indirizzamento mediante offset scalato simile a quella dell'x86 descritta nel Paragrafo 2.19 (Figura 2.39). Descrivere come si potrebbe utilizzare l'offset scalato nelle istruzioni di load per ridurre ulteriormente il numero di istruzioni assembler necessarie per implementare il codice C dell'Esercizio 2.7.

 © 978-88-08-19966-9

Risposte alle domande di autovalutazione

Paragrafo 2.2, pagina 62 – RISC-V, C, Java.

Paragrafo 2.3, pagina 68 – 2. Molto lento.

Paragrafo 2.4, pagina 74 – I. 2. -8_{dec}, II. 4. $18\,446\,744\,073\,709\,551\,608_{dec}$.

Paragrafo 2.5, pagina 82 – I. 3. `sub x11,x10,x9`, II. 28_{dec}.

Paragrafo 2.6, pagina 84 – Entrambe. L'operazione di AND può essere vista come una maschera di 1, scrive 0 dappertutto tranne che nei campi desiderati. Lo shift a sinistra di un numero di posizioni adeguato rimuove i bit alla sinistra del campo. Lo shift a destra di un numero di posizioni adeguato posiziona il campo nei bit meno significativi della parola e inserisce 0 nel resto della parola. Si noti che AND lascia il campo dov'era originariamente, mentre la coppia di operazioni di shift sposta il campo nei bit meno significativi della parola.

Paragrafo 2.7, pagina 89 – I. Sono tutte vere, II. 1.

Paragrafo 2.8, pagina 98 – Sono entrambe vere.

Paragrafo 2.9, pagina 103 – I. 1., II. 2. 3.

Paragrafo 2.10, pagina 111 – I. 4., II. 4.

Paragrafo 2.11, pagina 114 – Sono entrambe vere.

Paragrafo 2.12, pagina 123 – Indipendenza dal calcolatore.

3 L'aritmetica dei calcolatori

La precisione numerica è la vera anima della scienza.

Sir D'Arcy Wentworth Thompson,
On Growth and Form, 1917

3.1 Introduzione

Le parole dei calcolatori sono composte da bit e possono essere rappresentate come numeri binari. Nel Capitolo 2 abbiamo visto che i numeri interi possono essere rappresentati in forma decimale o binaria; più complessa è la rappresentazione di altri tipi di numeri di uso comune, come nei seguenti esempi.

- Come vengono trattati le frazioni e gli altri numeri reali?
- Che cosa succede se un'operazione genera un numero più grande di quello rappresentabile?
- Dietro a queste domande si nasconde un mistero: come fa l'hardware a eseguire le operazioni di moltiplicazione e divisione sui numeri?

Scopo di questo capitolo è svelare il mistero della rappresentazione dei numeri reali, degli algoritmi che implementano le operazioni, dell'hardware che implementa questi algoritmi e delle implicazioni che tutto questo ha sull'insieme delle istruzioni. I temi trattati possono chiarire anche alcuni dubbi sul funzionamento dei calcolatori. Vi mostreremo inoltre come sfruttare queste conoscenze per rendere molto più veloci i programmi ad alta intensità aritmetica.

3.2 Somme e sottrazioni

La sottrazione: l'amica complicata della somma.

N. 10, Top Ten Courses for Athletes at a Football Factory, David Letterman *et al.*, *Book of Top Ten Lists*, 1990

L'operazione di somma in un calcolatore è svolta in maniera del tutto simile a come la si svolge a mano. Le cifre vengono sommate bit a bit da destra verso